DUBBO 3 · SOURCE CODE READING

Dubbo 3

源码精读 · 一次 RPC 的十一层

不加一行配置地讲清：
@DubboService 怎么变成端口，
一次调用又怎么变成 16 字节头

基于 Apache Dubbo 3.3 · 3900+ Java 文件

目　录

01 序：为什么读 Dubbo 的源码 3

02 一、两条链路速查 3

03 二、11 章一句话速查 3

04 三、症状 → 章节 4

05 四、18 个自测问题 4

06 五、一个提醒 5

07 第1章　全局地图：一次调用经过哪些类 6

08 第2章　SPI：Dubbo 的扩展点体系 11

09 第3章　服务导出：一个 @DubboService 是怎么变成端口的 17

10 第4章　服务引用：一个 @DubboReference 是怎么变成代理的 24

11 第5章　和 Spring 的接缝：@DubboService 是怎么被扫描到的 31

12 第6章　Filter 链：一次调用的洋葱模型 37

13 第7章　集群容错：失败之后怎么办 43

14 第8章　负载均衡：四种算法和它们的代价 49

15 第9章　服务目录与路由：地址从哪来、被谁筛掉 53

16 第10章　网络层：Netty 之上还包了几层 57

17 第11章　Dubbo 协议：16 字节头和两种序列化路径 63

18 结语：一个 RPC 框架到底在解决什么 70

序：为什么读 Dubbo 的源码

会用 Dubbo 和懂 Dubbo 之间，隔着一整层接口。

加个 `@DubboService` 就能发布服务，加个 `@DubboReference` 就能调远程方法，`retries`、`timeout`、`loadbalance` 几个参数配一配就能上线。可一旦出问题——"服务启动了但消费端报 No provider available"、"重试之后订单被创建了两次"、"灰度发布请求还是打到了老版本"、"压测时 `selector.select()` 一直空转"——配置项就不管用了，你得知道这些参数最终落到了哪个类的哪一行。

这本书基于 **Apache Dubbo 3.3 源码**（3900+ 个 Java 文件）写成，不是二手文章的汇编。全书 11 章，按"一条调用链从上往下"的顺序组织：

部分	章节	回答的问题
骨架	1–2	一次调用经过哪些接口？为什么每个可变行为都是 SPI？
配置层	3–5	一个注解怎么变成监听端口和代理对象？Spring 在哪一步接进来？
集群层	6–9	Filter、容错、负载均衡、路由分别在链路的哪一段生效？
通信层	10–11	Netty 之上还包了几层？16 字节头里装了什么？

> 读法建议：第 1 章是地图，务必先读；第 2 章 SPI 是所有后续章节的前置；其余九章可以按需跳读，每章自包含，都有结构表、真实源码片段和"容易误解的地方"。

一、两条链路速查

一次同步调用，消费端把方法调用逐层往下翻译成字节，提供端再逐层往上翻译回来。中间那一层网络是唯一跨越进程的地方。

层	消费端	提供端
业务代码	调用接口方法	执行实现类方法
代理	`ProxyFactory.getProxy` 生成的代理对象	`ProxyFactory.getInvoker` 包出的 `AbstractProxyInvoker`
调用入口	`InvokerInvocationHandler.invoke`	`DubboProtocol` 的 `requestHandler`
Filter	`ClusterFilter` 链 → 实例级 `Filter` 链	`Filter` 链（`ProtocolFilterWrapper` 拼装）
集群	`ClusterInvoker`（Failover 等）	—
目录	`RegistryDirectory` / `RouterChain`	—
选路	`LoadBalance.select`	—
协议	`DubboInvoker` → `ExchangeClient`	`DubboExporter` → `ExchangeServer`
交换层	`HeaderExchangeChannel.request` → `DefaultFuture`	`HeaderExchangeHandler.received`
传输层	`NettyClient` / `NettyChannel`	`NettyServer` / `NettyChannel`
编解码	`DubboCountCodec` → `ExchangeCodec` → `DubboCodec`	同一套，方向相反
序列化	`Hessian2Serialization` 等（URL 参数选定）	同一套

关键分界：**ClusterInvoker 之前是"选谁"，之后是"怎么发"**；`Exchange` 层之上是"一次请求-响应"的语义，之下是"一条字节流"。很多配置不生效，第一件事就是判断它属于哪一侧。

二、11 章一句话速查

章	一句话	什么时候翻它
1 全局地图	一次 RPC 是十几个接口的接力，每个接口只负责一件事	刚接手 Dubbo，想建立整体印象

章	一句话	什么时候翻它
2 SPI	每个可变行为都是一个 `@SPI` 接口，配置只决定取哪个名字	想加自定义实现、或奇怪的配置没生效
3 服务导出	`ServiceConfig` 把注解和配置揉成一个 URL，再让 `Protocol` 把它变成监听端口	服务没注册上、端口没开、导出时机不对
4 服务引用	`ReferenceConfig` 攒出一串 Invoker，套上 Cluster 和代理，交给你一个假对象	`No provider available`、`check`、直连 vs 注册中心
5 Spring 接缝	注解扫成 `ServiceBean`，`Deployer` 统一导出	自动配置没扫到、`@DubboReference` 字段为 null
6 Filter 链	Filter 逆序包成 Invoker 链，逐层递归	自定义 Filter 不生效、想做全链路透传
7 集群容错	`invoke` 定四步模板，`doInvoke` 决定失败后做什么	重试导致重复提交、想换容错策略
8 负载均衡	选一台机器不难，难的是为"选错"付出多少运行时成本	流量不均、新机器预热被打满
9 目录与路由	`Directory` 回答"有哪些地址"，`RouterChain` 回答"这次能用哪些"	灰度/标签路由不生效、地址列表不更新
10 网络层	transport 决定"字节怎么进出"，exchange 决定"回来时还给谁"	超时、请求响应错配、线程模型选择
11 协议	16 字节头只回答四个问题，剩下全交给序列化层	半包解码异常、序列化兼容、要不要换 Triple

三、症状 → 章节

现象	先看
`No provider available` 启动就失败	第4、9章
服务启动了，注册中心里看不到	第3、5章
`@DubboReference` 字段是 null	第5章
自定义 Filter / LoadBalance 没被调用	第2、6章
重试后数据重复	第7章
流量总打到同一台 / 新机器被打满	第8章
灰度、标签路由不生效	第9章
调用超时但服务端其实很快	第10章
半包 / 解码异常 / 序列化不兼容	第11章
想加一种序列化或注册中心	第2章

四、18 个自测问题

骨架

- **Q1** `Invoker`、`Exporter`、`Protocol`、`ProxyFactory` 各自只负责哪一件事?
- **Q2** 和 JDK SPI 比，Dubbo SPI 多做了哪三件事?
- **Q3** `@Adaptive` 为什么要运行时生成字节码，而不是反射调用?
- **Q4** 扩展点加载的三级缓存分别缓存什么?

配置层

- **Q5** 一个 `@DubboService` 最终拼出的 URL 里，参数有哪几个来源?
- **Q6** 本地导出（injvm）和远程导出走的是同一条 `Protocol.export` 吗?
- **Q7** `ReferenceConfig.get()` 返回的对象里持有的是远程连接，还是 Invoker?
- **Q8** `check=false` 把失败从什么时候推迟到什么时候?

- **Q9** `ServiceAnnotationPostProcessor` 和 `ReferenceAnnotationBeanPostProcessor` 分别在 Spring 的哪个扩展点工作?

集群层

- **Q10** 3.x 消费端为什么是两条 Filter 链?分界线在哪?
- **Q11** Filter 链为什么必须逆序包装?
- **Q12** `retries=2` 到底是几次调用?为什么 Failover 只对幂等操作安全?
- **Q13** `AbstractClusterInvoker.invoke` 的四步模板是什么?
- **Q14** 随机负载均衡为什么用 `ThreadLocalRandom`?预热降权在哪里实现?
- **Q15** `RouterChain` 的"至少保留一个"保护逻辑解决什么问题?

通信层

- **Q16** `DefaultFuture` 靠什么把响应还给当初那个线程?
- **Q17** `Dispatcher` 的五种线程模型分别在什么场景用,默认为什么是 all?
- **Q18** Dubbo 协议头里的 flag 字节编码了哪几件事?

五、一个提醒

这本书讲的是 **3.3**,不是 2.7。两者差别不小,网上大部分源码分析文章写的是 2.x,直接套会错:

- `ProtocolFilterWrapper` 已不在拼装 Filter 链的那个方法里,节点变成 `CopyOfFilterChainNode` + `CallbackRegistrationInvoker`
- `ListenableFilter` 已废弃,新写法是 `implements Filter, Filter.Listener`
- 消费端拆成 `ClusterFilter`(负载均衡之前)和实例级 `Filter`(之后)两条链
- 多了 `ApplicationModel` / `ModuleModel` 这套分层,`ServiceConfig` 不再自己决定导出时机,交给 `DefaultModuleDeployer`
- 服务发现从接口级转向应用级,Triple 协议成为主推方向

每章的"**容易误解的地方**"比正文更值得反复看——那里写的都是"以为懂了、其实理解偏了"的位置。

> 判断有没有读懂一个机制,标准只有一个:**能不能说出"如果没有这一层,会出什么具体的错"**。说不出,就还是只记住了流程。

第1章　全局地图：一次调用经过哪些类

一句话：一次 RPC 是十几个接口的接力，每个接口只负责一件事。

为什么需要它：

先看四个真实的问题：

1. 你在 `@DubboReference(timeout = 300)` 上写了 300ms，压测还是卡 3 秒。
2. 你在 `Filter` 里调 `RpcContext.getServiceContext().getRemoteAddress()`，拿到 `null`。
3. 你写了个 `@Activate(group = CONSUMER)` 的 Filter，注册进 `META-INF/dubbo` 了，日志里一次都没打印。
4. 一个"创建订单"接口开了 `retries=2`，超时后重试，数据库里出现两条订单。

四个问题的共同点不是"某个类有 bug"，而是你不知道这个参数、这个对象、这个扩展点**在哪一层被读、被谁覆盖、在哪个时刻被构造**。Dubbo 有 3915 个 java 文件，但一次调用真正经过的类不超过二十个。这一章把这二十个挑出来按顺序摆好，标清每层的接口、典型实现，以及它唯一负责的那件事。

读源码最常见的浪费，是在 `dubbo-common` 里泡三天（那里有 URL、SPI、线程池、字节码），却没搞清 `dubbo-rpc` 里那几个接口的关系。`Invoker`、`Protocol`、`ProxyFactory`、`Filter`、`Exporter`、`Result` 加起来不到三百行代码，但它们定义了整条链的形状。先把骨架立住，再去填肉。

核心结构：

概念	作用
`ProxyFactory`	双向翻译。`getProxy(Invoker)` 把 Invoker 变成业务接口代理（消费端），`getInvoker(proxy, type, url)` 把业务对象包成 Invoker（提供端）。`@SPI("javassist")`，生成失败自动回退 JDK 动态代理
`Invoker<T>`	框架里唯一的"可调用体"。只有 `getInterface()` 和 `invoke(Invocation)` 两个方法，`extends Node` 拿到 `getUrl()` / `isAvailable()` / `destroy()`。链上每一层都是 Invoker 包 Invoker
`Invocation`	一次调用的全部输入：方法名、参数类型、实参、attachment、`InvokeMode`。唯一实现是 `RpcInvocation`
`Filter`	实例级拦截，挂在**单个 provider invoker** 上，不在 Cluster 外层
`ClusterFilter`	3.0 新增，在选机器**之前**拦截，一个服务一条链
`Cluster` / `ClusterInvoker`	失败策略：failover / failfast / failsafe / failback / forking / broadcast / available。**不负责选机器**
`Directory`	provider 列表的来源。`list(invocation)` 时才跑路由，不是缓存快照
`Router` / `RouterChain`	按规则裁剪 provider 列表：tag、condition、script、mesh、state
`LoadBalance`	从候选里选一个：random / roundrobin / leastactive / consistenthash / shortestresponse / adaptiveloadbalance
`Protocol`	`export` / `refer` 的边界，唯一知道"用什么协议通信"的层。`@SPI("dubbo")`
`Exporter`	export 的返回值，持有 Invoker，负责 `unexport` / `register`
`Result`	调用结果。`AsyncRpcResult` 是 `CompletionStage`，`AppResponse` 是它的载荷
`ExchangeClient`	请求/响应语义：一个 `Request` 换一个 `CompletableFuture`
`Codec`	字节 ↔ `Request` / `Response`。dubbo 协议头 16 字节，低 5 位存序列化 ID
`Transporter`	真正的 socket。默认 netty4
`MigrationInvoker`	3.x 特有的一层，同时持有接口级和实例级两个 ClusterInvoker，按迁移规则切换

关键代码：

第一个片段是 Cluster 层的全部工作。它做四件事，但没有一件是"选机器"：

```
// AbstractClusterInvoker.java:345
public Result invoke(final Invocation invocation) throws RpcException {
    checkWhetherDestroyed();

    InvocationProfilerUtils.enterDetailProfiler(invocation, () -> "Router route.");
    List<Invoker<T>> invokers = list(invocation);        // 355: 走 Directory → RouterChain，每次调用都算
    InvocationProfilerUtils.releaseDetailProfiler(invocation);

    checkInvokers(invokers, invocation);                 // 358: 空列表在这里就炸，轮不到 LoadBalance
    LoadBalance loadbalance = initLoadBalance(invokers, invocation);  // 360: 按第一个 invoker 的 url 取 LB
    RpcUtils.attachInvocationIdIfAsync(getUrl(), invocation);         // 361: 只有异步调用才塞 invocationId

    InvocationProfilerUtils.enterDetailProfiler(
            invocation, () -> "Cluster " + this.getClass().getName() + " invoke.");
    try {
        return doInvoke(invocation, invokers, loadbalance);  // 366: 子类决定重试几次、并行几次
    } finally {
        InvocationProfilerUtils.releaseDetailProfiler(invocation);
    }
}

// AbstractClusterInvoker.java:451
protected List<Invoker<T>> list(Invocation invocation) throws RpcException {
    return getDirectory().list(invocation);              // 只有一行，背后是整条 RouterChain
}
```

第二个片段是链路的最末端。同步调用在这里已经变成异步了——`doInvoke` 返回 Future 而不是值：

```
// DubboInvoker.java:121
Request request = new Request();
if (payload != null) {
    request.setPayload(payload);       // 122-124: 单请求载荷上限，超了在提供端拆包
}
request.setData(inv);                  // 125: 载荷就是 RpcInvocation 本身，不是"参数数组"
request.setVersion(Version.getProtocolVersion());

if (isOneway) {
    request.setTwoWay(false);
    currentClient.send(request, isSent);                       // 131: 单向调用，发完就走
    return AsyncRpcResult.newDefaultAsyncResult(invocation);
} else {
    request.setTwoWay(true);
    ExecutorService executor = getCallbackExecutor(getUrl(), inv);   // 135: 回调线程池，不是业务线程池
    CompletableFuture<AppResponse> appResponseFuture =
            currentClient.request(request, timeout, executor).thenApply(AppResponse.class::cast);  // 136-137
    AsyncRpcResult result = new AsyncRpcResult(appResponseFuture, inv);
    result.setExecutor(executor);
    return result;                                              // 144: 同步方法也返回 Future
}

// HeaderExchangeChannel.java:153 —— 请求真正发出去的地方
DefaultFuture future = DefaultFuture.newFuture(channel, req, timeout, executor);  // 153: 先登记 future
try {
    channel.send(req);                                                            // 155: 再发
} catch (RemotingException e) {
    future.cancel();
    throw e;
}
```

先登记再发送。反过来的话，响应可能在登记之前就回来了，`DefaultFuture` 按 request id 找不到对应的 future，这次调用只能等超时。这是所有 RPC 框架的固定写法，也是"偶发超时但服务端日志显示耗时 2ms"的经典成因之一。

拆解：

1.1 链是启动时搭好的，不是调用时拼的

消费端 `ReferenceConfig` 的初始化路径只有一条：

```
ReferenceConfig.java:522   proxyFactory.getProxy(invoker, ProtocolUtils.isGeneric(generic))
```

那个 `invoker` 从哪来？`protocolSPI.refer(type, url)`，此时 url 的协议是 `registry`：

```
Protocol$Adaptive → getExtension("registry")
  → InvokerCountWrapper → ProtocolSerializationWrapper → ProtocolFilterWrapper
  → ProtocolSecurityWrapper → ProtocolListenerWrapper → RegistryProtocol
    RegistryProtocol.java:577  doRefer(...)
    RegistryProtocol.java:592  getMigrationInvoker(...)   // ServiceDiscoveryMigrationInvoker
    RegistryProtocol.java:668  cluster.join(directory, true)
      AbstractCluster.java:58  join → buildClusterInterceptors(doJoin(directory))
      AbstractCluster.java:46  → InvocationInterceptorInvoker(ClusterFilterInvoker(FailoverClusterInvoker))
```

`MigrationInvoker` 这一层是 3.x 独有的，值得单独说一句。它内部有三个字段：`invoker`（接口级，`RegistryDirectory`）、`serviceDiscoveryInvoker`（应用级，`ServiceDiscoveryRegistryDirectory`）、`currentAvailableInvoker`（`MigrationInvoker.java:64-66`）。`invoke`（285）只做一件事——转发给 `currentAvailableInvoker`（296 / 311）。3.0 到 3.2 的注册模型迁移就是靠这一层平滑完成的：两套地址发现同时在跑，按比例切流量，切完把旧的那套 destroy 掉。你在排查"为什么同一个服务有两个 provider 列表"时，答案通常在这里。

所以启动结束时，代理对象背后已经是一棵固定的树：`InvocationInterceptorInvoker → ClusterFilter 链 → FailoverClusterInvoker → MigrationInvoker → Directory`。运行时只做"沿着树往下走"，不再有任何"查找扩展点"的动作。

记住这棵树的形状，最大的收益是**定位问题有了坐标系**。每个节点都实现 `toString`，而且都是"接口名 → url"的格式（`AbstractClusterInvoker.java:381-384`），所以在任意一层打断点、打印 `invoker`，你能直接看出当前在哪一层、url 是哪一份。排查"参数没生效"时，第一件事就是在这几层各打一次 url，看它是在哪一层被换掉的——这比在 3915 个文件里猜快得多。

还有一个容易被忽略的细节：`AbstractProxyFactory.getProxy` 会往代理的接口列表里额外塞两个接口——`EchoService` 和 `Destroyable`（`AbstractProxyFactory.java:45`、`99`）。所以你的业务接口代理天然支持 `$echo()` 和 `$destroy()`，这也是为什么 `InvokerInvocationHandler.invoke` 里要特判 `"$destroy"`（60）。

每个 provider 对应的 `DubboInvoker` 是另一条支线，在 `RegistryDirectory.toInvokers`（440）里由 `protocol.refer(serviceType, url)`（473）创建——**这时候** `ProtocolFilterWrapper.refer`（75）才把消费端 Filter 链套上去。也就是说，消费端 Filter 实例数 = provider 数 × 启用的 Filter 数，不是"一个服务一条"。500 个 provider、8 个 Filter，就是 4000 个 chain node 常驻堆内存。这也是为什么 Dubbo 的 Filter 文档反复强调"每加一个 Filter 都要考虑开销"。

1.2 消费端一次 invoke 的十二跳

按顺序，每一跳都标了类和行号，可以对着断点走：

1. **代理对象** → `InvokerInvocationHandler.invoke`（51）。先短路 `Object` 方法：`toString` / `hashCode` / `equals` / `$destroy`（52-68），否则构造 `RpcInvocation`（69）。构造参数是六元组：`serviceModel`、方法名、接口全名、`protocolServiceKey`、参数类型、实参。注意这里还往 attributes 里塞了 `CONSUMER_MODEL` 和 `METHOD_MODEL`（78-79），后面 Filter 和监控靠它拿元数据，不走反射。
2. `InvocationUtil.invoke`（39）。`RpcContext.storeServiceContext()` 保存现场，`setConsumerUrl`（48），最后 `invoker.invoke(rpcInvocation).recreate()`（102）。**同步阻塞就发生在这一行的 `.recreate()`**，不在 `DubboInvoker` 里。
3. `InvocationInterceptorInvoker` → ClusterFilter 链。节点类型是 `CopyOfClusterFilterChainNode`（`DefaultFilterChainBuilder:112`）。
4. `FailoverClusterInvoker.doInvoke`（59）。`calculateInvokeTimes`（130）= `retries + 1`；每次重试都 `list(invocation)` 重新取列表（73），因为地址可能刚变过。
5. `AbstractClusterInvoker.select`（155）→ `doSelect`（187）。粘滞（sticky）逻辑在这里，`stickyInvoker` 命中就跳过负载均衡（172）。
6. `AbstractDirectory.list`（201）→ `routerChain.getSingleChain`（223）→ `SingleRouterChain.route`。路由是**每次调用**都跑的。
7. `loadbalance.select(invokers, getUrl(), invocation)`（199）。选中的 invoker 如果不可用，会 `invalidateInvoker`（206）再 `reselect`（210）；`reselect` 最多挑 `reselectCount` 个候选，避免 provider 上千时全量遍历。
8. 选中 invoker 的消费端 Filter 链（`CopyOfFilterChainNode`）→ `DubboInvoker.doInvoke`（90）。
9. 多连接轮询：`exchangeClients.get(index.getAndIncrement() % size)`（101）。`connections=0` 时走共享连接（`getSharedClient:481`），否则每个服务独占。
10. `HeaderExchangeClient.request`（107）→ `HeaderExchangeChannel.request`（135）→ `DefaultFuture.newFuture`（153）。

11. `channel.send` → `NettyChannel` → `NettyCodecAdapter.InternalEncoder.encode`（66）→ `codec.encode`（86）→ `DubboCodec.encodeRequestData`（279）。序列化顺序是：协议版本、服务名、服务版本号、方法名、参数类型描述、逐个参数、attachment（283-300）。
12. 响应回来：`DubboCodec.decodeResponse` 完成 future → `AsyncRpcResult.getAppResponse()` 拿到 `AppResponse` → `recreate`（238）按 `InvokeMode` 分三种返回：FUTURE 返回 `RpcContext` 里的 Future，ASYNC 返回默认值（`createDefaultValue`），SYNC 阻塞取值。

这条链上有两处会"吞掉"你的参数，值得记住：

attachment 的传递。`RpcInvocation` 里的 attachment 不是自动跟着 `RpcContext` 走的。`RpcContext.getClientAttachment()` 里的东西要靠 `ConsumerContextFilter` 之类的 Filter 在调用前合并进 `invocation`，到了提供端再由 `ContextFilter` 还原到 `RpcContext.getServerAttachment()`。所以"我 set 了 attachment 但提供端读不到"通常不是序列化问题，是那一段 Filter 没启用或者被 `-context` 排除了。

重试的边界。`FailoverClusterInvoker` 捕获 `RpcException` 后先判断 `e.isBiz()`（104），业务异常直接抛出、不重试；只有框架异常（超时、网络、序列化）才进重试循环。所以第 4 个问题的正解不是关重试，而是让业务异常继承 `RpcException` 的 biz 语义，或者给写操作单独配 `cluster=failfast`。

三种调用模式在这一层分叉。`RpcUtils.isOneway`（105）为真时走 `send` 分支，返回一个已经完成的 `AsyncRpcResult`，永远拿不到异常——单向调用丢包不会报错。同步和异步走同一个 `request` 分支，区别只在最后 `recreate`（238）怎么解释这个 future：FUTURE 模式把 future 还给你，ASYNC 模式返回默认值，SYNC 模式阻塞。也就是说网络层根本不知道你是同步还是异步，这个区分是"消费端怎么读结果"，不是"怎么发请求"。

1.3 提供端：从字节到业务方法

Netty 的 pipeline 在 `NettyServer.initServerBootstrap`（185-188）里固定为四段：

```
negotiation(SslServerTlsHandler) → decoder → encoder → server-idle-handler → handler(NettyServerHandler)
```

1. `NettyCodecAdapter.InternalDecoder.decode`（94）循环调用 `codec.decode`（103）。返回 `NEED_MORE_INPUT` 时把 `readerIndex` 复位再 break（104-106）——半包就等下一批字节，不抛异常。
2. 解出来的是 `DecodeableRpcInvocation`，它同时实现 `Codec` 和 `Decodeable`（`DecodeableRpcInvocation.java:66`）。**解码时机是可控的**：默认在 IO 线程里就地 `inv.decode()`（`DubboCodec.java:200`）；只有在隔离线程池模式（`isDecodeDataInIoThread:231` 返回 false）下才不立即解码，把 `decode()` 推迟到业务线程，由 `DecodeHandler` 触发（`DecodeHandler.java:61`）。所以"参数反序列化占的是 IO 线程还是业务线程"取决于部署模式，不是固定的。
3. `NettyServerHandler.channelRead` → `handler.received`。这个 handler 是 `ChannelHandlers.wrap`（44-47）包出来的：`MultiMessageHandler → HeartbeatHandler → Dispatcher.dispatch(handler, url)`。默认 dispatcher 是 `all`，即"请求派发到业务线程池"；`Dispatcher` 本身也是 SPI，可以换成 `direct`（IO 线程直接跑）、`message`（按请求数派发）等。
4. `HeaderExchangeHandler.received`（196）→ `handleRequest`（88）→ `handler.reply(channel, msg)`（110）。这里把返回值写进 `Response`：成功 `setStatus(Response.OK)` + `setResult`，失败 `SERVICE_ERROR` + `setErrorMessage`（114-119），最后 `channel.send(res)`（120）。
5. `DubboProtocol.requestHandler.reply`（117）。`getInvoker`（131，实现 287）用 `path + version + group + port` 拼 `serviceKey`，去 `exporterMap` 查（310），查不到直接抛 `Not found exported service: ... may be version or group mismatch`，并把 `exporterMap.keySet()` 一起打出来。这个异常信息是全框架里最有用的几条之一。
6. `invoker.invoke(inv)`（167）→ 提供端 Filter 链（`ProtocolFilterWrapper.export:61` 建的，group=provider）→ `AbstractProxyInvoker.invoke`（87）→ `doInvoke`（100）→ `Wrapper.invokeMethod`（89）。最后一步是 javassist 生成的 `Wrapper` 直接调 `proxy.method(args)`，没有反射。
7. 返回值包成 `AppResponse`，`HeaderExchangeHandler` 在 future 完成时回写 `Response`，`DubboCodec.encodeResponseData` 编码。注意返回值的三种形态：`RESPONSE_NULL_VALUE`、`RESPONSE_VALUE`、`RESPONSE_WITH_EXCEPTION`，以及 3.x 加的带 attachment 的变体（`RESPONSE_NULL_VALUE_WITH_ATTACHMENTS` 等，`DubboCodec.java:313` 附近）。

提供端这条链上最反直觉的一点：**`invoker.invoke` 拿到的不是 `DubboInvoker` 也不是 `AbstractProxyInvoker`，而是 Filter 链的最外层**。所以在 `TimeoutFilter` 里 `invoker.invoke` 之后做的事，是在业务方法返回之后、序列化之前。想统计"业务耗时"要卡在这个位置，卡在 `reply` 外面会把序列化也算进去。

异常的回传也在这条链上被改写过。`ExceptionFilter`（`@Activate(group = PROVIDER)`）在 `onResponse` 里检查异常类型：checked 异常、方法签名里声明过的异常、JDK 异常、`RpcException` 都原样放过（65-110）；剩下的"未声明的运行时异常"，会被替换成 `new RuntimeException(StringUtils.toString(exception))`（113）——把原始堆栈压成字符串，这样消费端不需要有这个异常类也能拿到信息。所以"服务端抛的是 `MyBizException`，客户端收到的是 `RuntimeException`"不是 bug，是设计。

1.4 两条链路靠什么对齐

只有一个东西：`serviceKey`。

```
// AbstractProtocol.java:84
protected static String serviceKey(int port, String serviceName, String serviceVersion, String serviceGroup) {
    return ProtocolUtils.serviceKey(port, serviceName, serviceVersion, serviceGroup);
}
```

消费端在 `DubboInvoker.doInvoke` 里把 `path`、`version` 写进 attachment（93-94），提供端在 `DubboProtocol.getInvoker`（308）用同样的四项拼 key。任何一项对不上，就是"提供端明明启动了，消费端说找不到服务"。

`timeout` 也是同理。`DubboInvoker.doInvoke` 里 `RpcUtils.calculateTimeout(getUrl(), invocation, methodName, DEFAULT_TIMEOUT)`（107），读的是 **invoker 的 url**，不是注解上的值。而 invoker 的 url 在 refer 时由 `RegistryDirectory.mergeUrl`（452）把 provider url 和 consumer 参数合并固化。参数优先级大致是：

```
方法级（methods.xxx.timeout） > 接口级（timeout） > 全局默认（DEFAULT_TIMEOUT = 1000）
```

所以"改注解不生效"的典型原因不是优先级算错，而是 `ReferenceConfig` 被缓存复用了——同一个 `interface + group + version` 只 init 一次，第二次改参数不会重新 refer，url 还是旧的。这类问题只能靠 `dump` 出来的 url 确认，看代码看不出来。

那份 url 是怎么拼出来的，`mergeUrl`（600）写得很清楚：先 `ClusterUtils.mergeUrl(providerUrl, queryMap)` 把消费端参数并进 provider url，再 `overrideWithConfigurator` 应用注册中心下发的 override 规则。所以最终那份 url 上，`timeout` 可能来自三个地方——消费端配置、provider 端配置、注册中心的动态覆盖。`getMethodParameter` 只是最后读它的人，不是决定它的人。

容易误解的地方：

1. **消费端 Filter 不在 Proxy 后面第一层。** `ProtocolFilterWrapper.refer`（71）第一句是 `if (UrlUtils.isRegistry(url)) return protocol.refer(type, url);`——registry 协议的 url 直接放行，不建链。真正的链在 `RegistryDirectory.toInvokers`（473）里，套在每个 `DubboInvoker` 上。所以 Filter 的位置是 **LoadBalance 之后、网络之前**。想在选机器之前拦截，只能用 `ClusterFilter`（`AbstractCluster:92` 的 `buildClusterInvokerChain`）。这也是为什么"灰度路由"类需求必须用 Router 或 ClusterFilter，普通 Filter 拿不到"候选列表"。顺带一提：同一个 Filter 类如果 `@Activate(group = {CONSUMER, PROVIDER})`，会在两端各执行一次，中间隔着序列化和网络往返，两边的 `Invocation` 也不是同一个对象。
2. **Cluster 不管选机器。** `FailoverClusterInvoker.doInvoke` 全程只做三件事：算次数、调 `select`、捕异常。选谁在 `LoadBalance`（`AbstractClusterInvoker:199`）。"failover 会换机器重试"的准确表述是：failover 每次循环都重新 `select` 并传入 `invoked` 列表（77-78），由 `doSelect` 的 `isSelected` 判断（202）避开刚失败的那台；如果候选只有一台，它还是会选同一台。
3. **Directory 不是本地缓存。** `AbstractDirectory.list`（201）每次都拿 `validInvokers.clone()`（217）再过 RouterChain，还额外加了读锁（213、224）配合 RouterChain 的双缓冲切换（`RouterChain.getSingleChain:100`）。名字叫 Directory，行为是"查询"。在 Router 里做 IO，会直接打到每一次调用的延迟上。
4. **同步调用内部也是异步的。** `DubboInvoker.doInvoke` 无条件返回 `AsyncRpcResult`（142），阻塞点在 `InvocationUtil` 的 `.recreate()`（102）→ `AsyncRpcResult.getAppResponse()`。这就是为什么 `RpcContext.getClientAttachment().getFuture()` 在同步调用里也能拿到东西——future 一直都在，只是被 `recreate` 消费掉了。

记住这一句：Dubbo 的一次调用，是启动时用 URL 拼好的一棵 Invoker 树，在运行时被从上到下走一遍。

第2章　SPI：Dubbo 的扩展点体系

一句话：每个可变行为都是一个 @SPI 接口，配置只决定取哪个名字。

为什么需要它：

先算一笔账。给 Dubbo 加一种序列化方式，要改多少处?

如果按"工厂 + if/else"的常规写法：改工厂的 switch、改序列化 ID 枚举、改配置项校验、改文档；更麻烦的是每个用到序列化的地方——`ExchangeCodec` 编码、`DecodeableRpcInvocation` 解码、泛化调用、telnet 的 `invoke` 命令、`PermittedSerializationKeeper` 的白名单校验——都得同步加分支。少改一处，就是一个只在特定入口复现的 `SerializationException` 。

Dubbo 的实际做法是两个文件：

```
// 新增一个类
public class Hessian2Serialization implements Serialization { ... }
```

```
# dubbo-serialization-hessian2/src/main/resources/META-INF/dubbo/internal/org.apache.dubbo.common.serialize.Serialization
hessian2=org.apache.dubbo.common.serialize.hessian2.Hessian2Serialization
```

框架代码零改动。全仓 3915 个 java 文件里，这种"接口 + 目录里的 `name=class` "构成了框架全部的可替换面：`Protocol` 、`ProxyFactory` 、`Cluster` 、`LoadBalance` 、`Router` 、`Filter` 、`Serialization` 、`Transporter` 、`Codec` 、`Dispatcher` 、`ThreadPool` 、`Registry` 、`Configurator` 、`Compiler` ……都是同一个机制。

这一章要讲清这套机制的四个部件，以及它们各自的代价：**按名查找**（和 JDK SPI 的第一个差别）、**IoC 注入**、**Wrapper 自动包装**、**Adaptive 运行时生成类**。

核心结构：

概念	作用
`@SPI`	标在接口上，声明这是扩展点。`value()` 是默认扩展名，`scope()` 决定实例共享范围（FRAMEWORK / APPLICATION / MODULE / SELF）
`ExtensionLoader<T>`	一个扩展点一个 loader。读目录、缓存 Class、造实例、注入依赖、套 Wrapper、生成 Adaptive，全在这一个类里
`ExtensionDirector`	loader 的容器，按 Framework → Application → Module 三级持有，子级找不到向父级要
`@Adaptive`	标在方法上：运行时按 URL 参数选实现；标在类上：手写自适应类，跳过生成
`AdaptiveClassCodeGenerator`	拼出 `Xxx$Adaptive` 的 Java 源码，交给 `Compiler` 扩展点编译
`@Activate`	声明"什么条件下自动启用"，给 Filter 这类集合型扩展点用。有 `group` / `value` / `order` / `onClass`
`@Wrapper`	可选，限制某个 wrapper 只包装哪些名字（`matches` / `mismatches` / `order` ）
`@DisableInject`	标在 setter 上，跳过 IoC 注入
`LoadingStrategy`	目录发现策略，本身也是 SPI（用 JDK `ServiceLoader` 加载）
`ExtensionPostProcessor`	实例创建前后的钩子，`postProcessBeforeInitialization` / `postProcessAfterInitialization`
`cachedClasses`	名字 → Class，`Holder<Map>` 持有，一次加载终身使用
`cachedInstances`	名字 → 实例，`ConcurrentMap<String, Holder<Object>>`
`cachedAdaptiveInstance`	自适应实例，进程内一个
`unacceptableExceptions`	加载期就判定"这个名字废了"的集合，命中直接抛，不再尝试

关键代码：

第一段是取实例的主路径。两层缓存、一把细粒度锁，加一个容易忽略的特殊值：

```
// ExtensionLoader.java:557
public T getExtension(String name, boolean wrap) {
    checkDestroyed();
    if (StringUtils.isEmpty(name)) {
        throw new IllegalArgumentException("Extension name == null");
    }
    if ("true".equals(name)) {
        return getDefaultExtension();                // 562: "true" 是"取默认实现"的暗号，不是名字
    }
    String cacheKey = name;
    if (!wrap) {
        cacheKey += "_origin";                       // 565–568: 包装版与裸版分开缓存，避免互相污染
    }
    final Holder<Object> holder = getOrCreateHolder(cacheKey);
    Object instance = holder.get();
    if (instance == null) {
        synchronized (holder) {                      // 570: 锁在 holder 上，不同名字的创建互不阻塞
            instance = holder.get();
            if (instance == null) {
                instance = createExtension(name, wrap);
                holder.set(instance);
            }
        }
    }
    return (T) instance;
}
```

第二段是 `createExtension`。注意执行顺序：**先注入，再包装，最后初始化**。顺序错了会拿到没注入依赖的 wrapper；而且 `initExtension` 是对最外层实例调用的，最外层如果不是 `Lifecycle`，钩子压根不会执行——源码在 811-812 留了这条警告注释：

```
// ExtensionLoader.java:772
private T createExtension(String name, boolean wrap) {
    Class<?> clazz = getExtensionClasses().get(name);          // 773: 只查 cachedClasses，不重复扫目录
    if (clazz == null || unacceptableExceptions.contains(name)) {
        throw findException(name);                             // 775: 带上所有加载期异常，告诉你为什么没找到
    }
    try {
        T instance = (T) extensionInstances.get(clazz);         // 778: 按 Class 缓存的"裸实例"
        if (instance == null) {
            extensionInstances.putIfAbsent(clazz, createExtensionInstance(clazz));
            instance = (T) extensionInstances.get(clazz);
            instance = postProcessBeforeInitialization(instance, name);
            injectExtension(instance);                          // 783: IoC 在这里，注入的是"自适应"依赖
            instance = postProcessAfterInitialization(instance, name);
        }

        if (wrap) {
            List<Class<?>> wrapperClassesList = new ArrayList<>();
            if (cachedWrapperClasses != null) {
                wrapperClassesList.addAll(cachedWrapperClasses);
                wrapperClassesList.sort(WrapperComparator.COMPARATOR);   // 791: 按 order 升序
                Collections.reverse(wrapperClassesList);                 // 792: 反转后逐层包
            }
            for (Class<?> wrapperClass : wrapperClassesList) {
                // 798–801: @Wrapper 的 matches/mismatches 在这里生效
                instance = injectExtension(
                        (T) wrapperClass.getConstructor(type).newInstance(instance));  // 804: wrapper 自己也要注入
            }
        }

        initExtension(instance);                                // 813: Lifecycle.initialize，在最外层实例上
        return instance;
    } catch (Throwable t) { ... }
}
```

拆解：

2.1 和 JDK SPI 的三个差别

差别一：按名取，而且有默认值。

JDK 的 `ServiceLoader` 有三个具体缺陷：一是"全量迭代"，一个接口的所有实现全造出来，你只能拿第一个；二是没有"默认实现"的概念；三是没法只取指定名字的那一个，要么全遍历，要么自己过滤。Dubbo 的目录文件是键值对：

```
# META-INF/dubbo/internal/org.apache.dubbo.rpc.Filter
echo=org.apache.dubbo.rpc.filter.EchoFilter
generic=org.apache.dubbo.rpc.filter.GenericFilter
timeout=org.apache.dubbo.rpc.filter.TimeoutFilter
```

`@SPI` 的 javadoc 专门解释了为什么要从"一行一个全限定名"改成键值对：老格式下如果某个实现的静态字段引用了不存在的第三方库，类初始化失败时框架根本不知道该报哪个名字，只能笼统说"扩展加载失败"。改成键值对后，`name` 在解析阶段就拿到了，异常能精确定位到 `exceptions.put(line, e)`（`loadResource:1176`），最后由 `findException`（747）拼成"possible causes"列表。这就是为什么你看到的错误长这样：

```
No such extension org.apache.dubbo.rpc.Protocol by name dubbo, possible causes:
(1) dubbo=org.apache.dubbo.rpc.protocol.dubbo.DubboProtocol: ... ClassNotFoundException ...
```

差别二：IoC 注入。

`injectExtension`（854）扫描实例的所有 `set*` 方法，`isSetter`（937）的判定是：`public`、单参数、以 `set` 开头。然后逐条过滤：

- 标了 `@DisableInject` 的跳过（867）
- 声明类是 `ScopeModelAware` 的跳过（873）
- 参数是基本类型/包装类型的跳过（883）

剩下的，用 `getSetterProperty`（921）把 `setProtocol` 转成 `protocol`，调 `injector.getInstance(pt, property)`。这个 `injector` 是 `ExtensionInjector` 的自适应实例，而 `SpiExtensionInjector.getInstance`（37）的实现只有一句关键逻辑：

```
if (!loader.getSupportedExtensions().isEmpty()) {
    return loader.getAdaptiveExtension();     // 永远返回自适应实例，不按 name 找
}
```

注入进来的永远是自适应扩展，不是某个具体实现。 这就是为什么 `RegistryProtocol` 里那个 `protected Protocol protocol;`（`RegistryProtocol.java:185`）能用同一个字段既 `refer` 出 dubbo 又 `refer` 出 tri——它拿到的是 `Protocol$Adaptive`，按 url 的协议在调用时分发。也是为什么你在自定义扩展里 `setXxx` 注入进来的东西，`getClass().getName()` 会带一个 `$Adaptive` 后缀。

差别三：Wrapper 自动包装。

只要一个类有 `(T)` 单参构造，`isWrapperClass`（1408）就判它是 wrapper，它不进名字空间，而是进 `cachedWrapperClasses`，被套在所有实现外面。`Protocol` 有五个 wrapper，注册在三个不同的模块里：

```
dubbo-rpc-api/META-INF/dubbo/internal/org.apache.dubbo.rpc.Protocol
  listener=ProtocolListenerWrapper
  serializationwrapper=ProtocolSerializationWrapper
  securitywrapper=ProtocolSecurityWrapper
  invokercount=InvokerCountWrapper
dubbo-cluster/META-INF/dubbo/internal/org.apache.dubbo.rpc.Protocol
  filter=ProtocolFilterWrapper
```

这就是 Dubbo 的 AOP。代价是：wrapper 对**所有**实现生效，`getExtension("dubbo")` 拿到的不是 `DubboProtocol`，而是一个五层嵌套的壳。想 debug 到真正的 `DubboProtocol.export`，得在栈里往下穿五层；想确认某个 wrapper 有没有生效，看栈比看配置快。

2.2 Adaptive：为什么要运行时生成一个类

矛盾很具体：`Protocol.refer(Class, URL)` 在**调用时**才知道用哪个协议，而 IoC 需要先有一个 `Protocol` 实例才能注入。如果 `DubboProtocol` 在启动时被注入进去，那 tri 协议就没法用了。

解决办法是引入一个"延迟决策"的中间人：`Protocol$Adaptive`。它实现 `Protocol`，每个方法内部读 url、算扩展名、`getExtension`、转发。

问题在于：这个中间人如果手写，就要为每个 SPI 接口手写一个分发类。Dubbo 有几十个 SPI 接口，其中带 `@Adaptive` 方法的十几个，手写不可维护。所以：**运行时按接口的方法签名生成这个类**。

生成规则（`AdaptiveClassCodeGenerator`）：

- 类名固定 `接口简单名$Adaptive`（`CODE_CLASS_DECLARATION:49`），包名跟接口一致
- 没有 `@Adaptive` 的方法，直接 `throw new UnsupportedOperationException`（`CODE_UNSUPPORTED:57`）——所以自适应实例只能调用标了 `@Adaptive` 的方法，调别的直接炸
- 找 URL：方法参数里直接有 `URL` 类型就用它（`getUrlTypeIndex:167`）；没有就找参数上返回 `URL` 的 getter（`generateUrlAssignmentIndirectly:382`），优先 `getUrl`
- 空值检查是双重：先查参数本身，再查 `getUrl()` 的返回值（`generateGetUrlNullCheck:421-433`）
- 取名字的 key 来自 `@Adaptive.value()`；为空则把接口名驼峰拆点（`getMethodAdaptiveValue:365`），`Protocol` → `protocol`
- `protocol` 是特例：不读 `url.getParameter("protocol")`，直接读 `url.getProtocol()`（`generateExtNameAssignment:278-288`）
- 最后 `scopeModel.getExtensionLoader(X.class).getExtension(extName)`（`CODE_EXTENSION_ASSIGNMENT:74`）

产物长这样。这份文件在仓库里作为"期望产物"保存，测试拿它跟生成结果逐字比对：

```
// dubbo-common/src/test/resources/org/apache/dubbo/common/extension/adaptive/HasAdaptiveExt$Adaptive
public class HasAdaptiveExt$Adaptive implements org.apache.dubbo.common.extension.adaptive.HasAdaptiveExt {
  public java.lang.String echo(org.apache.dubbo.common.URL arg0, java.lang.String arg1) {
    if (arg0 == null) throw new IllegalArgumentException("url == null");
    org.apache.dubbo.common.URL url = arg0;
    String extName = url.getParameter("has.adaptive.ext", "adaptive");   // 带默认值，来自 @SPI("adaptive")
    if (extName == null) throw new IllegalStateException("Failed to get extension ...");
    ScopeModel scopeModel = ScopeModelUtil.getOrDefault(url.getScopeModel(), ...HasAdaptiveExt.class);
    ...HasAdaptiveExt extension = (...HasAdaptiveExt) scopeModel.getExtensionLoader(...).getExtension(extName);
    return extension.echo(arg0, arg1);
  }
}
```

编译用的是 `Compiler` 扩展点，默认 javassist：

```
// ExtensionLoader.java:1465
String code = new AdaptiveClassCodeGenerator(type, cachedDefaultName).generate();
org.apache.dubbo.common.compiler.Compiler compiler = extensionDirector
        .getExtensionLoader(org.apache.dubbo.common.compiler.Compiler.class)
        .getAdaptiveExtension();
return compiler.compile(type, code, classLoader);     // 1469: ClassLoader 必须是接口的，不能是框架的
```

三个细节值得单独记：

native image 下不生成。 1459-1461 先试 `classLoader.loadClass(type.getName() + "$Adaptive")`，AOT 阶段已经把类编好了，运行时只是加载。

分发发生在每次调用。 不是启动时决定一次。所以同一个 `Protocol$Adaptive` 实例，第一次调用 url 协议是 dubbo 就走 `DubboProtocol`，第二次是 tri 就走 `TripleProtocol`。这是 Dubbo 能同时消费多种协议的根本原因，也是为什么 `getExtension` 的缓存键必须是名字而不是 URL。

生成失败会连坐。 一个接口里只要有一个 `@Adaptive` 方法既没有 URL 参数、也没有返回 URL 的 getter，`generateUrlAssignmentIndirectly` 就抛 `IllegalStateException`（402），整个接口的自适应类都生成不出来。写自定义 SPI 时这条最容易踩。

2.3 Activate：排序与条件

`getActivateExtension(url, key, group)`（343）是 Filter 链的唯一来源。一个扩展要进链，三个条件同时成立：

条件一，group 匹配（`isMatchGroup:460`）。`@Activate(group = PROVIDER)` 只在 group 参数是 `provider` 时启用；调用方传 `null` 则匹配所有。`ProtocolFilterWrapper` 里两次调用分别是 `SERVICE_FILTER_KEY, "provider"`（61）和 `REFERENCE_FILTER_KEY, "consumer"`（75）。

条件二，没被点名排除。 如果 `names` 里出现 `-name`，该扩展不启用（397）；出现 `-default`，整批默认扩展都不启用（351）。这就是配置里 `filter="-monitor"` 的实现。

条件三，URL 条件（`isActive:470`）。`@Activate(value = "cache")` 要求 url 里有 `cache` 参数；`value = "k:v"` 还要求值等于 `v`——`keyPairs` 的解析在 377-386。真实的例子：`ExecuteLimitFilter` 是 `@Activate(group = PROVIDER, value = EXECUTES_KEY)`，不配 `executes` 就根本不加载。

要分清 `value` 和 `order` 是两套独立的过滤：`value` 决定**启不启用**，`order` 只决定**排第几**。写了 `value = "cache"` 但 url 里没有 `cache`，`order` 再小也不会出现——排查"filter 顺序不对"之前，先确认它到底有没有被启用。

排序靠 `TreeMap<Class<?>, T>(activateComparator)`（346）。`ActivateComparator` 的规则是：先比 `@Activate(before/after)`（2.7.0 起废弃），再比 `order`，`order` 相等则比类简单名（117-123）——注释写得很直白，为了避免不同加载顺序下两个 order 相同的 filter 顺序不一致。

真实数值可以拿来校准直觉：`ProfilerServerFilter` 的 `order = Integer.MIN_VALUE`，`ClassLoaderFilter` 是 `-30000`，`ProtocolFilterWrapper` 是 `100`，`ProtocolSecurityWrapper` 和 `ProtocolListenerWrapper` 都是 `200`。

order 小的在外层。 `DefaultFilterChainBuilder.buildInvokerChain`（68）从 `filters.size() - 1` 往 `0` 遍历，每轮用当前累积结果当 `next`，所以 `filters.get(0)` 最终在最外面。Wrapper 那边同理：升序排完再 `reverse`，倒序遍历时第一个（order 最大的）被最先包，落在最内层。

还有一个容易忽略的语法：`names` 里出现 `default` 时，返回顺序按 `names` 的字面顺序排，`default` 那一项展开成整批默认扩展（405-422）。源码注释的原话是 "`ext1,default,ext2` means ext1 will happens before all of the default extensions while ext2 will after them"。所以 `filter="a,default,b"` 的语义不是"a、b、再加默认"，而是"a 在最前，默认那批居中，b 在最后"。这是精确控制 Filter 位置的唯一手段。

`onClass` 是另一套机制，作用在**加载期**而不是启用期：`loadClassIfActive`（1325-1329）要求 `onClass` 里列的类**全部**存在于 classpath，否则这个扩展连名字都不会出现。用来避免 optional 依赖——比如某个 filter 依赖 Sentinel，没引 Sentinel 时它就不该出现在 `getSupportedExtensions()` 里。

2.4 缓存与目录优先级

文档常说"三级缓存"，实际是四层，第四层是 3.x 才加的：

缓存	声明位置	键	说明
`cachedClasses`	125	名字	`Holder<Map<String, Class<?>>>`，`loadExtensionClassesLock` 保护（953-979）
`cachedInstances`	130	名字	`ConcurrentMap<String, Holder<Object>>`，锁在 Holder 上，不同名字并行创建
`cachedAdaptiveInstance`	131	无	单例。失败时异常存进 `createAdaptiveInstanceError`（134）
`extensionInstances`	116	Class	`ConcurrentMap<Class<?>, Object>`，**裸实例**，别名共享

`extensionInstances` 的存在解释了一个常见困惑：配置里写 `a=X, b=X`，`getExtension("a")` 和 `getExtension("b")` 拿到的是**同一个对象**（778 先按 Class 查），但 `cachedInstances` 里是两个 Holder，各自包了不同的 Wrapper 链。所以"两个名字两个实例"只对包装层成立。如果你的扩展是有状态的，别名会让两个名字共享状态——这通常不是你想要的。

`Holder` 本身很简单，一个 `volatile` 字段加 getter/setter（`Holder.java:24-31`）。它的作用不是"原子性"，而是"提供一个可以加锁的对象"——`synchronized (holder)` 锁的是这个 Holder，不同名字的 Holder 不同，所以并发取不同扩展不互斥。

目录优先级由 `LoadingStrategy` 的 `getPriority()` 决定**加载顺序**，由 `overridden()` 决定**能不能覆盖**：

目录	优先级	overridden	撞名时
`META-INF/dubbo/internal/`	`MAX_PRIORITY`（最先加载）	`false`	抛 `IllegalStateException`，并加入 `unacceptableExceptions`
`META-INF/dubbo/`	`NORMAL_PRIORITY`	`true`	覆盖已加载的
`META-INF/services/`	`MIN_PRIORITY`（最后加载）	`true`	覆盖已加载的

注意 `Prioritized.MAX_PRIORITY` 的值是 `Integer.MIN_VALUE`——**数值越小优先级越高**，和 Spring 的 `Ordered` 相反，第一次看容易读反。

覆盖逻辑在 `saveInExtensionClass`（1345）：已有同名且 `overridden` 为真就 `put` 覆盖；`overridden` 为假且不是同一个类，直接抛异常。所以覆盖权是 **services > dubbo > internal**，跟"internal 最优先"的直觉正好相反。internal 目录的设计意图是"框架自己的实现，不许被悄悄替换"，所以它 `overridden()` 返回默认的 `false`——这不是疏忽，是策略。

还有一处副作用：撞名时 `unacceptableExceptions.add(name)`（1352）会把这个名字永久拉黑，之后 `createExtension`（774）看到它就直接抛，哪怕另一个模块后来提供了合法实现。多 ClassLoader 容器里"扩展明明在，就是加载不到"，通常就是这个集合在起作用。

`META-INF/services` 走的是 JDK 格式（一行一个全限定名，没有 key），名字由 `findAnnotationName`（1419）推出来：去掉接口简单名后缀再转小写。三个策略本身也是通过 `META-INF/services/org.apache.dubbo.common.extension.LoadingStrategy` 注册的，`loadLoadingStrategies`（173）用 JDK `ServiceLoader` 加载后按优先级排序——这里恰好是"用 JDK SPI 加载 SPI 加载器"，套了一层。

另外两个性能相关的细节：

- `special_spi.properties`（114、183-188）。某些 SPI 只由框架实现，扫多个 ClassLoader 会让启动慢几百毫秒，所以用这个文件把它们钉死成"只从框架 ClassLoader 加载"（1058-1066）。
- `urlListMapCache`（144-145）。SPI 文件内容按 URL 缓存在 `SoftReference<Map<URL, List<String>>>` 里（1190-1223），避免同一个 jar 里的目录被反复读盘。内存紧张时整块被回收，不会 OOM。

容易误解的地方：

1. **`getAdaptiveExtension()` 的失败是"粘住"的。** `createAdaptiveInstanceError` 一旦被赋值（737），后续每次调用都直接抛同一个异常（724-728），不会重试。本地改了代码、确认编译通过、重启 Spring 上下文都没用——必须重启 JVM。排查"明明改对了还报旧错"时，先怀疑这里。
2. **`@Adaptive` 标在类上会"关掉"生成。** `getAdaptiveExtensionClass`（1447）先看 `cachedAdaptiveClass`，有就直接用，压根不走代码生成。`AdaptiveExtensionInjector`（33）就是这么手写的——它要在多个 `ExtensionInjector` 之间做"第一个返回非 null 的赢"（56-60），这个逻辑没法用模板生成。
3. **`@SPI` 不写 value 就没有默认实现。** `cacheDefaultExtensionName`（1024）只在 `@SPI` 的 `value()` 非空时才给 `cachedDefaultName` 赋值，否则一直是 null，`getDefaultExtension`（598）因此返回 null；生成的自适应类里也退化成 `url.getParameter("x")`，不带默认值（295），取不到就抛 `IllegalStateException`。`Serialization` 就是这种：`@SPI(scope = FRAMEWORK)` 没有 value，所以必须显式配 `serialization=hessian2`，或者由上层 URL 补上这个参数。
4. **Wrapper 不占名字，`getExtension` 也管不到它。** `loadClass`（1283-1287）的分支顺序是：先看 `@Adaptive`，再看是不是 wrapper，都不是才进名字空间。所以注册文件里那行 `filter=...ProtocolFilterWrapper` 的 key 被直接丢弃，`getExtension("filter")` 会抛 "Not find extension"；你只能 `getExtension("dubbo")`，拿到的是被五层 wrapper 包好的实例。想让某个 wrapper 不生效，只有两条路：改代码，或者从注册文件里删掉那一行。

记住这一句：SPI 把"用哪个实现"从编译期搬到了 URL 参数里，Adaptive 就是这次搬运的执行者。

第3章　服务导出：一个 @DubboService 是怎么变成端口的

一句话：ServiceConfig 把注解和配置揉成一个 URL，再让 Protocol 把它变成监听端口。

为什么需要它：你在类上打了 `@DubboService`，Spring 里它只是一个普通 Bean，没有任何网络能力。可服务起来之后，20880 端口上确实有人在 listen，注册中心里也确实多了一条 `dubbo://10.0.0.7:20880/com.foo.DemoService?...`。这中间的转换过程不透明，于是故障也就无从下手：日志明明打印了 `Export dubbo service ... to url`，消费端还是 `No provider available`；端口被别的进程占了，异常栈却指向一个跟端口无关的配置类；同一个 JVM 内自测通过，跨进程调用就超时——那说明只走了 injvm，远程根本没导出。这一章要回答的就是：配置对象在什么时候被补齐、URL 里那几十个参数各自从哪来、本地导出和远程导出差在哪、端口究竟由谁打开、以及为什么重复 export 不会把端口开两次。

排查思路上，可以先把"导出"拆成三个可独立验证的事实：`exported` 是否为 true（配置对象层面）、`Exporter` 是否存在于 `exporters`（协议层面）、`host:port` 是否在 listen（网络层面）。三者依次成立才算真正导出成功，而日志里那行 `Export dubbo service` 只证明走到了 `exportRemote()`，不代表后面两步成功。

核心结构：

概念	作用
`ServiceConfig<T>`	导出入口。持有 `exported` / `unexported` 两个 volatile 标志、`exporters` 按 `RegisterTypeEnum` 分组的 Exporter 列表
`ServiceConfigBase<T>`	持有 `ref`（实现类实例）、`interfaceClass`、`path`、`provider`；`preProcessRefresh()` 在这里补默认 Provider
`AbstractServiceConfig`	持有 `export`、`delay`、`protocols`、`protocolIds`、`version`、`group`、`token`、`weight`
`AbstractInterfaceConfig`	持有 `registries`、`application`、`module`、`monitor`、`local` / `stub`；`completeCompoundConfigs()` 做配置继承
`AbstractConfig`	`refresh()` 模板方法、`appendParameters()` 反射把 getter 写进 Map
`ProviderModel`	一个已导出服务的运行时模型，被挂到 URL 的 `serviceModel` 上
`ProxyFactory`	`getInvoker(ref, interfaceClass, url)` 把实现类包成 provider 侧 Invoker
`Protocol`（adaptive）	`export(Invoker)` 按 URL 协议分派到 `RegistryProtocol` / `DubboProtocol` / `InjvmProtocol`
`Exporter`	导出的句柄，`unexport()` 时用来回收端口与注册信息
`ServiceConfigURL`	真正被拼出来的 URL，带 `scopeModel` / `serviceModel` 两个对象属性

关键代码：

```
// ServiceConfig.java:310
public void export(RegisterTypeEnum registerType) {
    if (this.exported) {                // 快路径：绝大多数重复调用在这一行就返回了，无锁
        return;
    }
    if (getScopeModel().isLifeCycleManagedExternally()) {
        getScopeModel().getDeployer().prepare();   // 生命周期由外部（Spring/Bootstrap）管
    } else {
        getScopeModel().getDeployer().start();     // 兼容旧 API：直接 new ServiceConfig 也能用
    }
    synchronized (this) {
        if (this.exported) {            // 双检：并发 export 只放一个线程进去
            return;
        }
        if (!this.isRefreshed()) {
            this.refresh();             // 补默认值 + 校验，不是"读注解"（注解在构造器里已处理）
        }
        if (this.shouldExport()) {
            this.init();                // 建 ServiceMetadata、加载 ServiceListener
            if (shouldDelay()) {
                doDelayExport();        // delay>0：丢给调度线程，export() 立刻返回
            } else if (Integer.valueOf(-1).equals(getDelay()) && Boolean.parseBoolean(
                    ConfigurationUtils.getProperty(getScopeModel(),
                            CommonConstants.DubboProperty.DUBBO_MANUAL_REGISTER_KEY, "false"))) {
                doExport(RegisterTypeEnum.MANUAL_REGISTER);   // delay=-1 且开了手动注册开关
            } else {
                doExport(registerType);
            }
        }
    }
}

// ServiceConfig.java:544
protected synchronized void doExport(RegisterTypeEnum registerType) {
    if (unexported) {                   // 单向门：unexport 过的配置不能再导出
        throw new IllegalStateException("The service " + interfaceClass.getName() + " has already unexported!");
    }
    if (exported) {
        return;                         // 第三道幂等检查
    }
    if (StringUtils.isEmpty(path)) {
        path = interfaceName;           // URL 的 path 默认就是接口全名
    }
    doExportUrls(registerType);
    exported();                         // 全部 URL 都导出成功之后才置位，失败可重试
}
```

```
// ServiceConfig.java:955
private void doExportUrl(URL url, boolean withMetaData, RegisterTypeEnum registerType) {
    if (!url.getParameter(REGISTER_KEY, true)) {
        registerType = RegisterTypeEnum.MANUAL_REGISTER;   // register="false" 的地址不自动注册
    }
    if (registerType == RegisterTypeEnum.NEVER_REGISTER
            || registerType == RegisterTypeEnum.MANUAL_REGISTER
            || registerType == RegisterTypeEnum.AUTO_REGISTER_BY_DEPLOYER) {
        url = url.addParameter(REGISTER_KEY, false);       // 把"注册意图"写进 URL，交给 RegistryProtocol 判断
    }
    Invoker<?> invoker = proxyFactory.getInvoker(ref, (Class) interfaceClass, url);  // 实现类 → Invoker
    if (withMetaData) {
        invoker = new DelegateProviderMetaDataInvoker(invoker, this);   // 把 ServiceConfig 挂到 Invoker 上
    }
    Exporter<?> exporter = protocolSPI.export(invoker);    // adaptive：按 url.getProtocol() 选实现
    exporters.computeIfAbsent(registerType, k -> new CopyOnWriteArrayList<>()).add(exporter);
}

// ServiceConfig.java:978
private void exportLocal(URL url) {
    URL local = URLBuilder.from(url)
            .setProtocol(LOCAL_PROTOCOL)      // injvm
            .setHost(LOCALHOST_VALUE)         // 127.0.0.1
            .setPort(0)                       // 端口 0：本地导出根本没有端口
            .build();
    local = local.addParameter(EXPORTER_LISTENER_KEY, LOCAL_PROTOCOL);
    doExportUrl(local, false, RegisterTypeEnum.AUTO_REGISTER);   // withMetaData=false：本地不需要元数据
}
```

拆解：

3.1 配置对象是四层继承 + 一个 refresh 模板方法

继承链是 `ServiceConfig` → `ServiceConfigBase` → `AbstractServiceConfig` → `AbstractInterfaceConfig` → `AbstractConfig`。字段分散在这几层，所以"这个属性归谁管"不能靠猜：`export` / `delay` / `protocols` 在 `AbstractServiceConfig`，`ref` / `path` / `provider` 在 `ServiceConfigBase`，`registries` / `application` 在 `AbstractInterfaceConfig`，`parameters` / `id` / `name` 在 `AbstractConfig`。

`AbstractConfig.refresh()` 是模板方法，四个钩子按顺序跑：

```
public void refresh() {
    if (needRefresh) {
        preProcessRefresh();                                // 1. 子类补前置依赖
        refreshWithPrefixes(getPrefixes(), getConfigMode());  // 2. 从 Environment 拉配置覆盖字段
        postProcessRefresh();                               // 3. 子类做校验与派生
    }
    refreshed.set(true);                                    // 4. 无论 needRefresh 与否都置位
}
```

对 `ServiceConfig` 来说，`preProcessRefresh()` 里做三件事：`convertProviderIdToProvider()`（把 `providerIds` 解析成 `ProviderConfig`）、若 `provider == null` 就从 `getModuleConfigManager().getDefaultProvider()` 取，取不到直接抛 `Default provider is not initialized`、最后用 `dubbo.services.{id}` / `dubbo.services.{name}` / `dubbo.service` 三个前缀做一次 `OVERRIDE_IF_ABSENT` 的刷新。第三步用 `ConfigMode.OVERRIDE_IF_ABSENT`，意思是"字段已有值就不覆盖"——这就是配置优先级里 XML/注解 > 配置文件的实现位置。

`checkProtocol()` 也值得看一眼：如果 service 自己既没配 `protocols` 也没配 `protocolIds`（`notHasSelfProtocolProperty()`），就先继承 `provider` 的协议；然后 `convertProtocolIdsToProtocols()` 在 `protocolIds` 为空且 `protocols` 仍为空时，从 `getConfigManager().getDefaultProtocols()` 取全局默认协议，取不到抛 `The default protocol has not been initialized`。所以"一个服务导出到哪些协议"最终由三层决定：service 自己 → provider → 全局 `dubbo.protocol`。`protocols` 是 `List<ProtocolConfig>`，一个服务可以同时导出到 dubbo 和 triple，这就是 `doExportUrls()` 里那个 `for (ProtocolConfig protocolConfig : protocols)` 循环的由来。

`postProcessRefresh()` 被 `ServiceConfig` 覆写成 `checkAndUpdateSubConfigs()`，这才是重活：`completeCompoundConfigs()` 把 `provider` / `module` / `application` 上的 `registries`、`monitor`、`configCenter` 逐级下灌；`ConfigInitializer` 扩展点补空值；非纯 injvm 时 `checkRegistry()`；`Class.forName(interfaceName)`；`checkRef()` 校验 `ref` 非空且确实实现了接口；`ConfigValidationUtils.validateServiceConfig(this)`；最后 `postProcessConfig()` 走 `ConfigPostProcessor` 扩展点。

`refresh()` 的幂等靠 `needRefresh` 和 `refreshed` 两个标志：已经 refresh 过的对象再次进入不会重复反射赋值，但 `refreshed.set(true)` 在 `if` 外面，所以"没做事"也会被标记成"已刷新"。这就是 `ServiceConfig.export()` 里 `if (!this.isRefreshed()) { this.refresh(); }` 能安全重复执行的原因。

`refreshWithPrefixes()` 本身只做一件事：在 `Environment.getConfigurationMaps()` 里按前缀顺序找第一个"有子属性"的前缀，把它下面的所有 key 抽成 `InmemoryConfiguration`，再 `assignProperties()` 反射写回字段。`getPrefixes()` 的顺序就是优先级：`ServiceConfigBase` 把它覆写成只有 `dubbo.service.{interfaceName}`，`AbstractConfig` 的版本是 `dubbo.services.{id}` → `dubbo.services.{name}` → `dubbo.service`。前者用于 `preProcessRefresh()` 里的兜底刷新，后者用于正常刷新——所以 `dubbo.service.com.foo.DemoService.timeout` 会盖掉 `dubbo.service.timeout`。

注解这条路值得单独说清楚。`@DubboService` 并不会直接 new 一个 `ServiceConfig`，Spring 侧生成的是 `ServiceBean<T> extends ServiceConfig<T>`（ServiceBean.java:42）。构造器 `ServiceBean(ApplicationContext, ModuleModel, Service service)` 调 `super(moduleModel, service)`，而 `ServiceConfigBase(ModuleModel, Service)` 的第一件事就是 `appendAnnotation(Service.class, service)`——注解上的十几个属性在这一刻被反射写进字段，远早于 `refresh()`。`ServiceBean.afterPropertiesSet()` 也只做两件事：`setPath(getInterface())` 兜底，以及 `moduleModel.getConfigManager().addService(this)` + `getDeployer().setPending()`。**它不导出。**真正的 `export()` 由 `DefaultModuleDeployer` 启动时的 `exportServices()` 统一触发，`exportServiceInternal()` 里先 `refresh()` 再 `sc.export(RegisterTypeEnum.AUTO_REGISTER_BY_DEPLOYER)`。所以"Bean 初始化完成"和"端口已打开"是两个时刻，中间隔着 Deployer 的一次状态流转。

有一个容易忽略的细节：`ServiceConfig.getDelay()`、`getExport()`、`getGroup()`、`getVersion()` 都做了 provider 兜底，例如 `getDelay()` 是 `(delay == null && provider != null) ? provider.getDelay() : delay`。所以你在 `ServiceConfig` 上读到的值，未必是它自己字段的值。

3.2 URL 是六层 Map 叠加出来的，不是"用户配的"

`doExportUrlsFor1Protocol()` 第一行 `Map<String, String> map = buildAttributes(protocolConfig)`，这个 Map 的写入顺序就是优先级顺序，后写的覆盖先写的：

```
map.put(SIDE_KEY, PROVIDER_SIDE);                  // side=provider
ServiceConfig.appendRuntimeParameters(map);        // dubbo=3.3.x, release=..., timestamp=..., pid=...
AbstractConfig.appendParameters(map, getApplication());   // application=..., qos..., 序列化偏好...
AbstractConfig.appendParameters(map, getModule());
AbstractConfig.appendParameters(map, provider);    // 注意：没有 "default." 前缀，2.x 的 default.xxx 已被去掉
AbstractConfig.appendParameters(map, protocolConfig);
AbstractConfig.appendParameters(map, this);        // service 级，优先级最高
```

之后才追加方法级参数：`appendParametersWithMethod()` 用方法名做前缀写 `sayHello.timeout` 这类 key，`arguments` 再按 `方法名.索引` 写。最后是 `generic` / `revision` / `methods` / `token`：`revision` 由 `Version.getVersion(interfaceClass, version)` 算接口的版本指纹；`methods` 用 `StringUtils.join(new TreeSet<>(Arrays.asList(methods)), ",")` 排序后拼接——排序不是洁癖，URL 的 `equals` 决定注册中心 diff 和 override 比较，方法顺序不稳定会导致"配置没变却一直重新导出"。

`AbstractConfig.appendParameters0()` 的规则值得记住：只看 getter，`@Parameter(excluded = true)` 跳过，值 `null` 或空串跳过，`@Parameter(append = true)` 时与已有值逗号合并并去重。这就是为什么 `protocols` 这种集合字段不会出现在 URL 里，而 `methods` 这种字符串会出现。

host 和 port 的解析在 `findConfiguredHosts()` / `findConfiguredPort()`，优先级是"协议前缀的系统属性 → 协议配置 → provider 配置 → 默认值"。`DUBBO_IP_TO_BIND` 和 `DUBBO_IP_TO_REGISTRY` 是两个独立概念：前者是网卡上 bind 的地址（可能是 `0.0.0.0`），后者是写进注册中心的地址，默认等于前者。端口同理，`findConfiguredPort` 会调 `extensionLoader.getExtension(name).getDefaultPort()` 拿协议默认端口（dubbo 是 20880），小于等于 0 时先查静态 `RANDOM_PORT_MAP`，再 `getAvailablePort(defaultPort)` 并缓存——所以同一进程内多个没配端口的 dubbo 服务会共用同一个随机端口。

`buildAttributes()` 末尾还有一段值得注意：`if (ConfigUtils.isEmpty(token) && provider != null) { token = provider.getToken(); }`，非空时 `ConfigUtils.isDefault(token)` 为 true 就 `map.put(TOKEN_KEY, UUID.randomUUID().toString())`。这是服务端鉴权：provider URL 带 `token=xxx`，注册中心把它暴露给消费端，消费端调用时带上同一个值，`TokenFilter` 比对不一致就拒绝。所以 `token="true"` 的语义不是"开启鉴权"，而是"生成一个随机 token"。

`buildUrl()` 里还有三段容易被跳过的逻辑。第一段是 IPv6：`findConfiguredHosts()` 返回的 host 如果是标准 IPv6 字面量，会被包成 `[::1]` 形式；否则若本机存在 v6 地址，会额外往参数里塞 `IPV6_KEY`，供注册中心侧做双栈展示。第二段是 contextPath：`getContextPath(protocolConfig).map(p -> p + "/" + path).orElse(path)`，所以 URL 的 path 可能是 `api/com.foo.DemoService` 而不是纯接口名，`ServiceConfigBase.getContextPath()` 先看协议配置、再看 provider 配置。第三段是 `ConfiguratorFactory`：如果当前协议有对应的工厂扩展，`url` 会被 `getConfigurator(url).configure(url)` 再加工一次，这是给 rest 之类需要改写 URL 的协议留的口子。加工完之后才 `setScopeModel()` / `setServiceModel()`，顺序不能反——`Configurator` 拿到的 URL 还没有模型。

最后一步 `new ServiceConfigURL(name, null, null, host, port, getContextPath(...).map(p -> p + "/" + path).orElse(path), params)`，`Configurator` 改完之后再 `setScopeModel()` / `setServiceModel(providerModel)`。这两个 setter 不是装饰：URL 是字符串拼出来的，`ServiceModel` 这种对象挂不进去，只能作为 URL 的属性带着走，后面 `DubboProtocol`、监控、异常上报全靠 `url.getServiceModel()` 反查 `ProviderModel`。

3.3 本地导出和远程导出是两条独立的 if，不是二选一

`exportUrl()` 用 `scope` 参数控制，默认 `scope == null`，两个分支都会进：

```
String scope = url.getParameter(SCOPE_KEY);
if (!SCOPE_NONE.equalsIgnoreCase(scope)) {
    if (!SCOPE_REMOTE.equalsIgnoreCase(scope)) {
        exportLocal(url);                // 默认走这里
    }
    if (!SCOPE_LOCAL.equalsIgnoreCase(scope)) {
        url = exportRemote(url, registryURLs, registerType);   // 默认也走这里
    }
}
this.urls.add(url);
```

`exportLocal()` 把协议换成 `injvm`、host 换成 `127.0.0.1`、port 换成 0，再调一次 `doExportUrl(local, false, AUTO_REGISTER)`。它最终落到 `InjvmProtocol.export()`，整个方法体只有一行：`new InjvmExporter<>(invoker, invoker.getUrl().getServiceKey(), exporterMap)`。也就是说 injvm 导出不开端口、不建连接，只是把 Invoker 塞进一个 `Map<String, Exporter<?>>`，key 是 `group/interface:version`。`InjvmProtocol.isInjvmRefer()` 反过来查这个 Map 判断"本机有没有这个服务"——这就是消费端默认优先走本地调用的依据。

`exportLocal()` 是无条件调用的（除非 `scope=remote`），哪怕你只配了 `injvm` 一个协议。`isOnlyInJvm()` 只影响一件事：`checkAndUpdateSubConfigs()` 里 `if (!isOnlyInJvm()) { checkRegistry(); }`——纯本地导出不校验注册中心。这也解释了一个经典现象：单测里 `@DubboReference` 调得通，打包部署后超时。单测时 provider 和 consumer 在同一个 JVM，`shouldJvmRefer()` 查到了 `exporterMap` 里的 exporter，走了 injvm；部署后不在同一进程，才真正走网络，序列化、超时、防火墙的问题一次性暴露。

`exportRemote()` 分两种情况。有注册中心时，对每个 registry URL 调 `doExportUrl(registryURL.putAttribute(EXPORT_KEY, url), true, registerType)`：注意传进去的是 `registry://...` 而不是 `dubbo://...`，真正的 provider URL 被塞进 `EXPORT_KEY` 属性里。这就是 `RegistryProtocol.export()` 里 `getProviderUrl()` 那行 `originInvoker.getUrl().getAttribute(EXPORT_KEY)` 的来历。没有注册中心时直接 `doExportUrl(url, true, registerType)`，adaptive 按 `url.getProtocol()` 选中 `DubboProtocol`。

`proxyFactory.getInvoker(ref, interfaceClass, url)` 返回的是 `AbstractProxyInvoker` 的匿名子类，`doInvoke` 只有一行：默认的 `JavassistProxyFactory` 是 `wrapper.invokeMethod(proxy, methodName, parameterTypes, arguments)`（`Wrapper.getWrapper()` 为每个实现类生成一个方法分发表），`JdkProxyFactory` 则是 `proxy.getClass().getMethod(methodName, parameterTypes).invoke(proxy, arguments)`。所以 provider 侧的 Invoker 非常薄：不做序列化、不做线程池、不做过滤器，只负责"按方法名和参数类型调到实现类"。`invoke()` 外层把返回值 `wrapWithFuture()` 包成 `CompletableFuture`，再用 `AppResponse` 承载结果或异常——异步方法和同步方法在这层被统一成 `CompletableFuture`，这也是 provider 实现类可以返回 `CompletableFuture` 而调用方无感的原因。

端口真正打开的位置在 `DubboProtocol.export()` → `openServer(url)`：

```
String key = url.getAddress();               // host:port 粒度，不是服务粒度
ProtocolServer server = serverMap.get(key);
if (server == null) {
    synchronized (this) {
        server = serverMap.get(key);
        if (server == null) {
            serverMap.put(key, createServer(url));   // Exchangers.bind() 在这里发生
            return;
        }
    }
}
server.reset(url);                           // 已存在则只更新参数（序列化、线程池等）
```

所以"一个服务一个端口"是错的：同一 `host:port` 上的所有服务共享一个 `ProtocolServer`，`DubboExporter` 按 `serviceKey` 进 `exporterMap`，请求进来后按同一个 key 反查 exporter。

这个反查是理解"服务明明导出了却找不到"的关键。`DubboProtocol.export()` 里 `String key = serviceKey(url)`，`serviceKey(URL)` 取 `BIND_PORT_KEY` 而不是 `url.getPort()`，再和 `path` / `version` / `group` 拼成 key，`DubboExporter` 带着这个 key 进 `exporterMap`。请求到达时 `getInvoker(channel, invocation)` 用 `channel.getLocalAddress().getPort()` 加上请求 attachment 里的 `path`、`version`、`group` 拼出同

一个 key 去查。三者任一不一致，就会看到 `Not found exported service` ——端口是通的、服务也确实导出了，只是 key 对不上。改 `path` 或 `group` 之后消费端没同步，就是这类故障。

还有一层：过滤器链不是 `ServiceConfig` 加的。`ProtocolFilterWrapper.export()` 开头就有 `if (UrlUtils.isRegistry(invoker.getUrl())) { return protocol.export(invoker); }` —— `registry://` 这一层直接透传。真正的 `buildInvokerChain(invoker, SERVICE_FILTER_KEY, PROVIDER)` 发生在 `RegistryProtocol.doLocalExport()` 里再以 provider URL 调 `protocol.export(invokerDelegate)` 的时候。换句话说，Filter 链是"本地导出"这一步织进去的，注册中心那一层不参与。

3.4 RegistryProtocol 先导出再注册，延迟与幂等各有各的实现

`RegistryProtocol.export()` 的执行顺序是有讲究的：

```
URL providerUrl = getProviderUrl(originInvoker);            // 从 EXPORT_KEY 取出真正的 provider url
final URL overrideSubscribeUrl = getSubscribedOverrideUrl(providerUrl);
overrideListeners.computeIfAbsent(...).add(overrideSubscribeListener);   // 先登记 override 监听
providerUrl = overrideUrlWithConfig(providerUrl, overrideSubscribeListener);  // 应用已有的配置覆盖
final ExporterChangeableWrapper<T> exporter = doLocalExport(originInvoker, providerUrl);  // ① 开端口
final Registry registry = getRegistry(registryUrl);
final URL registeredProviderUrl = customizeURL(providerUrl, registryUrl);
boolean register = providerUrl.getParameter(REGISTER_KEY, true) && registryUrl.getParameter(REGISTER_KEY, true);
if (register) {
    register(registry, registeredProviderUrl);               // ② 再写注册中心
}
```

①在②之前，保证"注册中心里出现的地址，端口一定已经在监听"。反过来不成立：`register()` 抛异常时，本地端口已经开了，这也是"端口被占用但异常看起来像注册失败"的来源之一。

`protocolSPI.export()` 选中的实现不一定是 `RegistryProtocol` 本身。`dubbo-registry-api` 的 SPI 文件里 `registry` 对应 `InterfaceCompatibleRegistryProtocol`，`service-discovery-registry` 对应 `RegistryProtocol`，前者是接口级兼容实现，后者是应用级实现。`ConfigValidationUtils.genCompatibleRegistries()` 在 provider 侧按 `register-mode` 决定生成哪些 registry URL：接口注册中心默认 `DEFAULT_REGISTER_MODE_ALL`，会额外补一个 `service-discovery-registry://`；服务发现注册中心默认 `DEFAULT_REGISTER_MODE_INSTANCE`，只有显式配 `all` 才补接口级地址。两个 URL 走两个不同的 RegistryProtocol 实现，各自导出一份、注册一份。

`doLocalExport()` 里有两层缓存：`bounds` 是 `Map<providerUrlKey, Map<registryUrlKey, ExporterChangeableWrapper<?>>>`，注释写得很直白——"To solve the problem of RMI repeated exposure port conflicts, the services that have been exposed are no longer exposed"。`computeIfAbsent` 保证同一 provider URL + 同一 registry URL 只导出一次，返回的 `ExporterChangeableWrapper` 支持在配置变更时 `setExporter(protocol.export(...))` 换掉内部 exporter 而不改外部引用。计数由 `ReferenceCountExporter` 维护。

`ExporterChangeableWrapper` 的存在是为了应对运行期配置变更。注册中心的 configurator 推送过来后，`OverrideListener.notify()` 会重算 URL，若 `!newUrl.equals(currentUrl)` 且 `newUrl.getParameter(Constants.NEED_REEXPORT, true)`，就调 `RegistryProtocol.reExport(originInvoker, newUrl)`：`exporter.setExporter(protocol.export(new InvokerDelegate<>(originInvoker, newUrl)))` 换掉内部 exporter，再 `reExportUnregister` 旧地址、`reExportRegister` 新地址。端口没动，`bounds` 里的映射也没动，变的只是 URL 和注册内容——这就是"改个 timeout 不用重启"的底层机制。返回给 `ServiceConfig` 的是 `DestroyableExporter`，一个薄壳，只转发 `getInvoker` / `unexport` / `register` / `unregister`，注释写明"Ensure that a new exporter instance is returned every time export"，防止调用方把 `ExporterChangeableWrapper` 当成可长期持有的对象。

`delay` 的实现很短：`shouldDelay()` 就是 `getDelay() != null && getDelay() > 0`，`doDelayExport()` 把 `doExport(AUTO_REGISTER)` 丢给 `ExecutorRepository.getServiceExportExecutor().schedule(..., getDelay(), MILLISECONDS)`。此时 `export()` 已经返回，`isExported()` 仍是 `false`，`exporters` 还是空的。配套的一行在 `DefaultModuleDeployer.registerServiceInternal()`：`if (sc.shouldDelay()) { return; }` ——延迟导出的服务，deployer 不会替它补注册，注册动作完全由延迟任务内部的 `doExport` 完成。

`delay` 的典型用途不是"延迟启动"，而是"等依赖就绪"。比如服务启动时要连数据库预热本地缓存，如果立刻注册，流量会在预热完成前打进来。把 `delay` 设成略大于预热时间，端口和注册都推迟，消费端在这段时间里看到的是"没有这个 provider"，请求会打到别的实例上。代价是这段时间本机的 `@DubboReference` 也引用不到自己——除非走 injvm，因为 `exportLocal()` 同样在 `doExport` 里，一起被推迟了。

幂等分三层：`exported` 的 volatile 快检、`synchronized` 内的二次检查、`doExport()` 里的第三次检查，加上 `unexported` 的单向门。`exported` 只在 `doExportUrls()` 全部跑完之后由 `exported()` 方法置位，所以中途失败不会留下"半导出但标记已完成"的状态，可以重试。`unexport()` 的顺序也值得看：先对所有 exporter 调 `unregister()`，再 `waitForIdle()` 等最后一次调用结束（默认最多等 `ConfigurationUtils.getServerShutdownTimeout()`），最后才 `unexport()` 真正关端口——先摘流量再关服务。`waitForIdle()` 的算法很细：`idleTime > timeout` 直接返回；`timeout - idleTime < timeout/3` 说明刚被调用过，睡到超时；否则以 `timeout/3` 为粒度轮询 `providerModel.getLastInvokeTime()`，最长不超过 `timeout`。这段代码解决的是"注册中心已经把我摘了，但还有请求在路上"的窗口期问题。

容易误解的地方：

1. **"导出就是打开端口"**。`injvm` 导出完全不碰网络，只是往 Map 里放一个 `InjvmExporter`；`dubbo` 导出的端口是 `ProtocolServer` 级别共享的，一个 `host:port` 上可以挂几十个服务。判断"到底开没开端口"要看 `exporter.getInvoker().getUrl().getProtocol()`，而不是看日志里有没有 `Export dubbo service`。
2. **"URL 是我配出来的"**。用户能控制的只有少数几个 key，`side`、`dubbo`、`release`、`timestamp`、`pid`、`methods`、`revision`、`bind.ip`、`bind.port`、`anyhost` 全是运行时算出来的。排查 URL 问题时不要在配置文件里找 `bind.port`，它来自 `findConfiguredPort()`。
3. **"先注册再导出"**。源码顺序是先 `doLocalExport` 再 `register`。所以注册中心有地址却调不通，方向应该往"端口在但业务逻辑/序列化/线程池有问题"查，而不是怀疑注册。
4. **"3.3 一个服务只注册一次"**。`ConfigValidationUtils.loadRegistries(..., true)` 之后的 `genCompatibleRegistries()` 会按 `register-mode`（接口注册中心默认 `all`）同时产出 `service-discovery-registry://` 和 `registry://` 两个 URL，于是同一个服务会导出两次、注册两次：一次应用级实例，一次接口级地址。

记住这一句：导出 = 把实现类包成 Invoker，把 Invoker 交给 Protocol 换成 Exporter；URL 是这条链上唯一的契约，端口只是 Protocol 的一个副作用。

第4章　服务引用：一个 @DubboReference 是怎么变成代理的

一句话：ReferenceConfig 攒出一串 Invoker，套上 Cluster 和代理，交给你一个假对象。

为什么需要它：`@DubboReference` 标注的字段在 Spring 容器里找不到任何实现类，却能直接调方法；`check = true` 时服务起不来，改成 `check = false` 就能起来，但第一次调用又炸；`lazy = true` 到底省掉了什么；两个配置类引用同一个接口，只改了 `timeout`，为什么超时时间没变。这些问题都指向同一件事：你手里那个"接口实例"到底是什么。这一章拆开 `ReferenceConfig.get()` 到 `ProxyFactory.getProxy()` 的全过程，说明代理里握着的不是连接而是 Invoker、`check` 检查的是哪一刻的什么状态、直连和注册中心在哪一行分叉、`Directory` / `Cluster` / `Invoker` 三层各自负责什么，以及为什么同 `group/interface/version` 的两个引用会共用同一个代理。

理解这条链的收益很直接：调不通时能判断是"没有地址"（Directory 空）、"地址被路由筛掉"（Router）、"选中了但连不上"（Invoker），而不是笼统地说"注册中心有问题"。这三类故障在日志、异常类型、错误码上都不一样。

核心结构：

概念	作用
`ReferenceConfig<T>`	引用入口。持有 `ref`（代理）、`invoker`（ClusterInvoker）、`initialized`、`destroyed` 四个 volatile 标志和一个 `ReentrantLock`
`ReferenceConfigBase<T>`	`shouldCheck()` / `shouldInit()` 的默认值推导；`preProcessRefresh()` 补默认 Consumer
`ConsumerModel`	消费端运行时模型，`consumerModel.setProxyObject(ref)` 后可通过它反查代理
`RegistryProtocol`	`refer()` 返回 `ClusterInvoker`，内部完成注册 consumer、订阅 provider、构建路由链
`ServiceDiscoveryMigrationInvoker`	3.3 默认返回的迁移型 Invoker，按 `MigrationStep` 决定用应用级还是接口级 Directory
`RegistryDirectory` / `ServiceDiscoveryRegistryDirectory`	地址簿。`notify()` 收到推送后把 provider URL 逐个 `refer` 成 Invoker
`Cluster` / `ClusterInvoker`	把 Directory 包成带容错、负载均衡、路由的 Invoker，`failover` 是默认值
`ProxyFactory`	`@SPI("javassist")`，默认 Javassist 字节码代理，`jdk` 是备选
`InvokerInvocationHandler`	代理的 `InvocationHandler`，持有 `Invoker`，把方法调用翻译成 `RpcInvocation`
`ReferenceCache` / `SimpleReferenceCache`	按 `group/interface:version` 缓存代理（老版本叫 `ReferenceConfigCache`）
`DubboInvoker`	真正发包的 Invoker，持有 `ClientsProvider`（内部是 `ExchangeClient` 列表）

关键代码：

```
// ReferenceConfig.java:230
public T get(boolean check) {
    if (destroyed) {                        // 销毁后不可复活
        throw new IllegalStateException("The invoker of ReferenceConfig(" + url + ") has already destroyed!");
    }
    if (ref == null) {                      // 双检第一层：ref 是 volatile
        if (getScopeModel().isLifeCycleManagedExternally()) {
            getScopeModel().getDeployer().prepare();
        } else {
            getScopeModel().getDeployer().start();   // 兼容旧 API 直接 new ReferenceConfig
        }
        init(check);                        // 真正的 refer 在这里
    }
    return ref;
}

// ReferenceConfig.java:332
protected void init(boolean check) {
    lock.lock();
    try {
        if (initialized && ref != null) {  // 锁内二次检查
            return;
        }
        try {
            if (!this.isRefreshed()) {
                this.refresh();
            }
            Map<String, String> referenceParameters = appendConfig();   // side=consumer 等消费端参数
            serviceDescriptor = repository.registerService(interfaceClass);   // nativestub 分支略
            consumerModel = new ConsumerModel(serviceMetadata.getServiceKey(), proxy,
                    serviceDescriptor, getScopeModel(), serviceMetadata, createAsyncMethodInfo(), interfaceClassLoader);
            repository.registerConsumer(consumerModel);                 // 先注册模型，再建 Invoker
            ref = createProxy(referenceParameters);                     // Invoker 链 + 代理都在这里面
            consumerModel.setProxyObject(ref);
            consumerModel.initMethodModels();
            if (check) {
                checkInvokerAvailable(0);      // 只有 check=true 才在 init 末尾校验
            }
        } catch (Throwable t) {
            logAndCleanup(t);                  // 失败要把 invoker/consumerModel/ref 全部清干净
            throw t;
        }
        initialized = true;
    } finally {
        lock.unlock();
    }
}
```

```
// ReferenceConfig.java:490
private T createProxy(Map<String, String> referenceParameters) {
    urls.clear();
    if (StringUtils.isNotEmpty(url)) {
        parseUrl(referenceParameters);                  // 配了 url：直连或多注册中心
    } else {
        aggregateUrlFromRegistry(referenceParameters);  // 没配 url：从 registry 配置生成 registry://
    }
    createInvoker();                                    // 所有 Invoker 的组装
    return (T) proxyFactory.getProxy(invoker, ProtocolUtils.isGeneric(generic));   // 最后一步才是代理
}

// ReferenceConfig.java:669
private void createInvoker() {
    if (urls.size() == 1) {
        URL curUrl = urls.get(0);
        invoker = protocolSPI.refer(interfaceClass, curUrl);      // registry:// → RegistryProtocol.refer
        if (!UrlUtils.isRegistry(curUrl) && !curUrl.getParameter(UNLOAD_CLUSTER_RELATED, false)) {
            invoker = Cluster.getCluster(getScopeModel(), Cluster.DEFAULT)   // 直连要自己补 Cluster
                    .join(new StaticDirectory(curUrl, invokers), true);
        }
    } else { /* 多注册中心：逐个 refer，有 registry 则 ZoneAwareCluster 包一层 */ }
}

// JdkProxyFactory.java:35
public <T> T getProxy(Invoker<T> invoker, Class<?>[] interfaces) {
    return (T) Proxy.newProxyInstance(
            invoker.getInterface().getClassLoader(), interfaces, new InvokerInvocationHandler(invoker));
}

// InvokerInvocationHandler.java:51
public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
    if (method.getDeclaringClass() == Object.class) { return method.invoke(invoker, args); }
    ...
    RpcInvocation rpcInvocation = new RpcInvocation(serviceModel, method.getName(),
            invoker.getInterface().getName(), protocolServiceKey, method.getParameterTypes(), args);
    return InvocationUtil.invoke(invoker, rpcInvocation);   // 代理的活到此结束，剩下全是 Invoker 链
}
```

拆解：

4.1 一次 refer 的时序：模型先建，代理后建

Spring 场景下还有一层包装。`ReferenceBean` 是 `FactoryBean`，`getObject()` 返回的不是 Dubbo 代理，而是一个"懒代理"：`createLazyProxy()` 用 `Proxy.getProxy(interfaces).newInstance(new LazyTargetInvocationHandler(new DubboReferenceLazyInitTargetSource()))` 生成（Javassist 失败则回退 JDK 代理），handler 里 `target` 为 null，第一次真正调方法时才 `target = lazyTargetSource.getTarget()` → `getCallProxy()` → `referenceConfig.get()`。源码注释把动机写得很清楚：Spring 按类型查找 Bean 时可能提前初始化 FactoryBean，而那时 Dubbo 配置 Bean 还没就绪。所以"字段注入完成"不等于"refer 完成"，被注入的那个对象在被调用之前是个空壳。

`init()` 的顺序不能乱：先 `refresh()` 补默认值，再 `appendConfig()` 生成消费端参数 Map（`side=consumer`、`interface`、`methods`、`revision`、`register.ip` 都在这里，注意 `register.ip` 是消费端自己的地址，用来在注册中心登记 consumer），再 `registerService(interfaceClass)` 拿到 `ServiceDescriptor`，再 `new ConsumerModel(...)` 并 `registerConsumer()`，然后才 `createProxy()`。

先建模型的好处是：`createProxy()` 内部构造 URL 时可以直接 `url.setServiceModel(consumerModel)`，`InvokerInvocationHandler` 的构造器又能从 `invoker.getUrl().getServiceModel()` 里拿到 `ConsumerModel` 和 `protocolServiceKey`。模型是 URL 上的对象属性，URL 是字符串拼的，这条路是对象在链路上传递的唯一通道。

`ConsumerModel` 里还装了两样东西：`createAsyncMethodInfo()` 从每个 `MethodConfig.convertMethodConfig2AsyncInfo()` 得到的 `Map<String, AsyncMethodInfo>`，以及 `initMethodModels()` 建出来的方法级模型。它们不只是统计用的——`InvokerInvocationHandler` 在 `serviceModel instanceof ConsumerModel` 时会把 `CONSUMER_MODEL` 和 `METHOD_MODEL` 塞进 `RpcInvocation` 的 attachment，下游的 metrics、异步回调、方法级超时都靠这两个 key 找到对应模型。

异常路径同样重要：任何一步抛出，都走 `logAndCleanup(t)` —— `invoker.destroy()`、`repository.unregisterConsumer(consumerModel)`、把 `initialized` / `invoker` / `ref` / `consumerModel` 全部置空。这就是为什么 `check=true` 启动失败之后，同一个 `ReferenceConfig` 对象在依赖重新可

用时还能再 `get()` 一次，而不是永久卡在坏状态。

`createProxy()` 里还有一个 `meshModeHandleUrl()`，在 Service Mesh 模式下会把 `url` 改写成 `triple://{providedBy}.{namespace}.svc.cluster.local:80`，并强制 `unloadClusterRelated=false`。这是 3.3 里 Proxyless 场景的入口，普通项目不会走到。

4.2 check 检查的是"现在有没有可用 Invoker"，不是"能不能连上"

`shouldCheck()` 的推导链是：`isCheck()` → `consumer.isCheck()` → 默认 `true`。`checkInvokerAvailable(timeout)` 的逻辑很朴素：

```
if (!shouldCheck()) { return; }
boolean available = invoker.isAvailable();
if (available) { return; }
long checkDeadline = System.currentTimeMillis() + timeout;
do {
    Thread.sleep(100);
    available = invoker.isAvailable();
} while (!available && checkDeadline > System.currentTimeMillis());
if (!available) { throw new IllegalStateException("Failed to check the status of the service ..."); }
```

注意 `init()` 里传的是 `0`，所以 `checkDeadline == startTime`，循环体仍然会执行一次 `sleep(100)` —— `check=true` 至少容忍 100ms 的地址推送延迟。

抛出的异常类型是 `IllegalStateException`，不是 `RpcException`，消息里固定包含 `No provider available for the service`。`logAndCleanup()` 里专门用 `t.getMessage().contains("No provider available for the service")` 匹配它，打 `CLUSTER_NO_VALID_PROVIDER` 错误码。所以启动日志里那条 `No provider available.` 的 ERROR，是 `ReferenceConfig` 打的，不是 Cluster 打的。

这条异常消息本身值得完整读一遍，它把排查要用的信息都带上了：`interfaceName`、`group/version`（拼成 `group/interface:version`）、`invoker.getUrl()`（注册中心地址）、消费端本机 IP、Dubbo 版本。缺的只有"注册中心里到底有没有地址"，那要看 Directory 的订阅日志。所以看到它的第一反应不该是重启，而是拿 `invoker.getUrl()` 去查这个 service key 的 provider 列表——列表为空是 provider 没注册，列表非空就是本地订阅/路由出了问题。

Spring 场景下启动期的 check 其实不在 `get()` 里。`DefaultModuleDeployer` 的顺序是：`referServices()` 用 `referenceCache.get(rc, false)`（不 check，先把所有引用建起来），全部建完再 `checkReferences()` 用 `referenceCache.check(rc, timeout)` 统一等，`timeout` 取 `ModuleConfig.getCheckReferenceTimeout()`，缺省 30000ms。这个设计是为了让"所有引用并行建立、统一等待"，而不是每个引用各自阻塞。它还有一个副作用：`check=true` 的失败发生在 `checkReferences()` 阶段，此时所有 `ReferenceConfig` 的 `ref` 都已经建好，失败的那个被 `checkOrDestroy(timeout)` → `logAndCleanup()` 清掉，其它引用不受影响。这个 timeout 是"等地址推送"的总预算，不是单次连接超时。

`check=false` 时启动不报，但调用期照报：`AbstractClusterInvoker.checkInvokers()` 在 `invokers` 为空时抛 `RpcException(NO_INVOKER_AVAILABLE_AFTER_FILTER, "Failed to invoke the method ... No provider available for the service ... Please check if the providers have been started and registered.")`。所以 `check` 只决定"启动期是否失败"，不决定"调用期是否失败"。

实践上 `check` 是个部署期开关：provider 和 consumer 一起发布时通常设 `check=false`，避免消费端比提供端先起来导致整个应用起不来；代价是失败被推迟到第一次调用，异常类型从 `IllegalStateException` 变成 `RpcException`，日志位置也从启动阶段挪到了业务线程里。

`lazy` 是另一个维度。它不进 `ReferenceConfig` 的任何分支，只作为 URL 参数 `lazy=true` 传到 `DubboProtocol.initClient()`：

```
return url.getParameter(LAZY_CONNECT_KEY, false)
        ? new LazyConnectExchangeClient(url, requestHandler)
        : Exchangers.connect(url, requestHandler);
```

`LAZY_CONNECT_KEY` 的值就是 `"lazy"`。所以 `lazy=true` 的含义是"refer 照做，但先不建 TCP 连接，首次 invoke 再连"。更坑的是 `LazyConnectExchangeClient.isConnected()` 在 `client == null` 时直接 `return true`（注释说这是为了防止 `ReconnectTask` 误重连），于是 `DubboInvoker.isAvailable()` 会返回 true—— `check=true` + `lazy=true` 组合下，provider 明明没起来，启动检查也能通过。所以它只适合能容忍首调延迟的场景，不能拿来规避启动检查。

真正"启动时不 refer"的是 `init`：`shouldInit()` 为 false 时，`referServices()` 里的 `if (rc.shouldInit())` 直接跳过，代理要等到第一次被注入/调用时才 `get()`。

4.3 直连和注册中心在第 495 行分叉，`refer` 在末端收口

`createProxy()` 的第一个判断是 `StringUtils.isNotEmpty(url)`，这里的 `url` 是 `ReferenceConfigBase.url` 字段，也就是 `@DubboReference(url = "...")`。

走直连时 `parseUrl()` 用分号切分多个地址，每个地址 `URL.valueOf(u)`，path 为空则 `setPath(interfaceName)`，然后看 `UrlUtils.isRegistry(url)`：是注册中心地址就 `putAttribute(REFER_KEY, referenceParameters)`；不是就 `mergeUrl(url, referenceParameters)` 并 `putAttribute(PEER_KEY, true)`。`mergeUrl` 把消费端参数合并进 provider URL——这就是直连时 `timeout`、`retries` 仍然生效的原因。`url` 里可以混写，比如 `dubbo://a:20880;zookeeper://b:2181`，分号分隔的每一项各自判断是不是注册中心，所以"一半直连一半注册"在语法上是合法的。直连省掉注册中心一跳，适合灰度和跨环境调试；代价是没有地址列表，provider 挂了只能靠 `failover` 对同一台重试。

走注册中心时 `aggregateUrlFromRegistry()`：`checkRegistry()` 校验注册中心配置有效，`ConfigValidationUtils.loadRegistries(this, false)` 生成 `registry://` URL（`provider=false` 时只取 `subscribe=true` 的），挂上 monitor URL，`putAttribute(REFER_KEY, referenceParameters)`。如果 `urls` 还是空且 `shouldJvmRefer()` 为 true，补一个 `injvm://127.0.0.1:0/{interface}`；仍然为空就抛 `No such any registry to reference ... please config <dubbo:registry address="..."/>`。

这里有个设计要点：消费端参数不是拼进 URL 字符串的，而是作为 `REFER_KEY` 属性的一个 `Map` 挂在 registry URL 上。原因是这些参数是"我要怎么调"，不是"provider 是什么"，混进 URL 字符串会被注册中心当成 provider 元数据。`RegistryProtocol.refer()` 里那句 `Map<String, String> qs = (Map<String, String>) url.getAttribute(REFER_KEY)` 就是把它取回来。

`shouldJvmRefer()` 的四步判断写在方法注释里：`injvm` 显式配置优先；配了 `url` 就认为是远程；否则看 `scope`；都没配时用 `InjvmProtocol.getInjvmProtocol(scopeModel).isInjvmRefer(tmpUrl)` 探测本机是否真的导出过这个服务。`isInjvmRefer()` 内部先看 `scope=local` / `injvm=true`，再看 `scope=remote`，再看 `generic`（泛化调用不算本地），最后 `getExporter(exporterMap, url) != null` 才返回 true；`cluster=broadcast` 时强制返回 false——广播调用要打到所有机器，不能被本地短路。

`createInvoker()` 的分支值得逐条读：

- `urls.size() == 1` 且是 registry：`protocolSPI.refer()` 直接返回 `ClusterInvoker`，因为 `RegistryProtocol` 内部已经 `cluster.join(directory)` 过了，外面不能再包一层。
- `urls.size() == 1` 且不是 registry（直连）：`protocolSPI.refer()` 返回的是裸的 `DubboInvoker`，必须自己 `Cluster.getCluster(..., Cluster.DEFAULT).join(new StaticDirectory(curUrl, invokers), true)` 包一层，否则没有容错和负载均衡。
- 多个 URL：每个都 refer，如果有 registry URL 就用 `ZoneAwareCluster`（多注册中心场景默认按 zone 优先），否则用默认 `failover`。

`Cluster.DEFAULT` 就是 `"failover"`，`Cluster.getCluster()` 内部走 `ExtensionLoader.getExtension(name, wrap)`，`wrap=true` 时会把 `AbstractCluster` 那层 wrapper 也套上。

多注册中心这条分支还有两个细节。`Cluster.getCluster(..., cluster, false)` 传了 `wrap=false`——因为外层 `StaticDirectory` 里装的是"每个注册中心一个 ClusterInvoker"，再套通用 wrapper 没有意义。`ZoneAwareClusterInvoker.doInvoke()` 的优先级是：`preferred=true` 的 registry → 请求 attachment 里的 `registry.zone`（或 `ZoneDetector` 探测出的 zone）匹配的 registry → 按 registry 权重做一次负载均衡 → 第一个可用的 → 都没有则抛 `No provider available in ...`。

无论走哪条路，最后都落到 `protocolSPI.refer()`，也就是 `RegistryProtocol.refer()`（或它的接口级兼容子类）。

`refer()` 先把 registry URL 规范化（`getRegistryUrl(url)` 会把 `registry=` 参数记下、协议改成 `service-discovery-registry`），`getRegistry()` 拿到 `Registry` 实例。`RegistryService.class.equals(type)` 是特判：引用注册中心本身时，直接把 `Registry` 对象包成 Invoker 返回，不走 Directory。

group 多值或 `*` 时用 `MergeableCluster`，否则 `Cluster.getCluster(url.getScopeModel(), qs.get(CLUSTER_KEY))`。然后 `doRefer()`：用 `REFER_KEY` 里的参数构造 `consumerUrl`（protocol 取 `consumer`、host 取 `register.ip`、port 固定 0），`putAttribute(CONSUMER_URL_KEY, consumerUrl)`，再 `new ServiceDiscoveryMigrationInvoker<>(...)`，最后 `interceptInvoker()` 交给 `RegistryProtocolListener` 扩展点。

关键点在 3.3：`refer()` 返回的不是 `RegistryDirectory`，而是一个还没建 Directory 的 `MigrationInvoker`。真正的 Directory 由 `MigrationRuleListener.onRefer()` → `MigrationRuleHandler.doMigrate()` 触发；因为返回的是 `ServiceDiscoveryMigrationInvoker`，`doMigrate` 一开始就 `refreshInvoker(MigrationStep.FORCE_APPLICATION, 1.0f, rule)` 并 return，于是走 `migrateToForceApplicationInvoker` → `refreshServiceDiscoveryInvoker` → `registryProtocol.getServiceDiscoveryInvoker()` → `new ServiceDiscoveryRegistryDirectory<>(type, url)`。

`RegistryDirectory`（接口级）只由 `RegistryProtocol.getInvoker(cluster, registry, type, url)` 创建，而这个方法上面挂着源码注释 `// FIXME, this method is currently not used, create the right registry before enable.`。也就是说 3.3 默认链路里跑的是应用级 Directory，`RegistryDirectory` 是接口级兼容和迁移路径的产物。读源码时不要把两者混为一谈。

两个 Directory 共用 `doCreateInvoker()`：

```
directory.setRegistry(registry);
directory.setProtocol(protocol);
Map<String, String> parameters = new HashMap<>(directory.getConsumerUrl().getParameters());
URL urlToRegistry = new ServiceConfigURL(parameters.get(PROTOCOL_KEY) == null ? CONSUMER : parameters.get(PROTOCOL_KEY),
        parameters.remove(REGISTER_IP_KEY), 0, getPath(parameters, type), parameters);
if (directory.isShouldRegister()) {
    directory.setRegisteredConsumerUrl(urlToRegistry);
    registry.register(directory.getRegisteredConsumerUrl());   // 消费端也往注册中心写一条 consumer://
}
directory.buildRouterChain(urlToRegistry);                     // 构建路由链（标签路由、条件路由）
directory.subscribe(toSubscribeUrl(urlToRegistry));            // category=providers,configurators,routers
return (ClusterInvoker<T>) cluster.join(directory, true);
```

`RegistryDirectory.notify()` 收到推送后按 category 分组，`providers` 走 `refreshInvoker()` → `toInvokers()`，逐个 `protocol.refer()` 生成 `DubboInvoker`（带 `Map<URL, Invoker>` 缓存，URL 没变就复用），最后 `setInvokers()`。地址变化靠注册中心推送，不靠轮询。

`toSubscribeUrl()` 只是加一个 `category=ALL_CATEGORIES`，一次订阅同时拿到 providers、configurators、routers 三类数据。`directory.buildRouterChain(urlToRegistry)` 建的是路由链（标签路由、条件路由、脚本路由），它在 `list()` 时对地址做过滤，所以 `No provider available after route` 和 `No provider available after connectivity filter` 是两个不同的失败点：前者是路由把地址筛没了，后者是筛完发现连不上。`registry.register(consumerUrl)` 这一步也容易被忽略——消费端会往注册中心写一条 `consumer://` 记录，治理台靠它显示"谁在调我"，配置覆盖也靠它匹配。

应用级订阅比接口级多一步"接口到应用名"的映射，这是 3.3 里 `No provider available` 的另一个高发原因。`ServiceDiscoveryRegistry.doSubscribe()` 先 `serviceDiscovery.subscribe(url, listener)`，再 `ServiceNameMapping.getMappingByUrl(url)` 拿这个接口属于哪些应用；拿不到就 `serviceNameMapping.getAndListen(...)` 注册一个 `DefaultMappingListener` 等推送，并直接 `return`，日志是 `No interface-apps mapping found in local cache, stop subscribing, will automatically wait for mapping listener callback`。provider 侧如果元数据中心不可用、`ServiceNameMapping.map()` 失败，消费端就会一直等这个回调——端口通、provider 也活着，但订阅列表始终是空的。排查这类问题时，要先确认元数据中心的接口-应用映射有没有写成功，而不是先怀疑网络。

4.4 代理里没有连接，只有一条 Invoker 链

`JdkProxyFactory.getProxy()` 就是 `Proxy.newProxyInstance(classLoader, interfaces, new InvokerInvocationHandler(invoker))`。默认实现是 `javassist`（`ProxyFactory` 上 `@SPI(value = "javassist")`），生成字节码而不是反射调用，但结构完全一样：代理 → `InvocationHandler` → `Invoker`。

`InvokerInvocationHandler` 只持有三样东西：`Invoker<?> invoker`、`ServiceModel serviceModel`、`String protocolServiceKey`。`invoke()` 里先拦 `Object` 的方法（`toString` / `hashCode` / `equals` 转发给 invoker，`$destroy` 调 `invoker.destroy()`），其余全部打包成 `RpcInvocation` 后 `InvocationUtil.invoke(invoker, rpcInvocation)`。

`InvocationUtil` 这一层做三件不显眼但重要的事：`rpcInvocation.setTargetServiceUniqueName(url.getServiceKey())` 把目标服务固定下来（后面重试、路由都读它）；`RpcServiceContext.getServiceContext().setConsumerUrl(url)` 把消费端 URL 放进上下文（Filter 里 `RpcContext.getServiceContext().getConsumerUrl()` 就来自这里）；最后 `invoker.invoke(rpcInvocation).recreate()` —— `recreate()` 把 `AppResponse` 里的 `CompletableFuture` 展开成真正的返回值，异步转同步的最后一环在这里。

所以调用栈是：代理 → `InvokerInvocationHandler` → `ClusterInvoker`（`MigrationInvoker` / `FailoverClusterInvoker`）→ `Directory`（地址列表 + 路由）→ `DubboInvoker`（`ClientsProvider` → `ExchangeClient`）→ TCP。代理里没有任何连接、地址、序列化器，这些都在链的末端。这也解释了为什么改 `timeout` 要重建代理：`timeout` 在 URL 里，URL 在 `DubboInvoker` 里，而代理是链的入口，链是按 URL 建的。

消费端的过滤器分两层织入。`ClusterFilter` 在 Cluster 层生效，例如 `FutureFilter`（`@Activate(group = CONSUMER)`）包在 `ClusterInvoker` 的调用链上；普通 `Filter` 在协议层生效，`ProtocolFilterWrapper.refer()` 对非 registry URL 执行 `builder.buildInvokerChain(protocol.refer(type, url), REFERENCE_FILTER_KEY, CONSUMER)`，包在 `DubboInvoker` 外面。所以 `InvokerInvocationHandler` 一次调用要依次穿过 ClusterFilter、ClusterInvoker、Directory、协议层 Filter，最后才到 `DubboInvoker` 发包。

`AbstractProxyFactory.getProxy(invoker, generic)` 还会往 interfaces 里加两个内部接口：`EchoService` 和 `Destroyable`，所以任何 Dubbo 代理都能 `$echo()` 和 `$destroy()`。

泛化调用走的是同一个 handler。`generic=true` 时 `AbstractProxyFactory` 会把真实接口类和 `GenericService` 一起加进 interfaces，代理上于是既有业务方法也有 `$invoke(String, String[], Object[])`。`$invoke` 被 `InvokerInvocationHandler` 打包成 method 名为 `$invoke` 的 `RpcInvocation`（三个参数分别是方法名、参数类型数组、参数值数组），到 provider 侧由 `GenericFilter` 拦下来：`inv.getMethodName().equals($INVOKE)` 成立时，用 `findMethodByMethodSignature(interface, name, types, serviceModel)` 反查真实 `Method`，再 `PojoUtils.realize(args, params, genericParameterTypes)` 把 Map 还原成 POJO。所以泛化调用不是另一条链路，只是同一条链路上多了两次"方法名到 Method"的解析。

代理本身也不便宜，所以 Dubbo 在它外面又加了一层缓存。3.3 里叫 `ReferenceCache` / `SimpleReferenceCache`，2.x 叫 `ReferenceConfigCache`。`DEFAULT_KEY_GENERATOR` 是：

```
return BaseServiceMetadata.buildServiceKey(iName, referenceConfig.getGroup(), referenceConfig.getVersion());
```

也就是 `group/interface:version`。`get(rc, check)` 在 `singleton` 为 true 时先 `get(key, type)`，命中就直接返回已有代理——**新 `ReferenceConfig` 上的 `timeout`、`retries`、`check` 全部不生效**，因为它们属于那个被丢弃的配置对象。没命中才 `rc.get(check)`，并把 `rc` 记进 `referenceKeyMap` 和 `referenceTypeMap`（一个 key 可以对应多个 `ReferenceConfig`，`destroy(key, type)` 会把它们全销毁）。注意 key 里没有 `timeout`、没有 `cluster`、没有 `loadbalance`：同一个接口的两个引用只要 `group` 和 `version` 相同，就一定是同一个代理，后者的调优参数会被静默忽略。要真正区分，只能改 `group` / `version`，或者不用缓存、直接调 `ReferenceConfig.get()`。

`SimpleReferenceCache` 的 `referenceTypeMap` 还支撑 `getAll(Class)` 和 `destroy(Class)`，Spring 的 `@DubboReference` 注入、`DubboBootstrap` 的引用检查都走这个入口。Spring 侧还有一层 `ReferenceBeanManager.referenceConfigMap`，按属性生成的 `referenceKey` 复用 `ReferenceConfig`，避免同参数重复建配置对象。

缓存也是内存泄漏的高发点。`SimpleReferenceCache` 是静态的 `CACHE_HOLDER`，按 name 存，默认 `_DEFAULT_`；`referenceKeyMap` 里存的是 `ReferenceConfigBase` 强引用，而每个 `ReferenceConfig` 又持有 `invoker`、`consumerModel`、代理。所以动态创建引用（比如按租户建 `ReferenceConfig`）而不调 `destroy(key, type)`，`invoker` 永远不会释放，注册中心的订阅也不会退订。`get(rc, check)` 里那条 `CONFIG_API_WRONG_USE` 警告就是为 `singleton=false` + 缓存同时使用准备的。

容易误解的地方：

1. **"代理里握着一条长连接"**。代理只握 `Invoker`。连接在 `DubboInvoker` 的 `ClientsProvider` 里（内部是 `ExchangeClient` 列表），而且默认是共享的（`connections=0` 时走 `SharedClientsProvider`，按 `host:port` 复用）。
2. **"`check=false` 就不会报 No provider available"**。它只关掉启动期检查。调用期 `AbstractClusterInvoker.checkInvokers()` 照样抛 `RpcException`，只是错误码从 `CLUSTER_NO_VALID_PROVIDER` 变成 `NO_INVOKER_AVAILABLE_AFTER_FILTER`。
3. **" `lazy=true` 表示启动不 refer"**。它只是让 `DubboProtocol` 用 `LazyConnectExchangeClient` 延迟建连，而且这个 client 未连接时 `isConnected()` 返回 true，会让 `check` 失效。启动不 refer 是 `init=false`。
4. **"refer 之后就是 `RegistryDirectory` 在工作"**。3.3 默认返回 `ServiceDiscoveryMigrationInvoker`，`doMigrate` 直接 `FORCE_APPLICATION`，跑的是 `ServiceDiscoveryRegistryDirectory`；`RegistryDirectory` 只在接口级/迁移路径里由 `RegistryProtocol.getInvoker()` 创建，源码自己标了 FIXME。

记住这一句：引用 = 一堆 provider Invoker 被 Directory 收拢、被 Cluster 包住、最后被 ProxyFactory 糊成一个你以为是本地对象的代理。

第5章　和 Spring 的接缝：@DubboService 是怎么被扫描到的

一句话：注解扫成 ServiceBean，Deployer 统一导出。

为什么需要它：一个标了 `@DubboService` 的实现类，Spring 容器里明明只是一个普通 Bean，为什么它会自动监听端口、注册到注册中心？更具体的麻烦是：同一个接口、两个不同 `group` 的实现放在一个 JVM 里，为什么不会互相覆盖？为什么在 Spring Boot 里只加一个 starter、配一行 `dubbo.scan.base-packages`，服务就起来了，而在纯 Spring XML 里却要写 `<dubbo:annotation package="..."/>`？还有，为什么有人给 `ServiceBean` 加 `ApplicationListener` 想监听"导出完成"，时序总是对不上？这些问题的答案都不在业务代码里，而在 Spring 的 `BeanDefinition` 生命周期和 Dubbo 的部署器之间那道接缝里。这章把这条链路拆开：XML 时代靠 NamespaceHandler 把标签翻译成 BeanDefinition，注解时代靠 `BeanDefinitionRegistryPostProcessor` 抢在 Bean 实例化之前注册 `ServiceBean`，`@DubboReference` 靠两段式后处理器注入懒加载代理，最后由 `DefaultModuleDeployer` 统一触发 `export()`；而 3.x 引入的 `FrameworkModel / ApplicationModel / ModuleModel` 三层，决定了这些 Bean 到底归谁管、配置往哪放、什么时候销毁。

核心结构：

概念	作用
`spring.handlers` / `spring.schemas`	把 `http://dubbo.apache.org/schema/dubbo` 命名空间映射到 `DubboNamespaceHandler`，XSD 落到 `META-INF/dubbo.xsd`
`DubboNamespaceHandler`	继承 `NamespaceHandlerSupport`，`init()` 里为 `service` / `reference` / `application` 等标签注册 `DubboBeanDefinitionParser`；`parse()` 里额外调 `DubboSpringInitializer.initialize(registry)`
`DubboBeanDefinitionParser`	实现 `BeanDefinitionParser`，把 XML 元素解析成 `RootBeanDefinition`；`beanClass` 为 `ServiceBean` 时设 `AUTOWIRE_CONSTRUCTOR` 并把 `<dubbo:service class="...">` 变成 `ref` 属性
`ServiceBean` / `ReferenceBean`	`ServiceBean<T> extends ServiceConfig<T>` 且实现 `InitializingBean`；`ReferenceBean<T>` 实现 `FactoryBean<T>`，产出消费端代理
`DubboComponentScanRegistrar`	`ImportBeanDefinitionRegistrar`，解析 `@DubboComponentScan` / `@EnableDubbo` 的 `basePackages`，注册 `ServiceAnnotationPostProcessor` 的 BeanDefinition
`ServiceAnnotationPostProcessor`	`BeanDefinitionRegistryPostProcessor`，用 `DubboClassPathBeanDefinitionScanner` 扫 `@DubboService`，为每个命中类注册一个 `ServiceBean` BeanDefinition
`ReferenceAnnotationBeanPostProcessor`	`BeanFactoryPostProcessor` + `InstantiationAwareBeanPostProcessor`：先注册 `ReferenceBean` 定义，后在属性注入阶段把代理灌进 `@DubboReference` 字段
`DubboAutoConfiguration`	Spring Boot 自动配置，类级 `@ConditionalOnProperty(prefix="dubbo", name="enabled", matchIfMissing=true)`，方法级再按 `dubbo.scan.base-packages` 决定要不要注册扫描器
`DubboSpringInitializer`	按 `BeanDefinitionRegistry` 缓存 `DubboSpringInitContext`，创建/复用 `ApplicationModel` 与 `ModuleModel`，并作为单例注册进 BeanFactory
`DefaultModuleDeployer`	每个 `ModuleModel` 一个，`start()` → `exportServices()` → 遍历 `configManager.getServices()` 调 `sc.export(...)`

关键代码：

```
// ServiceAnnotationPostProcessor.java:106
public class ServiceAnnotationPostProcessor implements BeanDefinitionRegistryPostProcessor, ... {

    // ServiceAnnotationPostProcessor.java:169
    @Override
    public void postProcessBeanDefinitionRegistry(BeanDefinitionRegistry registry) throws BeansException {
        this.registry = registry;
        scanServiceBeans(resolvedPackagesToScan, registry);   // 为什么：Registry 后处理器先于普通 Bean 实例化，此时才允许往容器里塞新 BD
    }

    // ServiceAnnotationPostProcessor.java:205（骨架节选）
    private void scanServiceBeans(Set<String> packagesToScan, BeanDefinitionRegistry registry) {
        scanned = true;
        // ... 空包校验、ServicePackagesHolder.isPackageScanned 去重
        DubboClassPathBeanDefinitionScanner scanner =
                new DubboClassPathBeanDefinitionScanner(registry, environment, resourceLoader);
        BeanNameGenerator beanNameGenerator = resolveBeanNameGenerator(registry);
        scanner.setBeanNameGenerator(beanNameGenerator);
        for (Class<? extends Annotation> annotationType : serviceAnnotationTypes) {
            scanner.addIncludeFilter(new AnnotationTypeFilter(annotationType));  // @DubboService / 旧 @Service / 兼容 @com.alibaba...@Service
        }
        for (String packageToScan : packagesToScan) {
            scanner.scan(packageToScan);                        // 为什么：先把实现类本身注册成 Spring Bean
            Set<BeanDefinitionHolder> beanDefinitionHolders =
                    findServiceBeanDefinitionHolders(scanner, packageToScan, registry, beanNameGenerator);
            for (BeanDefinitionHolder beanDefinitionHolder : beanDefinitionHolders) {
                processScannedBeanDefinition(beanDefinitionHolder);  // 为什么：每个 @DubboService 类再额外生成一个 ServiceBean BD
            }
        }
    }

    // ServiceAnnotationPostProcessor.java:453（骨架节选）
    private AbstractBeanDefinition buildServiceBeanDefinition(Map<String, Object> serviceAnnotationAttributes,
                                                              String serviceInterface, String refServiceBeanName) {
        BeanDefinitionBuilder builder = rootBeanDefinition(ServiceBean.class);
        AbstractBeanDefinition beanDefinition = builder.getBeanDefinition();
        beanDefinition.setAutowireMode(AbstractBeanDefinition.AUTOWIRE_CONSTRUCTOR);  // 为什么：让 ModuleModel 走构造器注入
        // ... AnnotationPropertyValuesAdapter 把注解属性灌进 propertyValues
        addPropertyReference(builder, "ref", refServiceBeanName);   // 为什么：ref 指向真正的实现类 Bean，不 new 一个实例
        builder.addPropertyValue("interface", serviceInterface);
        // ... parameters / methods / providerIds / registryIds / protocolIds
        builder.setLazyInit(false);
        return builder.getBeanDefinition();
    }

    // ServiceAnnotationPostProcessor.java:641（骨架节选）
    private void registerServiceBeanDefinition(String serviceBeanName,
                                               AbstractBeanDefinition serviceBeanDefinition,
                                               String serviceInterface) {
        if (registry.containsBeanDefinition(serviceBeanName)) {
            // ... 同名但内容不同 → 抛 BeanDefinitionStoreException（CONFIG_DUPLICATED_BEAN_DEFINITION）
        }
        registry.registerBeanDefinition(serviceBeanName, serviceBeanDefinition);  // 为什么：真正落地到 Spring 容器
    }
}
```

```
// ServiceBean.java:111  —— Spring 的 InitializingBean 钩子：只"登记"，不导出
@Override
public void afterPropertiesSet() throws Exception {
    if (StringUtils.isEmpty(getPath())) {
        if (StringUtils.isNotEmpty(getInterface())) {
            setPath(getInterface());
        }
    }
    ModuleModel moduleModel = DubboBeanUtils.getModuleModel(applicationContext);
    moduleModel.getConfigManager().addService(this);   // 为什么：交给模块级 ConfigManager 统一托管，Spring 不再持有导出主动权
    moduleModel.getDeployer().setPending();            // 为什么：标记"有待导出服务"，等 deployer 统一 start
}

// DefaultModuleDeployer.java:440
private void exportServices() {
    for (ServiceConfigBase sc : configManager.getServices()) {
        exportServiceInternal(sc);
    }
}

// DefaultModuleDeployer.java:463（骨架节选）
private void exportServiceInternal(ServiceConfigBase sc) {
    ServiceConfig<?> serviceConfig = (ServiceConfig<?>) sc;
    if (!serviceConfig.isRefreshed()) { serviceConfig.refresh(); }
    if (sc.isExported()) { return; }
    if (exportAsync || sc.shouldExportAsync()) {
        // ... CompletableFuture.runAsync(() -> sc.export(), getServiceExportExecutor())
    } else {
        if (!sc.isExported()) {
            sc.export(RegisterTypeEnum.AUTO_REGISTER_BY_DEPLOYER);  // 为什么：由 deployer 统一触发，而非 Spring 自己调
            exportedServices.add(sc);
        }
    }
}
```

拆解：

5.1 XML 时代：NamespaceHandler 只是把标签翻译成 BeanDefinition

`dubbo-config-spring` 的 `META-INF/spring.handlers` 里写着 `http\://dubbo.apache.org/schema/dubbo=org.apache.dubbo.config.spring.schema.DubboNamespaceHandler`（兼容老命名空间 `code.alibabatech.com` 也指向同一个类），`spring.schemas` 把 XSD 映射到 `META-INF/dubbo.xsd`。Spring 解析 `<dubbo:service>` 时按命名空间找到 `DubboNamespaceHandler`，`init()`（`DubboNamespaceHandler.java:51`）逐个注册解析器：`application`、`module`、`registry`、`protocol`、`provider`、`consumer` 都用 `new DubboBeanDefinitionParser(XxxConfig.class)`，而 `service` / `reference` 分别绑到 `ServiceBean.class` / `ReferenceBean.class`（第 64-65 行），`annotation` 绑到独立的 `AnnotationBeanDefinitionParser`。

注意 `DubboNamespaceHandler.parse()` 不是直接 `super.parse()`：它先 `registerAnnotationConfigProcessors(registry)`（第 97-100 行，`AnnotationConfigUtils.registerAnnotationConfigProcessors`），再调 `DubboSpringInitializer.initialize(parserContext.getRegistry())`（第 83 行），最后才 `super.parse()` 并 `setSource(beanDefinition)`。也就是说，即便你用的是纯 XML，Dubbo 也会顺手把注解驱动的基础设施初始化好——这是 2.7.5 之后两套机制合流的痕迹。

`DubboBeanDefinitionParser.parse()`（`DubboBeanDefinitionParser.java:97`）的产物永远是 `RootBeanDefinition`：`beanDefinition.setBeanClass(beanClass)`、`setLazyInit(false)`（第 99-101 行）。关键分支在第 102-103 行：`ServiceBean.class.equals(beanClass)` 时设 `AUTOWIRE_CONSTRUCTOR`；第 142-152 行，`<dubbo:service class="...">` 会额外造一个 `RootBeanDefinition` 装实现类，再以 `new BeanDefinitionHolder(classDefinition, beanName + "Impl")` 的形式塞进 `ref` 属性。beanName 的规则在第 117-126 行：有 `id` 用 `id`，否则 `beanClass.getName() + "#" + counter` 自增避重；第 127 行还把这个名字 `beanDefinition.setAttribute(BEAN_NAME, beanName)` 存进 BD 元数据，后面 `ServiceBean` 的 `setBeanName()` 才能拿回去。类里那几个常量（第 81-86 行）暴露了 `parseProperties` 的特殊分支：`ONRETURN` / `ONTHROW` / `ONINVOKE` / `EXECUTOR` 要转成 `RuntimeBeanReference`，`METHOD` 要拼成 `ManagedList` / `ManagedMap`，其余按 `beanPropsCache`（第 90 行，按 beanClass 缓存属性类型 Map）反射找 setter；类里还预置了 `GROUP_AND_VERSION`（匹配 `xxx:yyy` 形式）与 `resolvePlaceholdersEnabled` 开关，不过这份文件里该正则只声明、未见后续引用。也就是说 XML 时代没有注解驱动，全靠"标签 → 配置类属性"的硬映射加 `resolveAttribute` 做占位符替换。理解这一点后再看注解时代的 `AnnotationPropertyValuesAdapter`（`ServiceAnnotationPostProcessor.java:476`），会发现它干的是同一件事，只是输入从 `Element` 换成了注解属性 Map。

5.2 注解时代：为什么非得是 BeanDefinitionRegistryPostProcessor

`@DubboComponentScan` 的定义（`DubboComponentScan.java:42`）是 `@Import(DubboComponentScanRegistrar.class)`。`DubboComponentScanRegistrar implements ImportBeanDefinitionRegistrar`，`registerBeanDefinitions()`（第 55 行）第一件事同样是 `DubboSpringInitializer.initialize(registry)`，然后 `getPackagesToScan(importingClassMetadata)`：先读 `@DubboComponentScan.basePackages / basePackageClasses`，读不到再退回 `@EnableDubbo.scanBasePackages / scanBasePackageClasses`（第 86-90 行），都没有就用注解所在类的包名（第 93 行）。拿到包集合后，`registerServiceAnnotationPostProcessor()`（第 72 行）用 `rootBeanDefinition(SpringCompatUtils.serviceAnnotationPostProcessor())` 构造一个 BD，把 `packagesToScan` 作为**构造器参数**传进去，`setRole(ROLE_INFRASTRUCTURE)`，再 `BeanDefinitionReaderUtils.registerWithGeneratedName` 注册。

`ServiceAnnotationPostProcessor` 的类声明很清楚（`ServiceAnnotationPostProcessor.java:106-112`）：`implements BeanDefinitionRegistryPostProcessor, EnvironmentAware, ResourceLoaderAware, BeanClassLoaderAware, ApplicationContextAware, InitializingBean`。为什么必须是 `BeanDefinitionRegistryPostProcessor` 而不是普通的 `BeanFactoryPostProcessor`？因为 Spring 的调用顺序是 `postProcessBeanDefinitionRegistry()` 先于 `postProcessBeanFactory()`，只有前者在"所有 BeanDefinition 已就绪、但普通 Bean 尚未实例化"的窗口里允许注册**新的** BeanDefinition；新注册的 BD 还能继续被后续的后处理器和实例化流程处理。它的 `postProcessBeanDefinitionRegistry`（第 169 行）只做一件事：`scanServiceBeans(resolvedPackagesToScan, registry)`；`afterPropertiesSet()`（第 164 行）负责 `resolvePlaceholders` 解析包名的占位符。

扫描用的是 `DubboClassPathBeanDefinitionScanner`（继承 Spring 的 `ClassPathBeanDefinitionScanner`），include filter 是 `serviceAnnotationTypes` 里三种注解的 `AnnotationTypeFilter`（第 224-226 行；`loadServiceAnnotationTypes()` 在第 118-135 行，按 `Dubbo2CompactUtils` 是否启用决定是否兼容老 `@com.alibaba.dubbo.config.annotation.Service`）。`scanner.scan(packageToScan)`（第 245 行）把命中的实现类注册成 Spring Bean，`findServiceBeanDefinitionHolders()`（第 332 行）再 `scanner.findCandidateComponents(packageToScan)` 把它们找回来，逐个 `processScannedBeanDefinition()`（第 359 行）。这里有 doubly-register 的关键：一个 `@DubboService` 类最终在容器里对应**两个** BeanDefinition——实现类 Bean 和 `ServiceBean`；后者 `ref` 用 `addPropertyReference(builder, "ref", annotatedServiceBeanName)`（第 481 行）指回前者。

`buildServiceBeanDefinition()`（第 453 行）里几个容易被跳过的细节：`rootBeanDefinition(ServiceBean.class)`；`AUTOWIRE_CONSTRUCTOR` 是为了让 `ServiceBean` 的 `@Autowired ServiceBean(ModuleModel moduleModel)` 构造器（第 59-63 行）拿到正确的模块；`ignoreAttributeNames`（第 463 行）把 `provider/monitor/application/module/registry/protocol/methods/interfaceName/parameters/executor` 排除在"直接灌属性"之外，因为它们要单独转成 `xxxIds` 或 `MethodConfig`；`builder.setLazyInit(false)`（第 538 行）保证不会因懒加载漏掉导出。bean 名由 `generateServiceBeanName()`（第 412 行）用 `ServiceBeanNameBuilder.create(serviceInterface, environment).group(...).version(...).build()` 生成——`group` / `version` 参与命名，所以同接口不同分组/版本能共存；`registerServiceBeanDefinition()`（第 641 行）遇到同名但内容不同的 BD 会抛 `BeanDefinitionStoreException`，而不是静默覆盖。另外 `postProcessBeanFactory()`（第 175 行）还有一条支线：遍历已有 BD，用 `getServiceAnnotationAttributes()` 识别 `@Bean` 方法上的 `@DubboService`（第 575 行），走 `processAnnotatedBeanDefinition()`（第 616 行），让 Java Config 风格也能用，并额外把 `Constants.ID` 写进属性（第 636 行）。

`resolveBeanNameGenerator()`（第 296 行）也别跳过：它先尝试从 `SingletonBeanRegistry` 里取名为 `CONFIGURATION_BEAN_NAME_GENERATOR`（即 `AnnotationConfigUtils` 那个常量）的 `BeanNameGenerator`，取不到才退化到 `new AnnotationBeanNameGenerator()`，并且打日志提醒"it maybe a potential problem on bean name generation"（第 306-317 行）。源码注释（第 284-294 行）说得很直白：本该复用 `ConfigurationClassPostProcessor#componentScanBeanNameGenerator`，否则 Dubbo 扫描出来的 bean 名可能和 Spring `@ComponentScan` 扫出来的不一致。这正是"实现类 Bean"与 `ServiceBean.ref` 指引能否对上的关键——`buildServiceBeanDefinition` 里 `addPropertyReference(builder, "ref", refServiceBeanName)`（第 481 行）传的是 `beanDefinitionHolder.getBeanName()`（第 370 行），名字一旦算错，运行时就是 `NoSuchBeanDefinitionException`。这也解释了为什么 Dubbo 不自己写扫描器，而是继承 Spring 的 `ClassPathBeanDefinitionScanner`：bean 名生成、条件注解、`@Scope` 这些语义必须和 Spring 保持一致。

还有一层防重复机制容易被忽略：`scanServiceBeans()` 在每个包扫描前都问 `servicePackagesHolder.isPackageScanned(packageToScan)`（第 234 行），扫完 `addScannedPackage` / `addScannedClass`（第 265、280 行）；scanner 上还挂了 `ScanExcludeFilter`（内部类，第 686 行），它的 `match()` 用 `servicePackagesHolder.isClassScanned(className)` 把已注册过的类剔掉。这是为了应对同一个包被 `@DubboComponentScan` 与 Spring 自己的 `@ComponentScan` 重复扫、或者父子容器各扫一遍的情况——否则同一个接口会注册出两份 `ServiceBean`。扫描器类本身也不是写死的：`DubboComponentScanRegistrar` 用 `rootBeanDefinition(SpringCompatUtils.serviceAnnotationPostProcessor())`（第 74 行，`DubboAutoConfiguration` 第 70 行同样），而 `SpringCompatUtils.serviceAnnotationPostProcessor()`（`SpringCompatUtils.java:144`）在 Spring 6 / AOT 条件下会解析到 `org.apache.dubbo.config.spring6.beans.factory.annotation.ServiceAnnotationWithAotPostProcessor`，消费端还有对应的 `ReferenceAnnotationWithAotBeanPostProcessor`（第 138 行）；`AotWithSpringDetector.useGeneratedArtifacts()`（`DubboNamespaceHandler.java:98`、`ServiceAnnotationPostProcessor.java:241`）则控制 AOT 阶段跳过注解配置处理器、改用生成好的元数据。这些分支说明：扫描器不是"扫一次就完事"，它对"重复扫描"和"原生镜像"都做了显式让步。

5.3 @DubboReference 的注入时机：两段式，注入的是懒代理

先补一个前面跳过的环节：这个后处理器是谁注册进容器的？答案是 `DubboInfraBeanRegisterPostProcessor`（`dubbo-config-spring/.../context/`，`implements BeanDefinitionRegistryPostProcessor`，第 35 行），它由 `DubboSpringInitializer.initialize()` 一并注册。它的 `postProcessBeanFactory()`（第 50 行）做两件事：`beanFactory.getBean(ReferenceAnnotationBeanPostProcessor.BEAN_NAME, ...)` 后 `beanFactory.addBeanPostProcessor(referenceAnnotationBeanPostProcessor)`（第 58-60 行），注释写明"register ... early before PropertySourcesPlaceholderConfigurer/PropertyPlaceholderConfigurer for processing early init ReferenceBean"——必须抢在占位符配置器之前，否则 `@DubboReference` 里 `${...}` 还没解析就把 `ReferenceBean` 建好了；然后 `registry.removeBeanDefinition(BEAN_NAME)`（第 68 行）把自己从 BD 列表里删掉，避免重复执行。`ReferenceAnnotationBeanPostProcessor.postProcessBeanFactory()` 结尾（第 173-185 行）还有对称处理：发现自己已经被注册成 `BeanPostProcessor` 时，主动 `beanDefinitionRegistry.removeBeanDefinition(BEAN_NAME)`，注释点名是为避免 `BeanPostProcessorChecker` 报"bean 不适资格被所有 BPP 处理"的告警。换句话说，Dubbo 在 Spring 后处理器的**注册顺序**上做了两处手工干预，这不是普通 starter 会遇到的场景。

`ReferenceAnnotationBeanPostProcessor`（第 94 行）`extends AbstractAnnotationBeanPostProcessor implements ApplicationContextAware, BeanFactoryPostProcessor`；父类（`AbstractAnnotationBeanPostProcessor.java:66-68`）`implements InstantiationAwareBeanPostProcessor, MergedBeanDefinitionPostProcessor`。类上的 Javadoc（第 78-88 行）把两段写得明明白白：Step 1 实现 `BeanFactoryPostProcessor` 是为了"更早地注册 reference bean definition，以便与 XML 配置共享"；Step 2 实现 `InstantiationAwareBeanPostProcessor` 才是把实例注入 `@DubboReference` 的字段和 setter。

Step 1：`postProcessBeanFactory()`（第 140 行）遍历 `beanFactory.getBeanDefinitionNames()`，对每个能解析出 `beanType` 的定义调 `findInjectionMetadata(beanName, beanType, null)` → `prepareInjection(metadata)`（第 379 行）。`prepareInjection` 对字段/方法元素调 `registerReferenceBean(...)`（第 414 行），后者最终 `new RootBeanDefinition()`、`setBeanClassName(ReferenceBean.class.getName())`（第 538-539 行），把注解属性放进 `beanDefinition.setAttribute(Constants.REFERENCE_PROPS, attributes)`，再 `beanDefinitionRegistry.registerBeanDefinition(referenceBeanName, beanDefinition)`（第 564 行）。去重靠 `ReferenceBeanSupport.generateReferenceKey(attributes, applicationContext)`（第 441 行）与 `referenceBeanManager.getBeanNamesByKey(referenceKey)`（第 444 行）：命中已注册的 key 就 `registerAlias` 复用（第 528 行），不再新建。第 553-559 行还挂了一个 `GenericBeanDefinition` 作为 `setDecoratedDefinition`，注释说明是为了避免 `AbstractBeanFactory#getTypeForFactoryBean()` 时把 `ReferenceBean` 提前实例化。`postProcessMergedBeanDefinition()`（第 318 行）也会再 `prepareInjection` 一次作为兜底。方法最后 `applicationContext.publishEvent(new DubboConfigInitEvent(...))`（第 190 行）是个 early event。

Step 2：真正的字段赋值发生在 Spring 的 `populateBean` 阶段，`postProcessProperties(PropertyValues pvs, Object bean, String beanName)`（第 360 行）里 `findInjectionMetadata` → `prepareInjection` → `metadata.inject(bean, beanName, pvs)`；`AbstractAnnotationBeanPostProcessor` 的注入元素最终调 `doGetInjectedBean(...)`，这里的实现（第 571 行）只有一句核心：`return getBeanFactory().getBean((String) injectedElement.injectedObject);`——拿到的不是连接好的 Invoker，而是一个 `ReferenceBean`（`FactoryBean<T>`，`ReferenceBean.java:110-115`）产出的代理。`ReferenceBean.getObject()`（第 211 行）在 3.3 里返回懒代理 `DubboReferenceLazyInitTargetSource`（第 471 行），真正的 `referenceConfig.get()` 在 `LazyTargetSource.getTarget()` 里（第 464、467 行）才发生。所以结论是：`@DubboReference` 字段在属性注入阶段被填成一个代理对象，`check=false` 不会因为"注入得早"就报错；而同一个 `@DubboReference` 配置在多个类里出现，容器里也只有一个 `ReferenceBean` BD。

5.4 Spring Boot 自动配置与 ModuleModel/ApplicationModel 分层

3.3 的自动配置分两条线：`dubbo-spring-boot/dubbo-spring-boot-compatible/autoconfigure` 的 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 列了 `DubboAutoConfiguration`、`DubboRelaxedBindingAutoConfiguration`、`DubboListenerAutoConfiguration`，`spring.factories` 里用 `EnableAutoConfiguration=` 再写一遍（兼容 Boot 2.x）；而 Boot 3 的 `dubbo-spring-boot-autoconfigure` 的 `AutoConfiguration.imports` 只有 `DubboRelaxedBinding2AutoConfiguration` 和 `DubboTripleAutoConfiguration`。所以"自动配置类"具体在哪个 jar，取决于你引的是哪条 starter 线，别看到源码里有两份就以为是重复。

`DubboAutoConfiguration`（`DubboAutoConfiguration.java:50`）类上是 `@ConditionalOnProperty(prefix = DUBBO_PREFIX, name = "enabled", matchIfMissing = true)`，`DUBBO_PREFIX = "dubbo"`（`DubboUtils.java:52`）——`dubbo.enabled=false` 直接关掉整个配置类；`matchIfMissing = true` 意味着不配也生效。类上还有 `@AutoConfigureAfter(DubboRelaxedBindingAutoConfiguration.class)` 和 `@EnableDubboConfig`（第 52-53 行），后者负责把 `dubbo.application.*` / `dubbo.registry.*` 这些外部配置绑定成 Dubbo 的配置 Bean。唯一的 `@Bean` 方法 `serviceAnnotationBeanProcessor` 上叠了两个条件（第 62-63 行）：`@ConditionalOnProperty(prefix = DUBBO_SCAN_PREFIX, name = BASE_PACKAGES_PROPERTY_NAME)`（`dubbo.scan.base-packages`）和 `@ConditionalOnBean(name = BASE_PACKAGES_BEAN_NAME)`；`DUBBO_SCAN_PREFIX = "dubbo.scan."`（`DubboUtils.java:57-58`），`BASE_PACKAGES_BEAN_NAME = "dubbo-service-class-base-packages"`（第 147 行）。那个 `Set<String>` Bean 由 `DubboRelaxedBindingAutoConfiguration.dubboBasePackages(Environment)`（第 62 行）通过 `propertyResolver.getProperty(BASE_PACKAGES_PROPERTY_NAME, Set.class, emptySet())` 提供。于是逻辑闭环：不配 `dubbo.scan.base-packages` → 没有 base-packages Bean → 不注册 `ServiceAnnotationPostProcessor` → 一个类都不扫；但 `@EnableDubboConfig` 仍在，全局配置照绑。这解释了为什么纯注解项目里漏配这一行，服务"静默不暴露"。

`@EnableDubboConfig` 本身（`EnableDubboConfig.java:69`）是 `@Import(DubboConfigConfigurationRegistrar.class)`，而这个 Registrar 的 `registerBeanDefinitions()`（`DubboConfigConfigurationRegistrar.java:38`）只有一句 `DubboSpringInitializer.initialize(registry)`。真正的属性绑定在 `DubboConfigConfiguration` 里：内部类 `Single` 用一组 `@EnableConfigurationBeanBinding`（第 57-69 行）把 `dubbo.application` → `ApplicationConfig`、`dubbo.module` → `ModuleConfig`、`dubbo.registry` → `RegistryConfig`、`dubbo.protocol` → `ProtocolConfig`、`dubbo.provider` / `dubbo.consumer` / `dubbo.monitor` / `dubbo.config-center` / `dubbo.metadata-report` / `dubbo.metrics` / `dubbo.tracing` / `dubbo.ssl` 逐一映射；内部类 `Multiple`（`public static class Multiple {}`，第 95 行，绑定注解在第 76-93 行）则是 `dubbo.applications` / `dubbo.registries` / `dubbo.protocols` / `dubbo.metricses` 等复数前缀并带 `multiple = true`。`EnableDubboConfig.multiple` 默认 `true`（第 72-77 行）。这条线解释了 Spring Boot 里为什么写 `application.yml` 的 `dubbo.application.name` 就能变成 Dubbo 的 `ApplicationConfig`：它不是 `DubboAutoConfiguration` 里的 `@Bean`，而是 Registrar 注册进来的配置绑定基础设施。于是"自动配置"其实分成两半——配置绑定（一定有）和服务扫描（要看 `dubbo.scan.base-packages`），两者条件完全不同。

为什么要 `ModuleModel` / `ApplicationModel` / `FrameworkModel` 三层？看 `DubboSpringInitializer.initialize()`（第 51 行）：它用 `Map<BeanDefinitionRegistry, DubboSpringInitContext> REGISTRY_CONTEXT_MAP`（第 46 行）按 Spring 容器缓存初始化上下文；`initContext()`（第 103 行）判断若 `ApplicationModel.defaultModel()` 尚未绑定 context 就复用它，否则 `FrameworkModel.defaultModel().newApplication()`（第 121 行），再 `applicationModel.newModule()` 得到 `ModuleModel`（第 125-127 行），最后 `registerSingleton(beanFactory, context.getApplicationModel())` 与 `registerSingleton(beanFactory, context.getModuleModel())`（第 180-183 行）把两者作为单例放进 Spring BeanFactory。这么做的收益是**作用域隔离**：`Filter` 是 `@SPI(scope = ExtensionScope.MODULE)`（`Filter.java:68`），而 `FilterChainBuilder` 是 `@SPI(scope = APPLICATION)`（`FilterChainBuilder.java:43`）——不同模块可以有自己的 Filter 配置、自己的 `ConfigManager`、自己的 `DefaultModuleDeployer`，生命周期可以独立 `destroy()`；应用级的东西（如 `ExecutorRepository`、`FilterChainBuilder`）由 `ApplicationModel` 共享。Spring 侧的直接体现就是 `ServiceBean` 的 `@Autowired ServiceBean(ModuleModel moduleModel)`（第 59-63 行）和 `afterPropertiesSet()` 里的 `moduleModel.getConfigManager().addService(this)`（第 118-119 行）：一个 `ServiceBean` 从出生起就被钉在某个 `ModuleModel` 上。`DefaultModuleDeployer.start()`（第 155 行）里还专门判断 `moduleModel != moduleModel.getApplicationModel().getInternalModule()` 才 `applicationDeployer.prepareInternalModule()`（第 181-183 行）——内部模块是给元数据、配置中心等基础设施用的，不参与业务服务导出。第 6 章会看到，`DefaultFilterChainBuilder.getModuleModelsFromUrl()`（`DefaultFilterChainBuilder.java:138`）正是为了把"应用级服务发现产生的 `InstanceAddressURL`（持有 `ApplicationModel`）"反查回它下面所有 `ModuleModel`，才能取到模块级的 Filter 配置。

最后把链路收尾：`ServiceConfigBase.export()`（`ServiceConfigBase.java:430`）是 `final`，只做 `export(RegisterTypeEnum.AUTO_REGISTER)`；`ServiceConfig.export(RegisterTypeEnum)`（第 310 行）先 `getScopeModel().getDeployer().start()`（第 320 行，注释说"export service and auto start application instance"），再 `refresh()`、`shouldExport()`、`init()`，按 `shouldDelay()` / `MANUAL_REGISTER` 分支落到 `doExport(registerType)`（第 343 行），`doExport` 里 `doExportUrls(registerType)` 后调 `exported()`（第 555-556 行）。所以 `ServiceBean.exported()` 覆写（第 138 行）发出的 `ServiceBeanExportedEvent`，是在 deployer 驱动 `export` 之后才发布的——这就是"给 `ServiceBean` 加监听器监听导出完成却对不上时序"的根因。

容易误解的地方：

1. **`@DubboService` 不是 Spring 的组件注解**。`scanner.scan(packageToScan)`（`ServiceAnnotationPostProcessor.java:245`）先把实现类注册成一个 Spring Bean，`processScannedBeanDefinition()` 再为它注册第二个 `ServiceBean` BeanDefinition，两者靠 `ref` 属性关联（`buildServiceBeanDefinition` 第 481 行）。以为"一个类 = 一个 Bean"是后面所有困惑的起点：你在容器里按实现类类型 `getBean()` 拿到的是业务对象，按 `ServiceBean` 类型拿到的才是导出配置。
2. **`ServiceBean.afterPropertiesSet()` 不导出服务**。它只 `moduleModel.getConfigManager().addService(this)` + `moduleModel.getDeployer().setPending()`（第 118-120 行）。真正的 `export()` 由 `DefaultModuleDeployer.start()` → `exportServices()` → `exportServiceInternal()` 统一触发，还可能走 `CompletableFuture.runAsync` 异步分支（第 473-492 行）。想靠 `InitializingBean` 或 `@PostConstruct` 判断"端口已经开了"必然踩空。
3. **`@DubboReference` 注入的不是活连接，是懒代理**。`doGetInjectedBean()`（第 571 行）只是 `getBeanFactory().getBean(referenceBeanName)`，得到 `ReferenceBean.getObject()` 的代理；`referenceConfig.get()` 在 `DubboReferenceLazyInitTargetSource.getTarget()`（第 471-473 行）首次调用时才执行。而且 `ReferenceBean` 的 BD 早在 `BeanFactoryPostProcessor` 阶段就注册了（第 140 行），比字段注入早一个阶段；`postProcessProperties`（第 360 行）才做 `metadata.inject`。把两段当成一个阶段，容易误判 `check=false`、循环依赖和 `@Bean` 方法上 `@DubboReference` 的行为。
4. **`dubbo.enabled=false` 与"没配 `dubbo.scan.base-packages`"是两件事**，而且 3.3 里 `DubboAutoConfiguration` 在 `dubbo-spring-boot-compatible/autoconfigure`，不在主 `dubbo-spring-boot-autoconfigure` 模块。前者靠类级 `@ConditionalOnProperty`（第 50 行）整体关闭；后者靠方法级 `@ConditionalOnProperty(prefix="dubbo.scan.", name="base-packages")` + `@ConditionalOnBean`（第 62-63 行）只不注册扫描器，`@EnableDubboConfig` 依旧生效。另外别把 `ModuleModel` 当全局单例：`DubboSpringInitializer` 是按 `BeanDefinitionRegistry` 缓存 context 的（`REGISTRY_CONTEXT_MAP`，第 46 行），父子容器、多 Spring 上下文完全可能绑定到不同的 `ApplicationModel`。

记住这一句：扫描器负责"把注解变成 BeanDefinition"，Deployer 负责"把 BeanDefinition 变成网络端口"，中间靠 ModuleModel 的 ConfigManager 对接。

第6章　Filter 链：一次调用的洋葱模型

一句话：Filter 逆序包成 Invoker 链，逐层递归。

为什么需要它：给接口加了个自定义鉴权 Filter，上线后发现超时统计没了；另一个 Filter 里 `catch` 住异常返回了个默认值，结果消费端不再重试、订单状态卡住；还有人想加个"每次调用只打一次日志"的埋点，却发现日志被打印了好几遍，因为逻辑挂在负载均衡之后的实例级链上。这些都不是业务 bug，而是没搞清一次 RPC 从 Proxy 到真实 Invoker 中间套了几层、顺序由谁定、回调在哪触发。这章解决三件事：链是怎么被拼出来的（3.3 已经不是 2.x 那个私有 `buildInvokerChain` 了）、`@Activate(group = CONSUMER/PROVIDER)` 的 `group / value / order / before / after` 到底怎么筛和排、`BaseFilter.Listener` 的 `onResponse` / `onError` 是给谁用的，以及为什么一个 Filter 绝不能吞异常。

核心结构：

概念	作用
`ProtocolFilterWrapper`	`Protocol` 的 SPI wrapper，`@Activate(order = 100)`（`dubbo-cluster` 模块）；`export` / `refer` 时调 `FilterChainBuilder.buildInvokerChain`
`FilterChainBuilder` / `DefaultFilterChainBuilder`	`@SPI(value="default", scope=APPLICATION)`；`getActivateExtension` 拿已排序 Filter，逆序 new 节点，最外层再套回调注册器
`CopyOfFilterChainNode`	`@Experimental` 的链节点，`invoke()` 里 `filter.invoke(nextNode, invocation)` 构成递归；`getUrl/getInterface/isAvailable/destroy` 全委托 `originalInvoker`
`CallbackRegistrationInvoker`	包在链最外层，`asyncResult.whenCompleteWithContext` 里逆序触发所有 Listener 的 `onResponse` / `onError`
`Filter` / `BaseFilter`	`Filter extends BaseFilter`，`@SPI(scope = MODULE)`；`BaseFilter.invoke` 约定"必须调 `invoker.invoke()`"，`BaseFilter.Listener` 提供两个回调
`@Activate`	`group` 决定哪一端生效，`value` 是 URL 条件，`order` / `before` / `after` 决定顺序
`ActivateComparator`	`order` 升序 → `before` / `after` → 类名字符串兜底；`getActivateExtension` 用 `TreeMap<>(activateComparator)` 产出有序列表
`ClusterFilter`	3.0 新增，跑在负载均衡**之前**；`ConsumerContextFilter` 等位于 `dubbo-cluster/.../filter/support/`，与实例级 `Filter` 是两条链
`ListenableFilter`	3.3 已 `@Deprecated`；旧式"每调用一个 Listener"容器，现在推荐直接 `implements Filter.Listener`

关键代码：

```
// ProtocolFilterWrapper.java:56 (类上有 @Activate(order = 100)，本身是 Protocol 的 wrapper)
@Override
public <T> Exporter<T> export(Invoker<T> invoker) throws RpcException {
    if (UrlUtils.isRegistry(invoker.getUrl())) {
        return protocol.export(invoker);                    // 为什么：registry:// 是注册中心包装，不套业务 Filter 链
    }
    FilterChainBuilder builder = getFilterChainBuilder(invoker.getUrl());
    return protocol.export(builder.buildInvokerChain(invoker, SERVICE_FILTER_KEY, CommonConstants.PROVIDER));
}

// ProtocolFilterWrapper.java:70
@Override
public <T> Invoker<T> refer(Class<T> type, URL url) throws RpcException {
    if (UrlUtils.isRegistry(url)) {
        return protocol.refer(type, url);
    }
    FilterChainBuilder builder = getFilterChainBuilder(url);
    return builder.buildInvokerChain(protocol.refer(type, url), REFERENCE_FILTER_KEY, CommonConstants.CONSUMER);
    // 为什么：同一套 SPI，靠 key(service.filter / reference.filter) + group(PROVIDER / CONSUMER) 取不同子集
}

// DefaultFilterChainBuilder.java:44 (骨架节选)
@Override
public <T> Invoker<T> buildInvokerChain(final Invoker<T> originalInvoker, String key, String group) {
    Invoker<T> last = originalInvoker;
    URL url = originalInvoker.getUrl();
    List<ModuleModel> moduleModels = getModuleModelsFromUrl(url);
    List<Filter> filters;
    if (moduleModels != null && moduleModels.size() == 1) {
        filters = ScopeModelUtil.getExtensionLoader(Filter.class, moduleModels.get(0))
                .getActivateExtension(url, key, group);     // 为什么：group 在这里过滤，返回的 List 已经按 ActivateComparator 排
好序
    } else if (moduleModels != null && moduleModels.size() > 1) {
        // ... 逐个 ModuleModel 取扩展、合并、sortingAndDeduplication(filters, directors)
        filters = new ArrayList<>();
    } else {
        filters = ScopeModelUtil.getExtensionLoader(Filter.class, null).getActivateExtension(url, key, group);
    }
    if (!CollectionUtils.isEmpty(filters)) {
        for (int i = filters.size() - 1; i >= 0; i--) {     // 为什么：逆序包装 → order 小的在最外层，"前半段"先执行、"后半段"后执行
            final Filter filter = filters.get(i);
            final Invoker<T> next = last;
            last = new CopyOfFilterChainNode<>(originalInvoker, next, filter);
        }
        return new CallbackRegistrationInvoker<>(last, filters);  // 为什么：回调统一在最外层收口，覆盖整条链
    }
    return last;
}
```

```
// FilterChainBuilder.java:344（CopyOfFilterChainNode.invoke，骨架节选）
@Override
public Result invoke(Invocation invocation) throws RpcException {
    Result asyncResult;
    try {
        InvocationProfilerUtils.enterDetailProfiler(invocation, () -> "Filter " + filter.getClass().getName() + " invoke.");
        asyncResult = filter.invoke(nextNode, invocation);   // 为什么：nextNode 是下一层，Filter 自己决定何时"穿过"——洋葱的递归就在
这里
        if (!(asyncResult instanceof AsyncRpcResult)) {       // 为什么：3.x 全异步，Filter 必须还 AsyncRpcResult，否则链断
            String msg = "The result of filter invocation must be AsyncRpcResult. ...";
            LOGGER.error(INTERNAL_ERROR, "", "", msg);
            throw new RpcException(msg);
        }
    } catch (Exception e) {
        InvocationProfilerUtils.releaseDetailProfiler(invocation);
        // ... listener(invocation) 非空则 listener.onError(e, originalInvoker, invocation)
        throw e;                                              // 为什么：异常必须原样往外抛，链才不会假装成功
    }
    return asyncResult;
}

// FilterChainBuilder.java:196（CallbackRegistrationInvoker.invoke，骨架节选）
@Override
public Result invoke(Invocation invocation) throws RpcException {
    Result asyncResult = filterInvoker.invoke(invocation);
    asyncResult.whenCompleteWithContext((r, t) -> {
        RuntimeException filterRuntimeException = null;
        for (int i = filters.size() - 1; i >= 0; i--) {      // 为什么：回调按"内层先、外层后"逆序触发，与 invoke 的进入顺序相反
            FILTER filter = filters.get(i);
            try {
                // ... 取 Listener：ListenableFilter.listener(invocation) 或 (FILTER.Listener) filter
                if (t == null) {
                    listener.onResponse(r, filterInvoker, invocation);  // 为什么：正常/业务异常都走这里，靠
appResponse.hasException() 区分
                } else {
                    listener.onError(t, filterInvoker, invocation);     // 为什么：框架异常（超时/网络/Filter 自身抛错）走这里
                }
                // ... listenableFilter.removeListener(invocation) 清理
            } catch (RuntimeException runtimeException) {
                // ... 记 CLUSTER_EXECUTE_FILTER_EXCEPTION，t = runtimeException
            }
        }
        if (filterRuntimeException != null) { throw filterRuntimeException; }
    });
    return asyncResult;
}
```

拆解：

6.1 两条链：ClusterFilter 在负载均衡之前，Filter 在实例之后

`Filter.java` 的 Javadoc（第 38-59 行）直接画了 3.x 与 2.x 的差别：3.x 消费端是 `Proxy -> ClusterFilter -> ClusterInvoker -> Filter -> Invoker`，2.x 是 `Proxy -> ClusterInvoker -> Filter -> Invoker`。注释写明 "Starting from 3.0, Filter on consumer side has been refactored"，并区分两类：`Filter` 工作在实例级，"each Filter is bond to one specific Provider instance(invoker)"（第 40 行）；`ClusterFilter` 是 3.0 新增的，"intercepts request before Loadbalancer picks one specific Filter(Invoker)"（第 41 行）。

对应到源码，`DefaultFilterChainBuilder` 有两个入口：`buildInvokerChain`（第 44 行，实例级）和 `buildClusterInvokerChain`（第 83 行，消费端集群级），后者用 `ClusterFilter.class` 取扩展，节点是 `CopyOfClusterFilterChainNode`，最外层是 `ClusterCallbackRegistrationInvoker`（第 112-114 行）。`dubbo-cluster/src/main/java/org/apache/dubbo/rpc/cluster/filter/support/` 下放的是 `ConsumerContextFilter`、`CallbackConsumerContextFilter`、`ConsumerClassLoaderFilter`、`MetricsConsumerFilter`——注意消费端的上下文设置在这里，而提供端的 `ContextFilter` 在 `dubbo-rpc-api/.../filter/`。`ProtocolFilterWrapper` 只负责实例级那条链（第 56、70 行），并且 `UrlUtils.isRegistry(...)` 时直接透传：注册中心的包装 Invoker 不套业务 Filter。

那 `ClusterFilter` 链到底在哪拼？不在 `ProtocolFilterWrapper`，而在 `AbstractCluster.join(directory, buildFilterChain)`（`dubbo-cluster/.../support/wrapper/AbstractCluster.java:58`）：`buildFilterChain` 为 true 时返回

`buildClusterInterceptors(doJoin(directory))`，否则直接 `doJoin`。`buildClusterInterceptors`（第 45 行）先 `new ClusterFilterInvoker<>(clusterInvoker)`，这个内部类的构造器（第 82-97 行）取所有 `FilterChainBuilder` 的激活扩展，逐个 `builder.buildClusterInvokerChain(tmpInvoker, REFERENCE_FILTER_KEY, CommonConstants.CONSUMER)` 串起来；外面再叠 `InvocationInterceptorBuilder` 生成的 `InvocationInterceptorInvoker`（第 66-75 行），并可通过 `CLUSTER_INTERCEPTOR_COMPATIBLE_KEY` 打开 2.7.x 兼容分支（第 48-53 行）。所以第 7 章里 `Cluster.join()` 产出的那个 `Invoker`，外包的是 `ClusterFilter` 链；`ProtocolFilterWrapper.refer()` 拼的是更里面、绑定单个实例的 `Filter` 链。两条链的构造点、扩展类型、作用域都不相同，读源码时按 `join()` 和 `refer()` 分开追才不会绕晕。

这个区分有实际后果：想做"每次 RPC 只执行一次"的事（埋点、上下文初始化、参数校验），应该写 `ClusterFilter`；挂在 `Filter` 上，在多实例、Failover 重试的场景里就会被执行多次。反过来，想针对某个具体 Provider 实例做统计/熔断，就得用 `Filter`。

6.2 拼链与递归：逆序包装、三个角色、AsyncRpcResult 约束

`buildInvokerChain` 的形状并不复杂，但三个角色各司其职。第一，`getActivateExtension(url, key, group)` 返回的 `List<Filter>` **已经排好序**（`ExtensionLoader` 里是 `TreeMap<>(activateComparator)`，`ExtensionLoader.java:346`）。第二，`for (int i = filters.size() - 1; i >= 0; i--)` 逆序包装：`last` 初始是 `originalInvoker`，第一次循环用 `filters[last]`（order 最大）包住真实 Invoker，最后一次用 `filters[0]`（order 最小）包在最外面。于是 `filters[0]` 的 `invoke` 前半段最先执行、后半段最后执行——这正是洋葱模型。第三，`CallbackRegistrationInvoker` 再套一层，专门负责回调。

节点 `CopyOfFilterChainNode` 的 `invoke`（`FilterChainBuilder.java:344`）就是递归本身：`filter.invoke(nextNode, invocation)`，`nextNode` 是下一层节点；每个 Filter 的 `invoke` 自己决定要不要 `invoker.invoke(invocation)`（`BaseFilter.java:21` 的约定原文是 "Always call invoker.invoke() in the implementation to hand over the request to the next filter node"）。不调就是短路：`TokenFilter`（`TokenFilter.java:42-56`）token 不匹配直接 `throw new RpcException`，根本没走到 `invoker.invoke(inv)`。节点对外伪装成同一个 Invoker：`getUrl()` / `getInterface()` / `isAvailable()` / `destroy()` 全部委托 `originalInvoker`（第 330-341、382-384 行），而不是 `nextNode`——这样链上任意一层拿到的 URL 都是原始 Invoker 的 URL，配置读取不会随层级漂移。

3.x 新增的硬约束在第 350-357 行：`asyncResult` 必须是 `AsyncRpcResult`，否则记 `INTERNAL_ERROR` 并抛 `RpcException`，提示语原文是 "The result of filter invocation must be AsyncRpcResult. (If you want to recreate a result, please use AsyncRpcResult.newDefaultAsyncResult.)"。所以自定义 Filter 里要造返回值只能用 `AsyncRpcResult.newDefaultAsyncResult(...)`，`FailsafeClusterInvoker`（第 61 行）、`FailbackClusterInvoker`（第 139 行）都是这么写的。

`FilterChainBuilder` 里其实并存两套节点实现，值得对照读：老的 `FilterChainNode`（第 61 行）在 `invoke` 里除了 `filter.invoke(nextNode, invocation)`，还直接 `return asyncResult.whenCompleteWithContext((r, t) -> ...)`（第 118-142 行），也就是**每个节点自己注册回调**——正常时 `listener.onResponse(r, originalInvoker, invocation)`，异常时 `listener.onError(t, ...)`，并在 `finally` 里 `listenableFilter.removeListener(invocation)`。新的 `CopyOfFilterChainNode`（第 311 行，标 `@Experimental`，注释说"replace FilterChainNode with this one when proved stable enough"）的 `invoke` 只返回 `asyncResult`（第 378 行），回调全部交给外层 `CallbackRegistrationInvoker`。这个改动的意义是：回调不再散落在每一层，异常路径与成功路径统一收口，且 `ListenableFilter` 的 `removeListener` 只在最外层执行一次。`DefaultFilterChainBuilder` 当前用的是 `CopyOf*` 这一套，但 `FilterChainNode` / `ClusterFilterChainNode` 仍留在源码里，读老文章里"每个 Filter 自己 whenComplete"的描述时别以为写错了，那是旧节点的行为。此外 `CallbackRegistrationInvoker` 的 catch 里（第 226-244 行）会把某个 Listener 抛出的 `RuntimeException` 记成 `CLUSTER_EXECUTE_FILTER_EXCEPTION`、`t = runtimeException` 并在循环结束 `throw filterRuntimeException`（第 246-248 行）——即回调里抛异常不会被静默吞掉，会替换掉原本的结果。

调用进入链之前，`AbstractClusterInvoker.invokeWithContext()`（`AbstractClusterInvoker.java:402`）会 `setContext(invoker)`、`setRemote(invoker, invocation)`，所以 `ExceptionFilter`（第 88 行）里的 `RpcContext.getServiceContext().getRemoteHost()` 才有值；`finally` 里 `clearContext(originInvoker)` 还原。这是第 5 章那条 Spring 链路和第 7 章集群链路共同喂给 Filter 的上下文。

6.3 @Activate 怎么决定"谁生效、谁先跑"

`ExtensionLoader.getActivateExtension(URL url, String[] values, String group)`（`ExtensionLoader.java:343`）是总入口。它用 `Map<Class<?>, T> activateExtensionsMap = new TreeMap<>(activateComparator)`（第 346 行）保证有序，把 `url.getParameter(key)` 按逗号拆成 `names`（第 328-330 行，`key` 就是 `SERVICE_FILTER_KEY` / `REFERENCE_FILTER_KEY`，即 `service.filter` / `reference.filter`）。筛选条件在第 394-402 行，四个 `&&`：`isMatchGroup(group, activateGroup)`（`group` 与 `@Activate.group()` 匹配）、`!namesSet.contains(name)`、`!namesSet.contains(REMOVE_VALUE_PREFIX + name)`（`-xxx` 剔除）、`isActive(cachedActivateValues.get(name), url)`（`value` 指定的 URL key 存在）。`value` 这一项就是"条件激活"：`@Activate(group = PROVIDER, value = TOKEN_KEY)` 的 `TokenFilter` 只有 URL 带 `token` 才生效，`ExecuteLimitFilter` 要 `executes`、`TpsLimitFilter` 要 `tps`（`org.apache.dubbo.rpc.Constants.TPS_LIMIT_RATE_KEY = "tps"`）、`ActiveLimitFilter` 要 `actives`、`DeprecatedFilter` 要 `deprecated`、`GenericImplFilter` 要 `generic`。此外 `-default`（第 351 行）一次性禁掉所有默认扩展，`default` 关键字（第 405-422 行）用来把自定义 Filter 插到默认链之前/之后——`ext1,default,ext2` 的语义源码注释写得很清楚。两个被 `getActivateExtension` 调用的判定也值得看一眼：`isMatchGroup(String group, Set<String> groups)`（`ExtensionLoader.java:460`）在 `group` 为空时直接放行，否则要求

`@Activate.group()` 里包含传入的 `PROVIDER` / `CONSUMER`；`isActive(String[][] keyPairs, URL url)` 解析 `value` 里 `key:value` 或纯 `key` 两种形式——纯 key 要求 URL 上存在该参数，`key:value` 还要求参数值匹配（第 375-387 行把 `TPS_LIMIT_RATE_KEY` 这类 value 拆成 `keyPairs` 缓存）。所以 `@Activate(value = ...)` 不是"打个标记"，而是运行时按 URL 逐条比对的开关：`token`、`executes`、`tps`、`actives` 这些参数没配，对应的 Filter 根本不会进链。重载链也顺一下：`getActivateExtension(URL, String key)`（第 303 行）→ `getActivateExtension(url, key, null)`（第 328 行）→ 把 `url.getParameter(key)` 用 `COMMA_SPLIT_PATTERN` 拆成 `String[] values` → 三参数版本（第 343 行）。`DefaultFilterChainBuilder` 传进来的 `key` 是 `SERVICE_FILTER_KEY` / `REFERENCE_FILTER_KEY`，`group` 是 `CommonConstants.PROVIDER` / `CONSUMER`，所以同一个 Filter 扩展在提供端和消费端是两次独立的 `getActivateExtension` 调用，各自得到一条链。另有 `getActivateExtensions()`（第 440 行）不过滤 URL 条件、直接返回全部 `@Activate` 扩展，`AbstractCluster.ClusterFilterInvoker` 取 `FilterChainBuilder` 列表用的就是它（`AbstractCluster.java:86`）——注意区分"取扩展点实现"和"取 Filter 实例"是两种不同调用。

排序由 `ActivateComparator.compare()`（`ActivateComparator.java:53`）决定：先处理 `before` / `after`（`a1.isLess(n2)` / `a1.isMore(n2)`，第 83-109 行），再 `return a1.order > a2.order ? 1 : -1`（第 111 行）；没有 `before` / `after` 时走第 114-123 行，order 相同用 `o1.getSimpleName().compareTo(o2.getSimpleName())` 兜底——注释说这是"为了避免两个 filter 在不同加载场景下顺序不一致"。所以不写 `order` 的两个 Filter，先后顺序由类名字母序决定，不要依赖 SPI 文件里的书写顺序。

把 `dubbo-rpc-api/.../filter/` 里的真实注解排一遍就很直观：`ContextFilter` 与 `ProfilerServerFilter` 是 `order = Integer.MIN_VALUE`（最外层，先建上下文/先开 profiler），`EchoFilter` -110000，`ClassLoaderFilter` -30000，`GenericFilter` -20000；一大批默认 order = 0 的（`@Activate(group = PROVIDER)` 的 `AccessLogFilter`、`ExceptionFilter`、`TimeoutFilter`，带 `value` 的 `TokenFilter`、`ExecuteLimitFilter`、`TpsLimitFilter`，以及 CONSUMER 侧的 `RpcExceptionFilter`、`ActiveLimitFilter`、`DeprecatedFilter`）；`GenericImplFilter` 是 `order = 20000`（消费端，靠后），`ClassLoaderCallbackFilter` 是 `order = Integer.MAX_VALUE`（最内层，最后收尾）。`TokenHeaderFilter` 干脆是空的 `@Activate` ——`group`、`value` 都不写，两端都生效，纯靠 URL 条件之外无条件挂载。`AdaptiveLoadBalanceFilter` 用了多行 `@Activate`（`before` / `after` 形式）。这个布局的意图是：上下文和 ClassLoader 最先就位，业务语义的 Filter 居中，回调与 ClassLoader 还原放最后。跨模块的监控类不在这个目录：`MonitorFilter` 现在位于 `dubbo-metrics/dubbo-metrics-default`（`@Activate(group = {PROVIDER})`，`implements Filter, Filter.Listener`），另有 `MetricsFilter` / `MetricsClusterFilter`。

6.4 内置 Filter 分类，以及 onResponse 到底是给谁用的

按职责把内置 Filter 归一下类，注意每一类的 `group` 和触发条件：**超时**—— `TimeoutFilter`（PROVIDER），Javadoc 明说 "Log any invocation timeout, but don't stop server from running"（第 36 行），`invoke` 里只 `return invoker.invoke(invocation)`，真正的判断放在 `onResponse`：从 `RpcContext.getServerAttachment().getObjectAttachment(TIME_COUNTDOWN_KEY)` 取 `TimeoutCountDown`，`countDown.isExpired()` 才打 `PROXY_TIMEOUT_REQUEST` warn（第 50-61 行）。它不掐断请求，掐断是消费端 timeout 与 `AsyncRpcResult` 的事，别混淆。**异常**—— `ExceptionFilter`（PROVIDER，`implements Filter, Filter.Listener`，第 50 行）在 `onResponse` 里对 `appResponse.hasException()` 做一串"直接放行"判断（checked exception、方法签名声明的异常、同 jar、JDK 异常、`RpcException`），其余用 `appResponse.setException(new RuntimeException(StringUtils.toString(exception)))`（第 113 行）包一层，避免客户端反序列化失败；CONSUMER 侧对称的是 `RpcExceptionFilter`（`@Activate(group = CONSUMER)`，`implements Filter, Filter.Listener`，第 35-36 行），它的 `invoke` 只透传，全部逻辑在 `onResponse`：`appResponse.hasException()` 且 `exception instanceof RpcException` 时 `throw (RpcException) exception`（第 44-51 行）。注意这是在 `onResponse` **里抛异常**——结合 `CallbackRegistrationInvoker` 的 catch 块（第 226-244 行），这个 `RpcException` 会被记日志并替换掉结果，最终冒泡给 `AbstractClusterInvoker`。这正是"业务异常改写"和"框架异常上抛"两条路径在消费端合流的地方，也再次说明 `onResponse` 不等于"一定成功返回"。**Token**—— `TokenFilter`（PROVIDER，value=TOKEN_KEY）比对 `inv.getObjectAttachmentWithoutConvert(TOKEN_KEY)`，不等就抛 `RpcException`（第 46-53 行），配 `TokenHeaderFilter`。**AccessLog**—— `AccessLogFilter`（PROVIDER），`dubbo.accesslog` 开关，`ConcurrentMap<String, Queue<AccessLogData>>` 缓冲、`LOG_OUTPUT_INTERVAL = 5000` 定时刷（第 77-91 行）。**泛化**—— `GenericFilter`（PROVIDER，order=-20000）把 `$invoke` 的参数反序列化成真实方法调用，`GenericImplFilter`（CONSUMER，order=20000）反向，两者都在 `onResponse` 里做结果序列化。**上下文**—— `ContextFilter`（PROVIDER，`Integer.MIN_VALUE`）在 `invoke` 里把 invocation attachments 灌进 `RpcContext.getServerAttachment()`，并在 `onResponse` / `onError` 里 `removeServerAttachment()` / `removeClientAttachment()` / `removeServiceContext()` 等（第 225-229 行）。**监控**——3.3 挪到 `dubbo-metrics-default` 的 `MonitorFilter` / `MetricsFilter` / `MetricsClusterFilter`。**限流**—— `ActiveLimitFilter`（CONSUMER，ACTIVES_KEY）用 `RpcStatus.beginCount` + `synchronized (rpcStatus)` 里 `rpcStatus.wait(remain)` 等待并发额度（第 56-80 行）；`ExecuteLimitFilter`（PROVIDER，EXECUTES_KEY）超了直接 `LIMIT_EXCEEDED_EXCEPTION`（第 50-55 行）；`TpsLimitFilter`（PROVIDER，TPS_LIMIT_RATE_KEY）管 TPS。剩下的几个补齐分类：`EchoFilter`（PROVIDER，`order = -110000`）拦截 `$echo` 回声测试；`DeprecatedFilter`（CONSUMER，value=DEPRECATED_KEY）对标记废弃的方法打日志；`ClassLoaderFilter`（PROVIDER，`order = -30000`）在调用前把线程上下文 ClassLoader 换成 `invoker.getUrl()` 对应的那个，`ClassLoaderCallbackFilter`（PROVIDER，`order = Integer.MAX_VALUE`）在最内层负责换回来——两个一前一后正好夹住整条链；`ProfilerServerFilter`（PROVIDER，`Integer.MIN_VALUE`）与 `ContextFilter` 并列最外层，负责调用链 profiler；`CompatibleFilter`、`TokenHeaderFilter`、`AdaptiveLoadBalanceFilter` 分别处理兼容、token 头透传与自适应负载均衡。数一下 `dubbo-rpc-api/.../filter/` 目录：`AccessLogFilter`、`ExecuteLimitFilter`、`ActiveLimitFilter`、`AdaptiveLoadBalanceFilter`、`ClassLoaderCallbackFilter`、`ClassLoaderFilter`、`GenericFilter`、`GenericImplFilter`、`ProfilerServerFilter`、`RpcExceptionFilter`、`TimeoutFilter`、`TokenFilter`、`TokenHeaderFilter`、`TpsLimitFilter`、`ContextFilter`、`DeprecatedFilter`、`EchoFilter`、`ExceptionFilter`、`CompatibleFilter` 共十九个，加上 `dubbo-`

`metrics-default` 的 `MonitorFilter` / `MetricsFilter` 与 `dubbo-cluster/.../filter/support/` 的四个 `ClusterFilter`，才是 3.3 的完整内置链。

`ListenableFilter`（`ListenableFilter.java:29`）在 3.3 已经 `@Deprecated`。它做的是"一次 RPC 一个 Listener"：内部 `ConcurrentMap<Invocation, Listener> listeners`，暴露 `listener(invocation)` / `addListener(invocation, listener)` / `removeListener(invocation)`，Javadoc（第 23-27 行）明确说"It's recommended to implement Filter.Listener directly for callback registration, check the default implementation, see ExceptionFilter"，并解释只有"不想在 RPC 之间共享 Listener 实例"时才用它。`FilterChainBuilder` 里两种形态都兼容：`filter instanceof ListenableFilter` 走 `listenableFilter.listener(invocation)` 并在 `finally` 里 `removeListener`，`filter instanceof FILTER.Listener` 直接强转（第 360-373、204-225 行）。

`BaseFilter.Listener`（`BaseFilter.java:32-53`）的两个回调语义必须分清：`onResponse(Result appResponse, ...)` 的 Javadoc 说"This method will only be called on successful remote rpc execution"，且 `appResponse` "can represent both normal result and exceptional result"（第 38 行）——也就是说**业务异常走 `onResponse`，要用 `appResponse.hasException()` 自己判**（`ExceptionFilter` 第 60 行、`TimeoutFilter` 第 49 行都是这个模式）；`onError(Throwable t, ...)` 是"framework exceptions, for example, TimeoutException, NetworkException, Exception raised in Filters"（第 45-51 行）。`BaseFilter.java:29-30` 还有一句对迁移最关键的提示："try to move logics previously defined in the 'finally block' to both onResponse and onError"。这就解释了为什么 `ActiveLimitFilter` 的 `RpcStatus.endCount` 同时写在 `onResponse`（第 88-96 行）和 `onError`（第 98-112 行）、`ContextFilter` 的 `removeXxx` 两边都写、`ExecuteLimitFilter` 的 `endCount` 两边都写（第 70-84 行）——少写一个，成功/失败必有一路漏计数或漏清理。`CallbackRegistrationInvoker` 的 `whenCompleteWithContext` 里 `t == null` 走 `onResponse`、`t != null` 走 `onError`，且循环是 `for (i = filters.size()-1; i >= 0; i--)`（第 200 行），回调顺序与 `invoke` 的进入顺序相反，也是洋葱的"出栈"。

为什么 Filter 里不能吞异常？从源码能给出四层理由：其一，`CopyOfFilterChainNode.invoke` 的 `catch` 里先 `listener.onError(...)` 再 `throw e`（第 358-374 行），链的设计前提就是异常会一直冒泡到最外层；吞掉后 `whenCompleteWithContext` 的 `t == null` 分支成立，所有 `onError` 都不触发，`RpcStatus.endCount`、`RpcContext.removeXxx`、异常日志全部错位。其二，集群容错完全靠这个信号：`FailoverClusterInvoker.doInvoke` 只在 `catch (RpcException e)` 且 `e.isBiz()` 时直接抛，其余（超时、网络）都当作可重试失败（`FailoverClusterInvoker.java:103-109`）；Filter 返回一个"看起来正常"的 `AsyncRpcResult`，消费端 `AbstractClusterInvoker` 就看不到失败，Failover 不会重试、Failback 不会补发、Failsafe 那类策略也失去意义。其三，第 350-357 行的 `AsyncRpcResult` 检查：吞异常后若顺手 `return new AppResponse()`，节点直接抛 `RpcException`，反而变成更难定位的框架错。其四，合法做法只有三种——不 catch；`throw new RpcException(...)` 原样/包装抛出；或用 `AsyncRpcResult.newDefaultAsyncResult(exception, invocation)` 把异常放进 Result（此时走 `onResponse` 的 `hasException()`，仍是"显式表达失败"）。`ExceptionFilter` 的 `appResponse.setException(...)`（第 113 行）是"改写异常"而不是"吞异常"的范例。至于 `FailsafeClusterInvoker` 的 `return AsyncRpcResult.newDefaultAsyncResult(null, null, invocation)`（第 61 行），那是**集群层**显式选择的容错策略，不是 Filter 该干的事——层次别搞反。

容易误解的地方：

1. **3.3 的 `ProtocolFilterWrapper` 里已经没有 `buildInvokerChain` 方法了**，而且类在 `dubbo-cluster/src/main/java/org/apache/dubbo/rpc/cluster/filter/`，不在 `dubbo-rpc-api`。2.x 那个"私有 `buildInvokerChain` + 匿名 `Invoker` 递归 + `invoker.invoke` 前后手写逻辑"是老知识；现在真正拼链的是 `DefaultFilterChainBuilder.buildInvokerChain`（第 44 行），节点是 `CopyOfFilterChainNode`，最外层还多了一个 `CallbackRegistrationInvoker`。
2. **`ListenableFilter` 已废弃，别再继承它写新 Filter**。3.3 推荐直接 `implements Filter, Filter.Listener`（`ExceptionFilter`、`TimeoutFilter`、`ActiveLimitFilter` 都是这么写的）。而且 `onResponse` 不是"业务成功回调"：它同样会收到带业务异常的 `appResponse`，框架异常才走 `onError`；只实现其中一个回调，计数和上下文清理必然漏一半。
3. **`order` 相同不代表顺序稳定，更不代表按 SPI 文件顺序**。`ActivateComparator` 在 order 相等时用 `getSimpleName().compareTo(...)` 兜底（`ActivateComparator.java:120`）；`@Activate` 的 `group` 只是筛选、`value` 是 URL 条件，两者都不写（如 `TokenHeaderFilter`）就默认两端都生效。
4. **Filter 和 ClusterFilter 是两条链**。`Filter.java:40-41` 写得很清楚：`Filter` 绑定单个 Provider 实例，`ClusterFilter` 在负载均衡选 Invoker 之前。把"每调用一次"的埋点写在 `Filter` 上，遇到 Failover 重试或多实例就会重复执行；反过来想按实例做统计，也不能挂到 `ClusterFilter`。

记住这一句：Filter 链的每一层只负责"进"和"出"，异常必须原样往外抛，回调统一由最外层的 CallbackRegistrationInvoker 收口。

第7章　集群容错：失败之后怎么办

一句话：invoke 定四步模板，doInvoke 决定失败后做什么。

为什么需要它：最典型的现场是"重试之后订单重复创建了"：一个创建订单的接口超时，消费端默认 `failover` 把这次超时判成失败又发了一遍，服务端其实已经写库成功，于是下了两单。顺着往下全是问题：`retries=2` 到底是 2 次还是 3 次？为什么业务异常不重试、超时却重试？`check=false` 配了之后，服务明明没起来，为什么启动不报错、一点调用就炸？`broadcast` 到底有几种"模式"？这章把 `dubbo-cluster/src/main/java/org/apache/dubbo/rpc/cluster/support/` 下的源码拆开：`AbstractClusterInvoker.invoke` 的四步模板、`select` / `doSelect` / `reselect` 怎么用 LoadBalance 又怎么避开重复选到同一台、八种策略在 `doInvoke` 里的真实差别、以及 `check=false` 把错误推迟到了哪一层。

核心结构：

概念	作用
`Cluster`	`@SPI(Cluster.DEFAULT)`，`DEFAULT = "failover"`（`Cluster.java:37`）；`@Adaptive join(directory, buildFilterChain)` 把 Directory 合成一个 Invoker
`ClusterInvoker`	消费端代理最终持有的 Invoker，持 `Directory`；`getRegistryUrl/getDirectory/isDestroyed/hasProxyInvokers/isServiceDiscovery`
`AbstractClusterInvoker`	模板类：`invoke()` 做 `list` → `checkInvokers` → `initLoadBalance` → `doInvoke`，后半段留给子类
`list()` / `select()` / `reselect()`	`getDirectory().list(invocation)` 取候选；`doSelect` 走 LoadBalance；`reselect` 在"未选过且可用"子集里重选
`initLoadBalance`	从 `invokers.get(0).getUrl()` 的 `loadbalance` 方法参数取扩展名，默认 `DEFAULT_LOADBALANCE`
`sticky` / `availableCheck`	`CLUSTER_STICKY_KEY` 默认 false；`CLUSTER_AVAILABLE_CHECK_KEY` 默认 true（`Constants.java:61`）
八种 `doInvoke`	failover / failfast / failsafe / failback / forking / broadcast / available / mergeable
`ReferenceConfig.checkInvokerAvailable`	启动期按 `check` 校验 `invoker.isAvailable()`，抛 `IllegalStateException`
`AbstractClusterInvoker.checkInvokers`	调用期空候选抛 `RpcException(NO_INVOKER_AVAILABLE_AFTER_FILTER)`

关键代码：

```
// AbstractClusterInvoker.java:345
@Override
public Result invoke(final Invocation invocation) throws RpcException {
    checkWhetherDestroyed();
    // ... 绑定 attachments（3.3 已注释掉，交给 ClusterFilter 链）
    List<Invoker<T>> invokers = list(invocation);      // ① 获取 Directory → 候选 Invoker 列表（含路由）
    checkInvokers(invokers, invocation);              // 为什么：空列表立刻抛 NO_INVOKER_AVAILABLE_AFTER_FILTER —— check=false
的报错点
    LoadBalance loadbalance = initLoadBalance(invokers, invocation);  // ② 选 LoadBalance（从 invokers[0] 的 URL 取，默认
random）
    RpcUtils.attachInvocationIdIfAsync(getUrl(), invocation);
    try {
        return doInvoke(invocation, invokers, loadbalance);  // ③ 选 Invoker + 调用 + ④ 处理结果，全部由子类决定
    } finally {
        InvocationProfilerUtils.releaseDetailProfiler(invocation);
    }
}

// AbstractClusterInvoker.java:187（doSelect 骨架节选）
private Invoker<T> doSelect(LoadBalance loadbalance, Invocation invocation,
                            List<Invoker<T>> invokers, List<Invoker<T>> selected) throws RpcException {
    if (CollectionUtils.isEmpty(invokers)) { return null; }
    if (invokers.size() == 1) {
        Invoker<T> tInvoker = invokers.get(0);
        checkShouldInvalidateInvoker(tInvoker);
        return tInvoker;                              // 为什么：只有一个 provider 时没必要走 LB
    }
    Invoker<T> invoker = loadbalance.select(invokers, getUrl(), invocation);
    boolean isSelected = selected != null && selected.contains(invoker);   // 为什么：重试时不能又选到刚才失败的那台
    boolean isUnavailable = availableCheck && !invoker.isAvailable() && getUrl() != null;
    if (isUnavailable) { invalidateInvoker(invoker); }
    if (isSelected || isUnavailable) {
        Invoker<T> rInvoker = reselect(loadbalance, invocation, invokers, selected, availableCheck);
        if (rInvoker != null) {
            invoker = rInvoker;
        } else {
            int index = invokers.indexOf(invoker);
            // Avoid collision：兜底取下一台，避免与 LB 结果碰撞
            invoker = invokers.get((index + 1) % invokers.size());   // AbstractClusterInvoker.java:218
        }
    }
    return invoker;
}
```

```
// FailoverClusterInvoker.java:59（骨架节选）
public Result doInvoke(Invocation invocation, final List<Invoker<T>> invokers, LoadBalance loadbalance)
        throws RpcException {
    List<Invoker<T>> copyInvokers = invokers;
    String methodName = RpcUtils.getMethodName(invocation);
    int len = calculateInvokeTimes(methodName);              // 为什么：retries=2 → len=3，最多 3 次而不是 2 次
    RpcException le = null;                                  // last exception
    List<Invoker<T>> invoked = new ArrayList<>(copyInvokers.size());  // 已调用过的，传给 select 做去重
    Set<String> providers = new HashSet<>(len);
    for (int i = 0; i < len; i++) {
        if (i > 0) {
            checkWhetherDestroyed();
            copyInvokers = list(invocation);                 // 为什么：重试前重新拉 Directory，感知 provider 上下线
            checkInvokers(copyInvokers, invocation);
        }
        Invoker<T> invoker = select(loadbalance, invocation, copyInvokers, invoked);
        invoked.add(invoker);
        RpcContext.getServiceContext().setInvokers((List) invoked);
        try {
            Result result = invokeWithContext(invoker, invocation);
            // ... 之前失败过则打 CLUSTER_FAILED_MULTIPLE_RETRIES warn
            return result;
        } catch (RpcException e) {
            if (e.isBiz()) { throw e; }                     // 为什么：业务异常是服务端明确拒绝，重试没有意义
            le = e;
        } catch (Throwable e) {
            le = new RpcException(e.getMessage(), e);
        } finally {
            // ... 未成功则 providers.add(invoker.getUrl().getAddress())
        }
    }
    // ... 抛出带 "Tried N times of the providers" 的 RpcException（第 116–126 行）
    throw new RpcException(le.getCode(), "Failed to invoke the method " + methodName + " ... ",
                           le.getCause() != null ? le.getCause() : le);
}

// FailoverClusterInvoker.java:129
private int calculateInvokeTimes(String methodName) {
    int len = getUrl().getMethodParameter(methodName, RETRIES_KEY, DEFAULT_RETRIES) + 1;  // DEFAULT_RETRIES = 2
 （CommonConstants.java:417）
    RpcContext rpcContext = RpcContext.getClientAttachment();
    Object retry = rpcContext.getObjectAttachment(RETRIES_KEY);
    if (retry instanceof Number) {                          // 为什么：允许单次调用在 RpcContext 里临时覆盖 retries
        len = ((Number) retry).intValue() + 1;
        rpcContext.removeAttachment(RETRIES_KEY);
    }
    if (len <= 0) { len = 1; }                              // 为什么：retries=-1 也不是"零次"，至少调一次
    return len;
}
```

拆解：

7.1 四步模板：invoke 只做前半段，doInvoke 决定一切

在进 `invoke()` 之前，先搞清楚这个 `ClusterInvoker` 是怎么来的，否则容易把第 6 章的两条链和第 7 章的策略混成一层。`Cluster` 是 `@SPI(Cluster.DEFAULT)`（默认 `"failover"`，`Cluster.java:34–37`），`@Adaptive <T> Invoker<T> join(Directory<T> directory, boolean buildFilterChain)`（第 47-48 行）。具体策略类极薄，比如 `FailoverCluster extends AbstractCluster`，`doJoin()`（`FailoverCluster.java:32–33`）只有 `return new FailoverClusterInvoker<>(directory);`。真正的外壳在 `AbstractCluster`（`support/wrapper/AbstractCluster.java:43`）：`join(directory, buildFilterChain)`（第 58 行）在 `buildFilterChain` 为 true 时返回 `buildClusterInterceptors(doJoin(directory))`，否则直接 `doJoin(directory)`。`buildClusterInterceptors`（第 45 行）先 `new ClusterFilterInvoker<>(clusterInvoker)`——这个内部类的构造器（第 82-97 行）遍历所有激活的 `FilterChainBuilder`，逐个 `builder.buildClusterInvokerChain(tmpInvoker, REFERENCE_FILTER_KEY, CommonConstants.CONSUMER)`；再叠 `InvocationInterceptorBuilder` 生成的 `InvocationInterceptorInvoker`（第 66-75 行），还可以通过 `CLUSTER_INTERCEPTOR_COMPATIBLE_KEY` 打开 2.7.x 兼容分支（第 48-53 行）。于是消费端代理最终持有的 `Invoker` 是「ClusterFilter 链 → InvocationInterceptor → 具体 `AbstractClusterInvoker` → 实例级 Filter 链 → 真实 Invoker」这么一串；`join()` 负责前半段，

`ProtocolFilterWrapper.refer()` 负责后半段。`Cluster.getCluster(scopeModel, name, wrap)`（`Cluster.java:54`）则通过 `getExtension(name, wrap)` 决定是否套 wrapper。多注册中心场景还有 `ZoneAwareClusterInvoker` 这类实现，`ClusterInvoker` 的 Javadoc（第 28-30 行）说明它会"先选一个子 ClusterInvoker，再在里面做 LB"。

`AbstractClusterInvoker.invoke()`（第 345 行）是固定的四步骨架。① **获取 Directory**：`list(invocation)`（`list()` 在第 451-453 行，就是 `return getDirectory().list(invocation)`），这一步里已经跑完路由链。紧接着 `checkInvokers(invokers, invocation)`（第 386-400 行）：空列表直接抛 `RpcException(RpcException.NO_INVOKER_AVAILABLE_AFTER_FILTER, "Failed to invoke the method ... No provider available ...")`。② **选 LoadBalance**：`initLoadBalance(invokers, invocation)`（第 466-478 行）用 `applicationModel.getExtensionLoader(LoadBalance.class).getExtension(invokers.get(0).getUrl().getMethodParameter(methodName, LOADBALANCE_KEY, DEFAULT_LOADBALANCE))`；注意参数取自**候选列表第一个 invoker 的 URL**而不是 consumer URL，空列表时退化到 `DEFAULT_LOADBALANCE`。`RpcUtils.attachInvocationIdIfAsync(getUrl(), invocation)` 补异步调用 ID。③④ **选 Invoker、调用、处理结果**：全部塞进抽象方法 `doInvoke(invocation, invokers, loadbalance)`（第 448-449 行声明），八种策略只覆写这一个方法。这个切分很重要：`checkInvokers` 在 `doInvoke` **之前**，所以"没有 provider"这种错误根本进不了任何容错循环。

`select()`（第 155 行）是子类统一入口：先读 `CLUSTER_STICKY_KEY`（默认 `DEFAULT_CLUSTER_STICKY = false`，`Constants.java:73`），若 `stickyInvoker` 仍在当前 `invokers` 里且 `availableCheck && stickyInvoker.isAvailable()`，直接复用（第 164-176 行）；否则 `doSelect(...)`，成功后记 `stickyInvoker = invoker`（第 180-182 行）。`availableCheck` 来自 `url.getParameter(CLUSTER_AVAILABLE_CHECK_KEY, DEFAULT_CLUSTER_AVAILABLE_CHECK)`（第 92 行，默认 true，`Constants.java:61`），不可用会 `invalidateInvoker(invoker)` → `getDirectory().addInvalidateInvoker(invoker)`（第 336-342 行）。真正发起调用的是 `invokeWithContext(invoker, invocation)`（第 402-418 行）：`setContext(invoker)` 把当前 Invoker 放进 `RpcContext.getServiceContext()`，`setRemote(invoker, invocation)`（第 424-429 行）设 `remoteAddress` / `remoteApplicationName`，`finally` 里 `clearContext(originInvoker)` 还原——第 6 章 Filter 里读到的 `RpcContext.getServiceContext().getRemoteHost()` 就是这里塞进去的。`invoke()` 第一行 `checkWhetherDestroyed()`（第 346、372-379 行）检查 `AtomicBoolean destroyed`，已销毁就直接抛 "is now destroyed! Can not invoke any more"；`destroy()`（第 128-132 行）用 CAS 置位后 `getDirectory().destroy()`。这两个方法配合 `isAvailable()`（第 114-120 行，优先看 `stickyInvoker`，否则 `getDirectory().isAvailable()`）构成了 `ClusterInvoker` 的生命周期契约——`ReferenceConfig.checkInvokerAvailable()` 里轮询的那个 `invoker.isAvailable()`，最终落到的就是这里。另有一个专供异步场景的 `invokeWithContextAsync(invoker, invocation, consumerUrl)`（第 437-446 行），Javadoc（第 431-434 行）说明"用线程池 fork 子线程时 ThreadLocal 传不过去"，所以要显式传 `consumerUrl`；Forking 和 Failback 用的都是它。

7.2 select 与 reselect：怎么不重复选到同一台

`doSelect()`（第 187 行）先处理两个短路：`invokers` 空返回 `null`；`invokers.size() == 1` 时 `checkShouldInvalidateInvoker` 后直接返回唯一那台，不走 LB。正常路径是 `loadbalance.select(invokers, getUrl(), invocation)`（第 199 行），然后判断两个触发 `reselect` 的条件（第 201-208 行）：`isSelected = selected != null && selected.contains(invoker)`（这次选中的正是之前失败过的那台），或 `isUnavailable = availableCheck && !invoker.isAvailable() && getUrl() != null`。`selected` 由各策略自己传：Failover 传累积的 `invoked`（第 77 行），Failfast / Failsafe / Failback 传 `null`（不重试，无需去重），Forking 传自己的 `selected` 列表来凑够不重复的 forks 个（第 83-87 行）。

`reselect()`（第 254 行）不是"随便换一台"，而是四级降级。第 1 级：从 `invokers` 里挑"可用且不在 `selected` 中"的（第 274-302 行）；规模由 `reselectCount` 控制，默认 `DEFAULT_RESELECT_COUNT = 10`（`CommonConstants.java:527`，第 71、94 行从全局配置读），当 `reselectCount < invokers.size()` 时改成 `ThreadLocalRandom` 随机取 `reselectCount` 个候选并去重——源码注释（第 270-273 行）直说这是为了"reduce retry times if invokers' size is huge, which may lead to long time hang up"，即大集群别为了重选把全表扫一遍。第 2 级：`reselectInvokers` 非空就 `loadbalance.select(reselectInvokers, getUrl(), invocation)`（第 305-307 行）——**在剩余候选上重新走一遍 LoadBalance**，所以权重、一致性哈希依旧生效。第 3 级：若为空，回头把 `selected` 里 `invoker.isAvailable()` 的加回来再选一次（第 309-324 行），注释写 "available first"。第 4 级：全都不行返回 `null`，`doSelect` 兜底 `invokers.get((index + 1) % invokers.size())`（第 214-218 行，注释 "Avoid collision"），且整段包在 `try/catch(Throwable)` 里，异常只记 `CLUSTER_FAILED_RESELECT_INVOKERS` 日志并提示"you can set cluster.availablecheck=false in url"（第 228-236 行）。

三个字段值得单独记住，因为它们都是"实例状态"，会跨调用影响选路：`stickyInvoker`（第 77 行，`volatile`）配合 `CLUSTER_STICKY_KEY` 实现粘滞连接——开启后只要上次的 Invoker 仍在候选里、`isAvailable()` 为真、且不在 `selected` 中，就跳过 LB 直接复用（第 164-176 行），`doSelect` 成功后回写（第 180-182 行）；`availableCheck`（第 69 行）决定 `doSelect` 里 `isUnavailable` 是否成立；`reselectCount`（第 71 行，来自 `ConfigurationUtils.getGlobalConfiguration(...).getInt(RESELECT_COUNT, DEFAULT_RESELECT_COUNT)`，第 94 行）决定重选时最多扫多少候选。注意 `invalidateInvoker()`（第 336 行）并不销毁 Invoker，只是 `getDirectory().addInvalidateInvoker(invoker)`，把"这台连不上"反馈给 Directory，让后续路由/列表把它降权——这是集群层与注册发现层之间唯一的反向通道。另外 `RpcStatus`（`ActiveLimitFilter` / `ExecuteLimitFilter` 用的那个类）是按 URL + methodName 统计 `active` / `total` / `failed` 的全局状态，和这里的 `stickyInvoker` 一样是共享可变状态，所以限流与粘滞都必须在压测下一起验，别只盯着单线程逻辑读源码。

7.3 八种策略在源码里的真实差别

`Cluster.DEFAULT = "failover"`（`Cluster.java:37`），所以不配就是 Failover。还要注意 `Cluster.getCluster(scopeModel, name, wrap)`（`Cluster.java:54`）里 `getExtension(name, wrap)` 的 `wrap` 默认为 true，因此拿到的往往不是裸的 `FailoverCluster`，而是被 `support/wrapper/` 下的 wrapper 包过的：`MockClusterWrapper.join()`（`MockClusterWrapper.java:37-38`）返回 `new MockClusterInvoker<>(directory, this.cluster.join(directory, buildFilterChain))`，把 mock/降级能力套在最外层；同目录还有 `ScopeClusterWrapper` / `ScopeClusterInvoker`。所以"集群容错"最外层其实还有一层 mock 判断，读日志时看到 `MockClusterInvoker` 不要以为配错了策略。八个 `*Cluster` 类本身都极薄，唯一动作就是 `doJoin()` 里 new 对应的 `*ClusterInvoker`，所有差异都在 `doInvoke` 里。

Failover（`FailoverClusterInvoker.java:59`）：循环 `len` 次，每次 `select(loadbalance, invocation, copyInvokers, invoked)`；`i > 0` 时重新 `list(invocation)` + `checkInvokers` 以感知 provider 上下线（第 69-76 行，注释 "Reselect before retry to avoid a change of candidate invokers"）。`catch (RpcException e)` 里只有 `e.isBiz()` 立刻抛，其余全部赋给 `le` 继续循环（第 103-109 行）；循环耗尽抛带 "Tried N times of the providers" 的 `RpcException`（第 116-126 行）。**非幂等风险就在这里**：`invokeWithContext` 已经把请求发出，超时只代表"不知道服务端有没有执行完成"，重试可能让一次创建变成两次。`retries=2` 经 `calculateInvokeTimes` 的 `+1` 变成 3 次（`DEFAULT_RETRIES = 2`，`CommonConstants.java:417`），`retries=0` 才是不重试；`retries=-1` 被 `len <= 0 → len = 1`（第 137-139 行）兜成 1 次，不是 0 次。第 131-136 行还允许在 `RpcContext.getClientAttachment()` 里临时塞一个 `retries` attachment 单次覆盖，排查"为什么这次没重试"别漏看这里。

Failfast（`FailfastClusterInvoker.java:44`）：只 `select` 一次，`invokeWithContext` 失败就包一层 `RpcException` 抛出，`isBiz()` 直接抛原异常——非幂等写操作的常规选择。**Failsafe**（`FailsafeClusterInvoker.java:49`）：`catch (Throwable e)` 后记 `CLUSTER_ERROR_RESPONSE` 日志，`return AsyncRpcResult.newDefaultAsyncResult(null, null, invocation)`（第 61 行），显式吞掉异常返回 null，用于审计、日志等可丢的调用。**Failback**（`FailbackClusterInvoker.java:118`）：失败后 `addFailed(...)`（第 85 行）用 `HashedWheelTimer`（`failback-cluster-timer`，1 秒 tick、32 ticks、容量 `failbackTasks` 默认 `DEFAULT_FAILBACK_TASKS = 100`，`Constants.java:23`）在 `RETRY_FAILED_PERIOD = 5` 秒后调度 `RetryTimerTask` 后台重试，`retries` 默认 `DEFAULT_FAILBACK_TIMES = 3`（`CommonConstants.java:419`）；主线程立刻返回 `AsyncRpcResult.newDefaultAsyncResult(null, null, invocation)`（第 139 行）。因为后台重试跨线程、ThreadLocal 传不过去，这里必须用 `invokeWithContextAsync(invoker, invocation, consumerUrl)`（第 126 行，注释第 124-125 行明说原因）。适合通知类。

Forking（`ForkingClusterInvoker.java:72`）：`forks` 默认 `DEFAULT_FORKS = 2`（`Constants.java:25`），`timeout` 默认 `DEFAULT_TIMEOUT = 1000`（`CommonConstants.java:147`）；`forks <= 0 || forks >= invokers.size()` 就打全部，否则用 `select` 凑 `forks` 个互不相同的 Invoker（第 78-89 行）。并发靠 `CompletableFuture.supplyAsync(..., executor)`（`executor` 是 `FrameworkExecutorRepository.getSharedExecutor()`，第 62-67 行）投到线程池，结果汇入 `LinkedBlockingQueue<Object> ref = new LinkedBlockingQueue<>(1)`（第 92 行）：任一路成功 `ref.offer(v)`，全失败时 `count.incrementAndGet() >= selected.size()` 才把异常 offer 进去（第 103-112 行）。主线程 `ref.poll(timeout, TimeUnit.MILLISECONDS)`（第 115 行）拿第一个结果。注意一个真实行为：**超时后 `poll` 返回 `null`，源码直接 `return (Result) ret`（第 126 行）返回 null，不抛异常**——别把 Forking 超时当成 `TimeoutException` 来 catch。另外 `invokeWithContextAsync` 在这里也是必需的（第 100 行），并因 `RpcContext` 跨线程问题在 `finally` 里 `RpcContext.getClientAttachment().clearAttachments()`（第 133-136 行）。

Broadcast（`BroadcastClusterInvoker.java:54`）：遍历全部 `invokers` 逐个调用，每个都 `new RpcInvocation(...)` 复制一份（第 80-91 行），并用 `RpcContext.RestoreContext` 在非最后一次调用后 `restore()`（第 78、121-124 行），避免上一个 provider 改的上下文串到下一个。失败计数 `failIndex`，阈值 `failThresholdIndex = invokers.size() * broadcastFailPercent / MAX_BROADCAST_FAIL_PERCENT`（第 74 行），`broadcast.fail.percent` 默认 100（第 64 行）；循环结束只要 `exception != null` 就 `throw exception`（第 142 行）。**关于"两种模式"要纠正**：3.3 源码里根本没有 `failover` / `failsafe` 两个模式开关，只有一个 0-100 的百分比旋钮。类注释（第 60-63 行）说 "100 means that an exception will be thrown last, and 0 means that as long as an exception occurs, it will be thrown"，对应的是"全部调完才抛"（宽松/容错）与"一出错就抛"（严格）两种**设计意图**。但实现上 `failThresholdIndex` 是整数、判断用 `==`（第 104、118 行），而 `failIndex` 从 0 起、`failIndex++` 后变成 1——`broadcast.fail.percent=0` 时阈值算出来是 0，`1 == 0` 永远不成立，实际不会提前 break，仍是遍历完再抛。percent 落在 1~99 时（如 5 台配 20 → 阈值 1）才会在第 1 台失败后停。读文档别当成精确实现。

Available（`AvailableClusterInvoker.java:39`）：完全不用 LoadBalance，顺序遍历找第一个 `invoker.isAvailable()` 就调，全不可用抛 `RpcException("No provider available in " + invokers)`。**Mergeable**（`MergeableClusterInvoker.java:62`）：`merger` 取自 `getUrl().getMethodParameter(methodName, MERGER_KEY)`；没配就只调第一个可用 group，并对 `e.isNoInvokerAvailableAfterFilter()` 记录后继续试下一个 group（第 66-82 行）；配了就对每个 invoker 发一个 `setAttachment(ASYNC_KEY, "true")` 的 `RpcInvocation`（第 96-99 行），`asyncResult.get(Integer.MAX_VALUE, TimeUnit.MILLISECONDS)` 等齐（第 122 行），再按 `merger` 指定的 `Merger` SPI 或返回类型上的同名方法逐个 `method.invoke` 合并（第 148-171 行）。用于多 group 结果聚合。

7.4 check=false 在集群层的表现

`check` 不是 `AbstractClusterInvoker` 的字段，它属于 `ReferenceConfig`，只影响"什么时候发现没有 provider"。`ReferenceConfig.init(boolean check)`（`ReferenceConfig.java:332`）在 `createProxy` 之后有 `if (check) { checkInvokerAvailable(0); }`（第 392-394 行）；`checkInvokerAvailable(long timeout)`（第 716 行）开头 `if (!shouldCheck()) { return; }`（第 717-719 行），否则轮询 `invoker.isAvailable()` 到 `checkDeadline`，仍不可用就抛 `IllegalStateException("Failed to check the status of the service ... No provider available ...")`（第 747-756 行）。所以 `check=true`（默认）会让"没有 provider"在**启**

动/首次 `get()` 阶段就变成 Spring 容器刷新失败。入口不止一个：`ReferenceConfig.get(boolean check)`（第 230 行）先判 `destroyed`，再 `init(check)`（第 244 行）；`get()` 那条路径还会在 `init` 之后额外 `checkInvokerAvailable(timeout)`（第 256 行），而 `get()` 的 catch 块（第 287-288 行）专门识别 `IllegalStateException` 做清理。`checkInvokerAvailable` 的 `do/while` 每 100ms 轮询一次 `invoker.isAvailable()`（第 727-735 行），超时后打 `REGISTRY_EMPTY_ADDRESS` warn 并给出 "After Nms reties, finally failed"——这也是排查"启动很慢"时要看的地方：`check` 会实打实地把线程阻塞在等待 provider 上。

`check=false` 只是跳过这次启动期校验，代理照常创建。错误被推迟到第一次真实调用：`AbstractClusterInvoker.invoke()` → `list(invocation)` 返回空 → `checkInvokers(invokers, invocation)`（第 358 行）抛 `RpcException(NO_INVOKER_AVAILABLE_AFTER_FILTER, ...)`（第 386-400 行）。三个后果值得记牢：异常类型从 `IllegalStateException` 变成 `RpcException`，可以被业务 `catch`；因为 `checkInvokers` 在 `doInvoke` **之前**，空 provider 时连 Failover 的循环都进不去，重试次数一次都用不上，日志里不会出现 "Tried 3 times"；而 `MergeableClusterInvoker` 对 `e.isNoInvokerAvailableAfterFilter()` 是特判的——记 warn、继续尝试下一个 group（第 101-108 行），不直接抛。

集群层自己确实有一个相近但无关的开关：`availableCheck`（`cluster.availablecheck`，默认 true，第 92 行），它决定 `doSelect` 里要不要用 `invoker.isAvailable()` 判定并把失效 Invoker 加进 `addInvalidateInvoker`（第 203-207 行）。`reselect` 失败时日志里那句 "you can set cluster.availablecheck=false in url"（第 234 行）指的就是它，和 `check=false` 完全不是一回事。

最后把视角拉回第 5 章的部署器：`DefaultModuleDeployer.startSync()`（`DefaultModuleDeployer.java:162`）在 `exportServices()`、`referServices()` 之后还有 `checkReferences()`（第 197 行），方法体（第 455-461 行）是 `Optional<ModuleConfig> module = configManager.getModule(); long timeout = module.map(ModuleConfig::getCheckReferenceTimeout).orElse(30000L); for (ReferenceConfigBase<?> rc : configManager.getReferences()) { referenceCache.check(rc, timeout); }`。也就是说除了 `ReferenceConfig` 自己那次启动期校验，`ModuleDeployer` 在模块启动完成时还会按 `checkReferenceTimeout`（默认 30000ms）对模块内所有 reference 做一次集中检查。这两道检查加上调用期 `AbstractClusterInvoker.checkInvokers` 的第三道，构成"启动期 / 模块启动完成 / 首次调用"三个可能暴露"没有 provider"的时点；`check=false` 只是绕过第一道，第二道仍可能把错误提前到容器刷新阶段。排查"明明配了 `check=false` 却还是启动失败"时，要往 `ModuleConfig.checkReferenceTimeout` 和 `checkReferences()` 这条线查，而不是只盯着 `@DubboReference(check = false)`。

容易误解的地方：

1. **`retries=2` 是最多 3 次**。`calculateInvokeTimes()` 里 `getMethodParameter(methodName, RETRIES_KEY, DEFAULT_RETRIES) + 1`（第 130 行），且 `DEFAULT_RETRIES = 2`（`CommonConstants.java:417`），所以默认值就是 3 次。`retries=0` 才是"不重试"；`retries=-1` 被 `len <= 0 → len = 1`（第 137-139 行）兜成 1 次，不是 0 次。另外 `RpcContext` 里临时塞的 `retries` attachment 会单次覆盖配置（第 131-136 行）。
2. **Failover 只对幂等操作安全**。它只放过 `RpcException.isBiz()` 的业务异常（第 104-106 行），超时/网络异常一律重试；而超时不代表服务端没执行完。非幂等写要么 `retries=0`，要么换 `failfast`。别指望"服务端抛个异常就不会重试"——业务异常不重试，但包装成普通 `RuntimeException` 的框架异常会重试。
3. **`check=false` 不是"忽略错误"**，只是把报错从启动期（`IllegalStateException`，Spring 刷新失败）推迟到首次调用（`RpcException(NO_INVOKER_AVAILABLE_AFTER_FILTER)`）。由于 `checkInvokers` 位于 `doInvoke` 之前（第 358 行），此时任何集群策略都还没开始工作，Failover 帮不上忙。别把它和 `cluster.availablecheck` 混为一谈。
4. **3.3 的 Broadcast 没有 "failover/failsafe 两种模式"**，只有 `broadcast.fail.percent`（0-100，默认 100，第 44、64 行）；而且 `failIndex == failThresholdIndex` 的 `==` 比较加上 `failIndex` 从 1 起算（第 75、104、118 行），percent=0 时阈值是 0、不会提前 break，实际仍是遍历完再抛。源码注释描述的是设计意图，不是精确语义。

记住这一句：容错策略只决定 doInvoke 里"失败之后做什么"，而失败信号能不能传到这里，取决于异常有没有被 Filter 或 Directory 吞掉。

第8章　负载均衡：四种算法和它们的代价

一句话：选一台机器不难，难的是为"选错"付出多少运行时成本。

为什么需要它：三个 provider，A 是 32 核新机器 weight=300，B/C 是 8 核 weight=100。上线后 A 的 RT 反而最高、B 被打满；再过一天扩容出 D，D 一启动就一片超时。第一个问题的答案是"你的权重没生效"，第二个的答案是"warmup 生效了，但你以为它是故障"。这一章只回答一件事：一次 `select` 里到底算了什么，每种算法为了"算得准"在进程里留下了什么状态。

核心结构：

概念	作用
`AbstractLoadBalance.select`	空返回 null、单元素直接返回，其余交给 `doSelect`
`getWeight`	方法级 `weight` 参数 + 基于 `timestamp` 的预热降权
`calculateWarmupWeight`	把 uptime 按比例折算成 `[1, weight]`
`RpcStatus.active`	进程内、方法级的在途调用数，LeastActive / ShortestResponse 的数据源
`WeightedRoundRobin.current`	平滑轮询的"当前权值"，选完扣掉 totalWeight
`SlideWindowData`	两个基线偏移量 + `avg × (active+1)` 的预估响应时间
`ConsistentHashSelector`	`TreeMap<Long, Invoker>` 组成的哈希环
`hash.nodes` / `hash.arguments`	虚拟节点数（默认 160）/ 参与哈希的参数下标（默认 "0"）

关键代码：

```
// AbstractLoadBalance.java:49-52  预热降权的全部数学
static int calculateWarmupWeight(int uptime, int warmup, int weight) {
    int ww = (int) (uptime / ((float) warmup / weight));   // = uptime * weight / warmup
    return ww < 1 ? 1 : (Math.min(ww, weight));            // 下限 1，上限原权重
}

// AbstractLoadBalance.java:75-102
protected int getWeight(Invoker<?> invoker, Invocation invocation) {
    int weight;
    URL url = invoker.getUrl();
    if (invoker instanceof ClusterInvoker) {
        url = ((ClusterInvoker<?>) invoker).getRegistryUrl();      // 多注册中心时权重是注册中心维度的
    }
    if (REGISTRY_SERVICE_REFERENCE_PATH.equals(url.getServiceInterface())) {
        weight = url.getParameter(WEIGHT_KEY, DEFAULT_WEIGHT);     // 注册中心自身被引用：权重在 url 上
    } else {
        weight = url.getMethodParameter(RpcUtils.getMethodName(invocation), WEIGHT_KEY, DEFAULT_WEIGHT);
        if (weight > 0) {
            long timestamp = invoker.getUrl().getParameter(TIMESTAMP_KEY, 0L);  // provider 注册时写入的启动时间
            if (timestamp > 0L) {
                long uptime = System.currentTimeMillis() - timestamp;
                if (uptime < 0) {
                    return 1;                                      // 时钟不同步：当成"最不可信"，直接给最低权重
                }
                int warmup = invoker.getUrl().getParameter(WARMUP_KEY, DEFAULT_WARMUP); // 默认 10 分钟
                if (uptime > 0 && uptime < warmup) {
                    weight = calculateWarmupWeight((int) uptime, warmup, weight);
                }
            }
        }
    }
    return Math.max(weight, 0);
}
```

```
// RoundRobinLoadBalance.java:102-129  平滑加权轮询的全部状态机
for (Invoker<T> invoker : invokers) {
    String identifyString = invoker.getUrl().toIdentityString();
    int weight = getWeight(invoker, invocation);
    WeightedRoundRobin weightedRoundRobin = ConcurrentHashMapUtils.computeIfAbsent(map, identifyString, k -> {
        WeightedRoundRobin wrr = new WeightedRoundRobin();
        wrr.setWeight(weight);
        return wrr;
    });
    if (weight != weightedRoundRobin.getWeight()) {
        weightedRoundRobin.setWeight(weight);          // 权重变了会把 current 归零，见"容易误解"
    }
    long cur = weightedRoundRobin.increaseCurrent();   // current += weight
    weightedRoundRobin.setLastUpdate(now);
    if (cur > maxCurrent) {                            // 贪心取当前最大
        maxCurrent = cur;
        selectedInvoker = invoker;
        selectedWRR = weightedRoundRobin;
    }
    totalWeight += weight;
}
if (invokers.size() != map.size()) {                   // 地址数变了才做一次清理
    map.entrySet().removeIf(item -> now - item.getValue().getLastUpdate() > RECYCLE_PERIOD); // 60s
}
if (selectedInvoker != null) {
    selectedWRR.sel(totalWeight);                      // current -= totalWeight
    return selectedInvoker;
}
```

拆解：

1. 入口只有三行，权重却有一整套降级逻辑

`select` 本身短得没什么可讲：空返回 `null`、`size() == 1` 直接返回、其余交给 `doSelect`。单元素短路不是省事，是省算力——只有一个候选时不该为它建 `weights` 数组、建哈希环、或者读 `RpcStatus`。

真正有信息量的是 `getWeight` 的三个分支。第一，`invoker instanceof ClusterInvoker` 时取 `getRegistryUrl()`——多注册中心场景下，权重表达的是"这个注册中心整体该分多少流量"，不是某台 provider 的配置。第二，`REGISTRY_SERVICE_REFERENCE_PATH` 特判，此时权重挂在 url 上而非方法参数上。第三才是正常路径：`getMethodParameter(methodName, "weight", 100)`，注意是**方法级**参数，`@DubboReference(methods = @Method(name = "x", weight = 200))` 才能命中，配在接口级只是兜底。

预热降权是大多数人不知道的那段。`timestamp` 取自 `invoker.getUrl()`（provider 注册时写入的进程启动时间），`uptime = now - timestamp`，`warmup` 默认 `DEFAULT_WARMUP = 10 * 60 * 1000`。一个 weight=300 的实例，启动 1 分钟时权重只有 30，5 分钟 150，满 10 分钟才回到 300。理由很物理：JVM 刚起来，JIT 没编译、连接池是空的、本地缓存冷的，按满权重灌流量等于拿它做压力测试。

`calculateWarmupWeight` 就是一行比例折算 `uptime * weight / warmup`，再 clamp 到 `[1, weight]`——下限 1 保证新实例不会被彻底饿死，上限保证不会超过配置值。

有个反直觉分支值得单独记：`uptime < 0` 时直接 `return 1`。消费者时钟比提供者慢时算出来是负数，Dubbo 不认为"刚启动"，而是当成"时间戳不可信"处理，直接给最低权重。这是把时钟漂移当成未知状态，代价是——一台时钟快 20 分钟的机器，在别人眼里永远是权重 1。

还要注意它是**每次 select 都重算**的，不是启动时算一次存起来。所以权重是一条随时间上升的曲线，不是阶跃。

2. Random 与 RoundRobin：一个无状态，一个要记账

`RandomLoadBalance` 在 3.3 里多了一个 `needWeightLoadBalance` 短路：如果第一个 invoker 的 url 上既没有显式 `weight` 参数、也没有 `timestamp`，直接 `ThreadLocalRandom.current().nextInt(length)` 均匀随机，连 `weights` 数组都不建。这解释了为什么"没配权重的集群"里 Random 便宜得几乎免费。

为什么是 `ThreadLocalRandom` 而不是 `Random`：`java.util.Random.next()` 的 seed 存在一个 `AtomicLong` 里，每次取随机数都是一次 CAS，高并发下 CAS 失败重试，seed 变成全局竞争点。`ThreadLocalRandom` 把 seed 放在 `Thread` 对象字段里，`nextInt` 只读写本线程内存，无竞争、无伪共享。负载均衡是每次 RPC 都走的路径，这个差别在几万 QPS 下是能被 profiler 抓出来的。

权重区间法的实现细节比想象中讲究：`weights[i]` 存的不是第 i 个 invoker 的权重，而是**前缀和**（`totalWeight += weight; weights[i] = totalWeight;`），同时用 `totalWeight != weight * (i + 1)` 顺手判断"是不是全都一样重"。选点就变成"在 `[0, totalWeight)` 取一个 offset，找

第一个前缀和大于 offset 的下标"。`length <= 4` 线性扫，`> 4` 走 `Arrays.binarySearch`。

二分那段有个只有读源码才会注意的点：命中时（`offset` 恰好等于某个前缀和）要 `while (weights[i + 1] == offset) i++;` 再 `i++`，因为区间是左闭右开、offset 落在相等前缀和的最后一个上。而这里不会数组越界，理由是 `nextInt(totalWeight)` 保证 `offset < totalWeight = weights[length - 1]`，所以精确命中点永远不可能是最后一个元素。一个看起来会崩的 `weights[i + 1]`，被上一行的取值范围救了。

`RoundRobinLoadBalance` 在 3.x 已经完全不是"取模"了。2.6 那套 `AtomicLong sequence` + `mod` 的问题不在正确性，在**突发**：A=5、B=1、C=1 时，如果按 `totalWeight` 取模再映射区间，序列是 A,A,A,A,A,B,C,...，前五个请求全砸在 A 上，B/C 要等到第六第七个才出现。按 `length` 取模则更糟——权重白配了。

3.3 用的是平滑加权轮询（和 nginx 的 smooth WRR 同构）：`methodWeightMap` 是 `Map<serviceKey.method, Map<identityString, WeightedRoundRobin>>`，每个节点一个 `current`。每轮所有节点 `current += weight`，取最大者，被选中的 `current -= totalWeight`。A=5、B=1、C=1 走一遍：

`(0,0,0) → (5,1,1) 选 A → (-2,1,1) → (3,2,2) 选 A → (-4,2,2) → (1,3,3) 选 B → (1,-4,3) → (6,-3,4) 选 A → (-1,-3,4) → (4,-2,5) 选 C → (4,-2,-2) → (9,-1,-1) 选 A → (2,-1,-1) → (7,0,0) 选 A → (0,0,0)`。

七次一轮，序列 A,A,B,A,C,A,A。A 拿 5 次，但被均匀插开，最大连续只有 2 次。这就是"平滑"的含义：比例对，且短窗口内也均匀。

两个实现细节要记牢。一是 `current` 虽然是 `AtomicLong`，但"遍历所有节点读 current → 比较 → `sel`"这整段不是原子的，并发下两个线程可能同时认为同一个节点最大。Dubbo 接受这个近似：`sel` 会扣减，下一轮自然纠正，代价是极端并发下比例不精确。二是 map 的回收：只有 `invokers.size() != map.size()` 时才 `removeIf` 掉 `lastUpdate` 超过 `RECYCLE_PERIOD`（60000ms）的条目，这是地址下线后 WRR 状态的清理路径，不加就是内存泄漏。

3. LeastActive 与 ShortestResponse：同一套骨架，两种"谁更空"

两者的代码结构几乎逐行对应：一个 `leastIndexes/shortestIndexes` 数组、一个 `leastCount/shortestCount`、一个 `sameWeight` 标志，结尾三步也一模一样——唯一候选直接返回；权重不同按权重随机；权重相同均匀随机。差别只在"用什么指标排序"。

LeastActive 的指标是 `RpcStatus.getStatus(invoker.getUrl(), methodName).getActive()`。这个 `active` 由 `beginCount` / `endCount` 维护：`beginCount` 对 method 级 status 做 CAS 自增（带 `max` 上限判断，超限返回 false，被 `ActiveLimitFilter` 用来拒绝请求），再对 service 级 status 无条件 `incrementAndGet`；`endCount` 对两级都 `decrementAndGet`，顺带累加 `total`、`totalElapsed`、`succeededMaxElapsed`。所以它是**本进程、本方法**此刻还没收到响应的调用数——你有 10 个消费者，每个看到的 active 都只是自己那一份，没有任何全局聚合。

`RpcStatus` 的 key 是 `url.toIdentityString()`，不是 `ip:port`。这意味着同一台机器上的同一个方法，如果因为 `version`、`group` 或某个参数不同被 refer 成两个 invoker，就会落到两个独立的 `RpcStatus` 上，各自的 active 互不相加。排查"活跃数明显偏低"时，先确认地址有没有被参数拆成多份。

为什么活跃数相同还要再随机？因为冷启动或低 QPS 时所有实例 active 都是 0。如果按列表顺序取第一个满足条件的，流量会全压在第一台，第二台永远拿不到请求、永远 active=0、永远排在后面——正反馈直接把均衡打死。代码里 `leastCount == 1` 直接返回是安全的（只有一个最少），但只要 `leastCount > 1` 就必须用权重或随机打散。这不是锦上添花，是算法正确性的一部分。

ShortestResponse 把指标换成"预估响应时间"：`SlideWindowData.getEstimateResponse() = 窗口内成功调用平均耗时 × (active + 1)`。`+1` 是把自己这次算进去：一台正在处理 10 个请求、平均 5ms 的机器预估 55ms，一台空闲但平均 20ms 的机器预估 20ms，选后者。这比 LeastActive 多了一层"这台机器是不是本来就很慢"的判断。

"滑动窗口"这个词容易骗人。它不是时间轮，也不是环形缓冲区，而是**两个基线偏移量**：`SlideWindowData` 存 `succeededOffset` 和 `succeededElapsedOffset`，`getSucceededAverageElapsed()` 用 `(当前累计 - 基线) / (当前成功数 - 基线成功数)`，`reset()` 把当前累计值写成新基线。触发条件是 `now - lastUpdateTime > slidePeriod`（默认 `30_000`，可由 `shortestResponseSlidePeriod` 配置），并用 `AtomicBoolean onResetSlideWindow` 的 CAS 保证同一时刻只有一个线程提交，任务丢到共享 executor 异步跑。所以窗口长度是"上次 reset 到这次 select"的近似值，而且是**惰性**的——没有请求进来就不会 reset。

它有两个真实缺陷，读代码就能看出来：一是只统计成功调用（`getSucceeded` / `getSucceededElapsed` 都是 total 减 failed），一台疯狂失败但"成功的那些很快"的机器会显得最优；二是冷启动时平均耗时是 0，所有实例 estimate 都退化成由 active 主导，等价于 LeastActive。

4. ConsistentHash：160 个点，一次 MD5

构造 `ConsistentHashSelector` 时：`replicaNumber = hash.nodes`（默认 160），循环 `replicaNumber / 4` 次，每次对 `address + i` 做 MD5，一次 MD5 出 16 字节，`hash(digest, h)` 取 4 个 32 位值，所以 40 × 4 = 160 个环上点。**160 的来历就在这里**：不是拍脑袋的经验数，是"一次 MD5 能切出 4 个节点"这个实现细节乘以 40 次循环。改 `hash.nodes=100` 就是 25 次循环、100 个点。

虚拟节点的作用是压方差。10 台真实机器只放 10 个点时，环上的间距由哈希运气决定，某台可能独占 30% 的环；加到 160 个点后，每台的平均份额趋近 1/N，加一台机器时迁移量才接近理论的 1/(N+1)。这是 Karger 那套一致性哈希的原始论证，memcached 客户端用同样的 160。

`select` 路径：`toKey(args)` 按 `argumentIndex` 拼接参数（`hash.arguments` 默认 `"0"`，只取第一个参数），MD5 后取 `hash(digest, 0)`，`TreeMap.ceilingEntry` 顺时针找第一个 `>=` 该 hash 的节点，找不到就 `firstEntry` 回绕。环上 key 用的是 `invoker.getUrl().getAddress()`

（ip:port），不是完整 url——同一台机器换端口会被当成新节点。

`hash` 本身也值得看一眼：每个字节 `& 0xFF`，最后 `& 0xFFFFFFFFL`，全程无符号扩展。Java 的 `byte` 是有符号的，直接移位会把高位置 1，环上所有点挤到负数区间，`TreeMap` 的分布就废了。这行掩码不是装饰。

selector 缓存在 `selectors` 里，key 是 `serviceKey.method`，失效判据是 `invokers.hashCode() != selector.identityHashCode`。`List.hashCode` 只看元素和顺序，所以地址列表一增删或重排就重建整个环，成本是 N × 40 次 MD5。地址频繁抖动的集群里，这是 ConsistentHash 最贵的地方——比每次 select 一次 MD5 + 一次 TreeMap 查找贵得多。

它适合"同一参数打到同一台"：本地缓存命中、有状态计算、批量任务分片。代价同样明确：热点 key 无法分散（同一个 userId 永远同一台，哪怕那台已经满了），以及节点变化时**部分** key 会迁移——"一致性"的意思是只迁移 1/N，不是不迁移。

把五种算法的单次成本摊开看，选型就没那么玄了：Random 在无权重配置时是一次 `ThreadLocalRandom`，有权重时是 n 次 `getWeight`（每次都在 url 上做参数查找）+ 一次二分；RoundRobin 是两次 map 查找（外层按 `serviceKey.method`、内层按 `identityString`）+ n 次 `AtomicLong` 读改写；LeastActive 是 n 次 `RpcStatus` 两级 map 查找 + 一次 `AtomicInteger` 读；ShortestResponse 在它之上再加一次 `SlideWindowData` 查找，以及每 30 秒一次异步全量 reset；ConsistentHash 是一次 MD5 + 一次 `TreeMap` 查找，但地址一变就要重建整个环。

3.3 还多了一个 `AdaptiveLoadBalance`（`NAME = "adaptive"`），走的是 P2C：随机取两个 invoker（`pos1 = nextInt(length)`、`pos2 = nextInt(length - 1)`，`pos2 >= pos1` 时加一避免重合），比它们的 load 取小者。它的指标不是 `RpcStatus`，而是 `AdaptiveMetrics` 按 `address:protocolServiceKey` 聚合的负载，还带超时权重。它不是"第六种经典算法"，而是 LeastActive 思路在"看全局指标"方向上的延伸——代价是要维护一套指标采集，所以默认仍然是 `random`（`@SPI(RandomLoadBalance.NAME)`）。

容易误解的地方：

1. **"预热降权是 bug，新机器权重被砍了"**。不是。`timestamp` 来自 provider 注册时写入的启动时间，`getWeight` 每次 select 都重算，所以权重是一条 10 分钟爬升的曲线。真要关掉，配 `warmup=0` 或让 provider 不写 `timestamp`；别去改 `calculateWarmupWeight`。
2. **"LeastActive 统计的是全集群并发"**。是 `RpcStatus` 里本进程、本方法的在途计数，粒度到 method。10 个消费者各自看到 1/10 的真相，所以它只能反映"我这边压了多少"，不能反映 provider 真实负载。
3. **"RoundRobin 就是按权重取模"**。3.x 是平滑加权轮询，状态在 `methodWeightMap` 里，且并发下读 `current` 不是原子的，比例是近似值。另外 `setWeight` 会把 `current` 归零——权重一变，节奏就抖一下，这是设计取舍不是缺陷。
4. **"`consistenthash` 保证同一个 key 永远同一台"**。只在 invoker 列表不变时成立；而且默认只看第一个参数，参数对象靠 `toString()` 拼接（`toKey` 里就是 `buf.append(args[i])`），自定义对象没重写 `toString` 等于所有请求哈希到同一个字符串。

记住这一句：负载均衡只解决"这一批 invoker 里选谁"，它看不见下游真实压力——除非你愿意为维护 `RpcStatus` 和 WRR 状态付内存与竞争的钱。

第9章　服务目录与路由：地址从哪来、被谁筛掉

一句话：Directory 回答"有哪些地址"，RouterChain 回答"这次能用哪些"。

为什么需要它：灰度发布，给新版本实例打了 `tag=gray`，消费者也带了 `dubbo.tag=gray`，但请求还是打到老版本；查日志发现路由链返回了空，然后"自动"回退到了全部实例。这个"自动回退"就是本章的核心：`force` 的默认值、空列表保护、`cachedInvokerUrls` 兜底，三条机制叠在一起，才有了你看到的"路由规则不生效"。

核心结构：

概念	作用
`Directory`	地址来源抽象，`list(invocation)` 返回已过滤的 invoker
`StaticDirectory`	固定 invoker 列表的快照，用于 group 合并、直连
`DynamicDirectory`	订阅注册中心，实现 `NotifyListener`，`forbidden` 标记无可用地址
`RegistryDirectory`	接口级目录，按 category 拆 configurator/router/provider
`ServiceDiscoveryRegistryDirectory`	应用级目录，收 instance 列表，`isServiceDiscovery()=true`
`RouterChain` / `SingleRouterChain`	双链切换 + 读写锁，真正执行路由
`StateRouter`	3.0 起的主路由模型，`BitList` + `next` 链式调用
`force`	路由结果为空时，是返回空还是回退到输入
`BitList`	`long[]` 位图 + `Object[]` 数组的 invoker 容器，支持位运算求交

关键代码：

```
// RegistryDirectory.java:274-311  空地址的三种处理，全部在这二十行里
private void refreshInvoker(List<URL> invokerUrls) {
    if (invokerUrls.size() == 1
            && invokerUrls.get(0) != null
            && EMPTY_PROTOCOL.equals(invokerUrls.get(0).getProtocol())) {   // "empty://"：注册中心明确宣告"没提供者了"
        refreshRouter(BitList.emptyList(), () -> this.forbidden = true);    // 先清路由链，再置 forbidden
        destroyAllInvokers();
    } else {
        this.forbidden = false;
        Set<URL> localCachedInvokerUrls = this.cachedInvokerUrls;
        if (invokerUrls.isEmpty()) {
            if (CollectionUtils.isNotEmpty(localCachedInvokerUrls)) {
                invokerUrls.addAll(localCachedInvokerUrls);                  // 空保护：注册中心推了空列表，用上次的地址顶住
            }
        } else {
            localCachedInvokerUrls = new HashSet<>();
            localCachedInvokerUrls.addAll(invokerUrls);
            this.cachedInvokerUrls = localCachedInvokerUrls;
        }
        if (invokerUrls.isEmpty()) {
            return;                                                          // 没有缓存可回退，直接不动，保留旧 invokers
        }
        Map<URL, Invoker<T>> newUrlInvokerMap = toInvokers(oldUrlInvokerMap, invokerUrls);
        // ... newUrlInvokerMap 为空则 return（协议不匹配等）
        BitList<Invoker<T>> finalInvokers =
                multiGroup ? new BitList<>(toMergeInvokerList(newInvokers)) : new BitList<>(newInvokers);
        refreshRouter(finalInvokers.clone(), () -> this.setInvokers(finalInvokers));  // 双链先更新，再换引用
        this.urlInvokerMap = newUrlInvokerMap;
        destroyUnusedInvokers(oldUrlInvokerMap, newUrlInvokerMap);
        this.invokersChanged();
    }
}
```

```
// ConditionStateRouter.java:245-278  "至少保留一个"真正的落点
BitList<Invoker<T>> result = invokers.clone();
result.removeIf(invoker -> !matchThen(invoker.getUrl(), url));

if (!result.isEmpty()) {
    if (needToPrintMessage) { messageHolder.set("Match return."); }
    return result;
} else if (this.isForce()) {
    logger.warn(CLUSTER_CONDITIONAL_ROUTE_LIST_EMPTY, "execute condition state router result list is empty. and force=true",
            "", "The route result is empty and force execute. ...");
    return result;                        // force=true：空就是空，交给上层报 No provider
}
// 走到这里说明 force=false 且结果为空 —— 返回输入的 invokers，等于"这条规则没生效"
return invokers;
```

拆解：

1. 两类 Directory：静态是快照，动态是订阅

`Directory` 接口只有几个方法，但 `list(Invocation)` 是整条调用链的分水岭——`ClusterInvoker` 拿到的候选列表就来自这里。`AbstractDirectory` 把公共逻辑收得很干净：`list` 先拿 `routerChain.getLock().readLock()`，克隆 `validInvokers`（初始化前是 `invokers`），取 `SingleRouterChain`，再调子类 `doList`；结果为空就打一条 `CLUSTER_NO_VALID_PROVIDER` 的 warn。

`invokers` 和 `validInvokers` 的区分是 3.x 才有的。`invokers` 是注册中心给的全量，`validInvokers` 减掉了两类：`invokersToReconnect`（连接断了，`addInvalidateInvoker` 塞进去，后台 `checkConnectivity` 每 `reconnectTaskPeriod` 试一次）和 `disabledInvokers`（服务下线通知，`addDisabledInvoker` 塞进去，只有注册中心重新推送才恢复）。这个设计让"暂时不可用"和"已下线"走了两条不同的恢复路径，不用重建整个列表。

`checkConnectivity` 有个容易忽略的约束：一个 `Semaphore(1)` 的 `tryAcquire` 保证每个 directory 同一时刻最多只有一个重连任务在跑，任务末尾如果 `invokersToReconnect` 还不为空就再提交一次，形成自驱动的循环。这样即使有 500 个地址全挂了，也不会瞬间往调度线程池里塞 500 个任务——它按 `reconnectTaskTryCount` 分批随机抽着试。

还有一个反直觉的分支在 `DynamicDirectory.doList`：`if (multiGroup) { return this.getInvokers(); }`。多 group（`group = "*"` 或逗号分隔）时**直接跳过整条路由链**，返回全量 invoker。原因是多 group 的每个 group 在 `toMergeInvokerList` 里已经各自建过 `StaticDirectory` 并合并过了，再走一遍外层路由链没有意义，反而可能把已经合并好的 cluster invoker 再过滤一次。这条短路也是"配了 `group` 之后路由规则不生效"的一个隐藏原因。

`StaticDirectory` 是纯快照：构造时 `setInvokers(new BitList<>(invokers))`，`notify` 直接换列表，没有注册中心、没有订阅。它出现在两个地方——`RegistryDirectory.toMergeInvokerList` 里每个 group 建一个 `StaticDirectory` 再 `cluster.join`（多 group 合并），以及直连 `dubbo://ip:port`。`buildRouterChain()` 会为它建一条独立的路由链，因为 group 合并后的列表需要单独路由。

`DynamicDirectory` 实现 `NotifyListener`，`subscribe(url)` 就是 `registry.subscribe(url, this)`。它的 `doList` 有两道闸：`forbidden && shouldFailFast` 时直接抛 `RpcException(FORBIDDEN_EXCEPTION)`——这是"确定没有提供者"的快速失败；路由抛异常时 catch 住、记 `CLUSTER_FAILED_SITE_SELECTION`、返回空列表，而不是把异常往上冒。

`RegistryDirectory.notify` 收到的是原始 `List<URL>`，先按 category 分组（`configurators` / `routers` / `providers`）。configurator 不是过滤，是**改写**：`overrideWithConfigurator(getOriginalConsumerUrl())` 把 `timeout`、`weight` 这类参数覆盖进 `directoryUrl`，所以配置中心的动态配置最终体现在 URL 参数上，而不是另一套机制。router 走 `toRouters` 生成老式 `Router` 实例，`addRouters` 追加到链尾。

然后是 `refreshInvoker`，本章最关键的一段。三种"空"被区别对待：收到单条 `empty://`（`EMPTY_PROTOCOL = "empty"`）表示注册中心明确说"这个服务没有提供者了"，此时 `forbidden = true` 并销毁全部 invoker；收到**空列表**但有 `cachedInvokerUrls`，走空保护，把上次的地址塞回去；两者都没有，直接 `return`，保留现状。这三种情况在日志里长得完全不一样，排查"服务突然不可用"时先看是哪一种。

`toInvokers` 的缓存策略也值得看：`oldUrlInvokerMap.remove(url)` 命中就复用旧 invoker，不重新 `protocol.refer`——只有 URL 参数变了（key 是 merge 后的 url）才会重建连接。这是地址推送频繁时不会疯狂建连的原因。

`ServiceDiscoveryRegistryDirectory` 是应用级服务发现的目录，`isServiceDiscovery()` 返回 `true`。它收到的不是带 category 的 URL，而是实例列表，所以没有 `judgeCategory` 这一步，`toInvokers` 的 key 也变成 `ProtocolServiceKeyWithAddress`；一个实例暴露多个协议时，会用 `InstanceWrappedInvoker` 包一层。`isNotificationReceived()` 的判断也变了：要求 `serviceListener.getAllInstances().size() == getServiceNames().size()`，即所有相关应用都推过一轮才算"收到"。

2. RouterChain：双链切换，和"至少保留一个"的真正位置

`RouterChain.buildChain` 一次建**两条** `SingleRouterChain`（`mainChain` / `backupChain`）。`setInvokers` 的流程是刻意绕的：先把 `currentChain` 切成 `backupChain`（此时新调用走旧链、旧地址），再去改 `mainChain`（拿它的写锁，等所有正在用 main 的调用结束），改完把 `notifyingInvokers` 指向新列表、执行 `switchAction`（真正替换 directory 里的 invokers 引用），最后切回 `currentChain = mainChain` 并补更新 `backupChain`。

为什么要两条链：地址刷新期间，正在跑的调用不能看到"链里是旧地址、列表是新地址"这种半成品状态。`getSingleChain` 里那段判断就是在处理这个窗口——如果 `availableInvokers` 来自新列表而 main 已更新完，就用 main；否则用 backup。代价是每次刷新要改两次链、加两次写锁，换来的是地址变更期间不出现"路由到已下线实例"。

`simpleRoute` 的执行顺序：先跑 StateRouter 链（`headStateRouter.route(...)`），结果为空且 `(shouldFailFast || routers.isEmpty())` 就返回空；再逐个跑老式 `Router`，`CollectionUtils.isEmpty(commonRouterResult) && shouldFailFast` 就返回空，`!routeResult.isNeedContinueRoute()` 则提前返回。`shouldFailFast` 来自 `dubbo.router.should-fail-fast`，默认 `"true"`。

这里有个广泛流传的误解：**"RouterChain 保证至少返回一个 invoker"**。代码里没有任何"保底"逻辑，`RouterChain` 只在为空时短路返回。真正的"至少保留一个"发生在每个 router 内部——`ConditionStateRouter` 在 `force=false` 且结果为空时 `return invokers`（返回输入），等于"这条规则不生效"；`TagStateRouter` 在没有匹配 tag 时走 FAILOVER 分支，返回所有"没有 tag 的实例"。所以保护是**逐个 router 协商**出来的，不是链条层面的兜底。`force=true` 就是主动放弃这层保护：宁可报 `No provider available`，也不要打到不该打的机器上。

`AbstractStateRouter.route` 是 `final` 的模板方法，三个动作值得记：`doRoute` 返回后，如果结果不是同一个引用，就 `routeResult = invokers.and(routeResult)`——用位运算强制结果是输入的子集，任何 router 都别想凭空造出地址；`supportContinueRoute()` 为 false 时才由框架调 `continueRoute`（tag 这类需要看全局的 router 会重写它自己控制）；`shouldFailFast && routeResult.isEmpty()` 时不继续往下传。

顺序由 `@Activate(order)` 决定，`getActivateExtension` 排序后建链：Mock(-100) → Tag(100) → Service(140) → ProviderApp(145) → App(150) → Script(200)。Mock 排最前是因为它要把"该走 mock"的请求提前摘出去；Tag 在条件路由之前，是因为标签是更粗的圈选。

3. 条件路由与标签路由：语法、force，以及那个 `|`

条件路由的规则是 `when => then`，解析在 `ConditionStateRouter.init`：按 `=>` 切两半，左边空或 `true` 视为无条件，右边空或 `false` 时 `thenCondition = null`——这不是"没规则"，而是"黑名单"，`doRoute` 里 `thenCondition == null` 直接返回空列表。

匹配语法由 `ROUTE_PATTERN = Pattern.compile("([&!=,]*)\\s*([^&!=,\\s]+)")` 切分，支持 `=`、`!=`、`,`（或）、`&`（与）。值侧交给 `ValuePattern` 扩展：通配 `*`、区间 `1~100`；键侧交给 `ConditionMatcherFactory`，默认三类——`param`（url 参数）、`argument`（`arguments[0]`）、`attachment`（`attachments[foo]`）。所以 `method = sayHello => region = hangzhou`、`arguments[0] = 123 => host != 10.20.153.10` 都是合法规则。

匹配方向要分清：`matchWhen` 拿 `consumerUrl` 的 `toOriginalMap()` 当样本，`matchThen` 拿每个 provider 的 url 当样本。`method` 和 `methods` 这两个键特殊处理，值取自 `RpcUtils.getMethodName(invocation)` 而不是 url——这就是"按方法路由"能生效的原因。

`force` 的语义前面说过，这里补一句实操：默认 `false`（`AbstractRouterRule.force = false`），意味着**任何写错的条件路由都会静默失效**，只在日志里留一条 `CLUSTER_CONDITIONAL_ROUTE_LIST_EMPTY`。灰度/隔离这类"宁可失败也不能打错"的场景，必须显式写 `force: true`。

标签路由有两套规则源。有动态规则（配置中心下发的 `TagRouterRule`）时，按 `tagnameToAddresses` 做地址匹配；规则无效或未启用时退化到 `filterUsingStaticTag`，直接比 provider url 上的 `dubbo.tag` 参数。

`tag` 的取值顺序是：`invocation.getAttachment("dubbo.tag")` 优先，其次 `url.getParameter("dubbo.tag")`。这意味着单个请求可以覆盖全局标签，灰度时按用户维度打标就靠这个。

三个 `force` 必须分清，混了就排查不出来：

- 规则里的 `force`（`TagRouterRule.force`，默认 false）：动态标签组匹配到空时，是返回空还是继续往下走。
- 请求级的 `dubbo.force.tag`（`FORCE_USE_TAG`）：没有匹配 tag 的实例时，是否允许回退到"无 tag 实例"。`isForceUseTag` 先看 attachment，再看 router url 参数。
- `ConditionStateRouter` 的 `force`：条件路由结果为空时是否强制生效。

`doRoute` 的分支顺序值得逐行看一遍：有 tag 且动态组里能找到地址 → 过滤，结果非空**或** `rule.force` 为真就返回；动态组里没有这个 tag → 退到静态 tag；静态 tag 也没有匹配 → 若 `!isForceUseTag` 则 FAILOVER，返回"所有不在动态组里、且自己也没打 tag"的实例。这条 FAILOVER 就是"灰度流量漏到老版本"的官方通道——它不是 bug，是默认行为。

tag 还支持层级，`selectAddressByTagLevel` 处理 `beta|team1|partner1` 这种写法：从长到短逐级回退（先找 `beta|team1|partner1`，空了找 `beta|team1`，再空了找 `beta`）。`force` 为真时不做层级回退，直接取完整 selector。这让"团队内灰度 → 团队全量 → 全局"能用一条 tag 表达。

4. StateRouter：3.x 到底换掉了什么

旧的 `Router` 接口吃 `List<Invoker>`、吐 `List<Invoker>`，每次过滤都要新建 ArrayList，链式执行靠 `RouterChain` 里的 for 循环。`StateRouter` 换了三样东西。

第一是容器：`BitList<Invoker>` 内部是 `long[]` 位图 + `Object[]` 数组，`removeIf` 是把对应 bit 置 0，`and(other)` 是按位与。10 个 router 串行过滤时，省掉的是 10 次数组拷贝——在几千个地址、每秒几万次调用的场景下，这是 GC 压力的主要来源之一。

第二是链式结构：`initWithStateRouters` 倒序遍历，`nextStateRouter.setNextRouter(stateRouter)`，把列表编成单链表，`headStateRouter` 是入口。`route` 是 `final` 模板方法，子类只实现 `doRoute`，无法绕过"结果必须是输入子集"和"是否继续"这两条约束。

第三是 `RouterResult.needContinueRoute`：老 `Router` 也能提前终止，`RouterResult` 带上 `needContinueRoute` 和 `message`（用于 QoS 的路由快照）。这是 3.x 排障能力的一部分——`RpcContext.getServiceContext().isNeedPrintRouterSnapshot()` 打开后，每个 router 的输入输出都会被记录成 `RouterSnapshotNode` 树，`No provider available` 时直接告诉你"是哪个 router 把列表清空的"。这比 2.7 时代靠猜强太多。

脚本路由（`ScriptStateRouter`）是这条链上唯一执行外部代码的节点，安全边界值得说清。它用 `javax.script`，默认引擎 `javascript`（`DEFAULT_SCRIPT_TYPE_KEY`），`Compilable.compile(rule)` 预编译一次，执行时 `AccessController.doPrivileged(..., accessControlContext)`，`AccessControlContext` 里只放了 `RuntimePermission("accessDeclaredMembers")` 一个权限。脚本里能拿到 `invokers`、`invocation`、`context` 三个绑定，返回值经 `getRoutedInvokers` 做 `retainAll`——只能做减法，不能新增地址。

但这层沙箱的边界要说实话：它限制的是 Java 权限模型里的反射能力，不是完整的进程隔离；规则来自配置中心，等于把"能改配置的人"提升到"能在消费者进程里执行任意脚本"。而且 JDK 15 起 Nashorn 被移除，JDK 17 上默认引擎直接 `null`，构造时就抛 `unsupported route engine type`。生产环境用它之前，先确认 JDK 版本和谁能改配置中心。

`MockStateRouter`（`MockInvokersSelector`）是另一个有意思的实现：`notify` 时就把 invoker 预分成 `mockedInvokers` 和 `normalInvokers` 两个 `BitList`，`doRoute` 只看 `invocation.need.mock` 这个 attachment，`invokers.and(...)` 一下就完事。它把"每次调用都做分类"变成了"地址变更时分类一次"，是 `StateRouter` 缓存思路的典型用法。

容易误解的地方：

1. **"`force=true` 就是强制打到指定机器"**。反了。`force=true` 是"允许返回空"，匹配不到就真的没有 provider，调用方直接报错。它的用途是"宁可失败也不打错"，不是"无论如何都路由过去"。
2. **"注册中心推空列表，消费者就清空地址"**。`refreshInvoker` 里有 `cachedInvokerUrls` 空保护，空列表会用上次的地址顶住；只有收到单条 `empty://` 才 `forbidden = true` 并销毁 invoker。这两条路径的日志完全不同，排查时先分清。
3. **"`RouterChain` 兜底保证至少一个 invoker"**。链条里没有任何保底逻辑，只有 `shouldFailFast` 的短路。保护在 router 内部：`force=false` 时返回输入列表。所以"路由规则不生效"和"路由规则生效了但没匹配到"在日志上都表现为"用了全部地址"，只能靠 `force` 和路由快照区分。
4. **"`tag` 路由的 `force` 和 `dubbo.force.tag` 是一回事"**。前者是规则级（动态标签组为空时是否返回空），后者是请求级（没有匹配 tag 时是否允许回退到无 tag 实例）。默认都 false，叠加起来的效果是"灰度实例没起来时，灰度流量会静默流到老版本"。

记住这一句：地址由 Directory 提供，路由由 RouterChain 裁剪，两者之间只传 `BitList`——任何 router 都只能做减法，减到零还是减到一，取决于 `force`。

第10章　网络层：Netty 之上还包了几层

一句话：transport 决定"能不能连上、字节怎么进出"，exchange 决定"发出去的那一条，回来时还给谁"。

为什么需要它：线上出现这样的日志——消费者侧 `Waiting server-side response timeout by scan timer. start time: ..., client elapsed: 2 ms, server elapsed: 4998 ms, timeout: 5000 ms, request: Request [id=1024, version=2.0.2, twoWay=true, ...]`，而服务端业务日志里这个方法 3ms 就返回了。第二个现象：`message can not send, because channel is closed . url:dubbo://...`，重启消费者后恢复。这两个问题都不在 Netty 里，也不在业务代码里，而在 Netty 和 DubboProtocol 之间那两层：一层负责"把 `Object` 变成字节发出去"，另一层负责"给每条请求编号、等它的响应、超时了怎么办、连接断了未完成的请求怎么处理"。这一章把 `Transporter → Exchanger → Protocol` 这条链路拆开，顺带说清 `Dispatcher` 到底把 handler 丢到了哪个线程。

核心结构：

概念	作用
`Transporter`	`@SPI("netty")`，只两个方法 `bind` / `connect`，`@Adaptive({server, transporter})` / `@Adaptive({client, transporter})`
`Client` / `RemotingServer`	传输层的端点，`Client` 额外有 `reconnect()`，`RemotingServer` 有 `getChannels()`
`ChannelHandler`	五个回调 `connected/disconnected/sent/received/caught`，整条链都靠它串起来
`Codec2`	`encode/decode(Channel, ChannelBuffer)`，用 `NEED_MORE_INPUT` 表示"字节还不够"
`Exchanger`	`@SPI("header")`，`bind` / `connect` 返回 `ExchangeServer` / `ExchangeClient`
`Request` / `Response`	带 `mId` 的消息信封，`mTwoWay` / `mEvent` / `mBroken` / `mStatus`
`DefaultFuture`	`CompletableFuture` 子类，靠 `FUTURES: Map<Long, DefaultFuture>` 做 requestId → 等待者 的路由
`HeaderExchangeHandler`	exchange 层的总入口，`received` 分派 Request/Response，`sent` 打时间戳
`Dispatcher`	`@SPI(AllDispatcher.NAME)`，把 handler 包一层决定"在哪个线程执行"
`ChannelHandlerDispatcher`	一对多扇出，`CopyOnWriteArraySet` 存多个 handler，逐个调用且吞掉单个异常

关键代码：

```
// DefaultFuture.java:99-100  构造即注册：id 是唯一的路由键
FUTURES.put(id, this);        // id -> 等待中的 future
CHANNELS.put(id, channel);    // id -> 它走的连接，断链时按 channel 反查

// DefaultFuture.java:199-215  响应回来时，只有这一步能唤醒调用线程
public static void received(Channel channel, Response response, boolean timeout) {
    try {
        DefaultFuture future = FUTURES.remove(response.getId());  // 先摘掉，天然防重复完成
        if (future != null) {
            Timeout t = future.timeoutCheckTask;
            if (!timeout) {
                t.cancel();          // 正常响应要撤销定时任务，否则 5s 后还会来一发假超时
            }
            future.doReceived(response);
            shutdownExecutorIfNeeded(future);
        } else {
            logger.warn(PROTOCOL_TIMEOUT_SERVER, "", "",
                    "The timeout response finally returned at " + ...);  // 迟到的响应只 warn，不抛
        }
    } finally {
        CHANNELS.remove(response.getId());
    }
}
```

```
// HeaderExchangeHandler.java:196-213  received 是 exchange 层的分派中枢
public void received(Channel channel, Object message) throws RemotingException {
    final ExchangeChannel exchangeChannel = HeaderExchangeChannel.getOrAddChannel(channel);
    if (message instanceof Request) {
        Request request = (Request) message;
        if (request.isEvent()) {
            handlerEvent(channel, request);                 // 心跳/读写事件，不进业务
        } else {
            if (request.isTwoWay()) {
                handleRequest(exchangeChannel, request);    // 要回包：走 reply 拿 CompletionStage
            } else {
                handler.received(exchangeChannel, request.getData());  // oneway：直接下沉
            }
        }
    } else if (message instanceof Response) {
        handleResponse(channel, (Response) message);        // 只有这一条路能唤醒 Future
    } else if (message instanceof String) {
        // 客户端收到 String 直接报错，服务端当成 telnet 命令
    }
}

// HeaderExchangeHandler.java:162-174  sent 的唯一职责：给 future 记一个"已发出"的时间点
public void sent(Channel channel, Object message) throws RemotingException {
    ...
    if (message instanceof Request) {
        DefaultFuture.sent(channel, request);   // -> future.doSent()，写 volatile long sent
    }
}
```

拆解：

10.1 三层 SPI，每层只加一件事

`Transporter` 的接口小到不能再小：

```
@SPI(value = "netty", scope = ExtensionScope.FRAMEWORK)
public interface Transporter {
    @Adaptive({Constants.SERVER_KEY, Constants.TRANSPORTER_KEY})
    RemotingServer bind(URL url, ChannelHandler handler) throws RemotingException;
    @Adaptive({Constants.CLIENT_KEY, Constants.TRANSPORTER_KEY})
    Client connect(URL url, ChannelHandler handler) throws RemotingException;
}
```

`NettyTransporter` 的实现就是 `new NettyServer(url, handler)` / `new NettyClient(url, handler)`，没有任何逻辑。这一层的价值在于"换传输实现"——netty4、netty3、mina、grizzly 在历史版本里都在这一个点上替换，上层 `Exchanger` 完全不知道底下是 NIO 还是别的。

往上一层是 `Exchanger`，`@SPI("header")`，`Exchangers.bind(url, handler)` 会先 `url.addParameterIfAbsent(Constants.CODEC_KEY, "exchange")`，然后 `HeaderExchanger.bind` 做了这件关键的事：

```
// HeaderExchanger.java:50-58
server = new HeaderExchangeServer(
        Transporters.bind(url, new DecodeHandler(new HeaderExchangeHandler(handler))));
```

注意包装顺序：**`DecodeHandler` 在外，`HeaderExchangeHandler` 在内**。`DecodeHandler.received` 会先对 `Decodeable` 的消息调 `decode()`，再往下传。这个顺序在第 11 章会变得很重要——它决定了反序列化发生在哪个线程。

三层各加什么，一句话概括：transport 加"连接与字节"；exchange 加"Request/Response 信封 + requestId + Future + 心跳 + 重连"；protocol（`DubboProtocol`）加"`Invocation` 语义 + exporter 查找"。

顺带解释 `@Adaptive` 的键顺序。`bind` 上是 `@Adaptive({Constants.SERVER_KEY, Constants.TRANSPORTER_KEY})`，`connect` 上是 `@Adaptive({Constants.CLIENT_KEY, Constants.TRANSPORTER_KEY})`——**前一个键优先**。所以 URL 里同时有 `server=netty4&transporter=netty` 时用 `netty4`，只有 `transporter` 时才回退到它。`server` / `transporter`、`client` / `transporter` 是两组同义键，分别对应历史上不同版本的配置写法，Dubbo 没有删掉任何一个，而是让 adaptive 按顺序取第一个存在的。同理 `Exchanger` 的 adaptive 键是 `exchanger`，默认 `header`；`Exchangers.bind` 里那句 `url.addParameterIfAbsent(CODEC_KEY, "exchange")` 看着像是给传输层指定 codec，但

`DubboProtocol.createServer` 在调它之前已经 `addParameter(CODEC_KEY, DubboCodec.NAME)`，`IfAbsent` 不会覆盖，所以 dubbo 协议下这句其实不生效——读源码时别被它误导。

顺带一个容易忽略的细节：`AbstractEndpoint` 在构造时就确定了 codec，`getChannelCodec(url)` 的顺序是 `url.getParameter(CODEC_KEY)` → 为空则用 `url.getProtocol()` → 都没有则用 `Codec2` 的 `"default"` 扩展；如果只有老的 `Codec`（单参数的 `encode(Channel, OutputStream, Object)`），会套一层 `CodecAdapter` 适配。`DubboProtocol.createServer` 里明确写了 `.addParameter(CODEC_KEY, DubboCodec.NAME)`，所以 dubbo 协议走的是 `Codec2` 扩展 `dubbo` → `DubboCountCodec`。

10.2 Exchange 层补的第一件事：requestId 与超时

`Request` 的 id 是 `long`，来源是静态的 `AtomicLong INVOKE_ID`：

```
// Request.java:58-72
static {
    long startID = ThreadLocalRandom.current().nextLong();   // 起点随机，避免重启后 id 撞车
    if (Boolean.parseBoolean(SystemPropertyConfigUtils.getSystemProperty(DUBBO_USE_SECURE_RANDOM_ID, "false"))) {
        startID = new SecureRandom(SecureRandom.getSeed(20)).nextLong();
    }
    INVOKE_ID = new AtomicLong(startID);
}
private static long newId() {
    return INVOKE_ID.getAndIncrement();   // 溢出到负数也合法，注释里明说了
}
```

为什么不用 `(channel, seq)` 做键？因为一条连接上并发着成百上千个请求，`channel` 对区分它们毫无帮助；而 `id` 是全局单调的，跨连接也唯一，`FUTURES` 一张表就够。`Response` 只带同一个 id 回来，`HeaderExchangeHandler.handleResponse` 把它交给 `DefaultFuture.received`，后者 `FUTURES.remove(id)` 拿到等待者。**先 remove 再 complete**，所以重复响应、超时后迟到的响应都不会二次完成 Future——迟到的那次只会走到 `else` 分支打一条 warn。

超时不是"到点抛异常"，而是"到点造一个假响应"。`newFuture` 里立刻注册定时任务：

```
// DefaultFuture.java:106-109
private static void timeoutCheck(DefaultFuture future) {
    TimeoutCheckTask task = new TimeoutCheckTask(future.getId());
    future.timeoutCheckTask = TIME_OUT_TIMER.get().newTimeout(task, future.getTimeout(), TimeUnit.MILLISECONDS);
}
```

`TIME_OUT_TIMER` 是 `HashedWheelTimer`，tick 30ms，线程名 `dubbo-future-timeout`。`TimeoutCheckTask` 到点后做的是：

```
// DefaultFuture.java:337-341
timeoutResponse.setStatus(future.isSent() ? Response.SERVER_TIMEOUT : Response.CLIENT_TIMEOUT);
timeoutResponse.setErrorMessage(future.getTimeoutMessage(true));
DefaultFuture.received(future.getChannel(), timeoutResponse, true);   // timeout=true，跳过 cancel
```

`isSent()` 判断的是 `sent > 0`，而 `sent` 正是 `HeaderExchangeHandler.sent` → `DefaultFuture.sent` → `doSent()` 写进去的。于是超时被分成两种语义：请求根本没发出去（`CLIENT_TIMEOUT`），还是发出去了但服务端没回（`SERVER_TIMEOUT`）。`TimeoutException` 的构造参数 `new TimeoutException(res.getStatus() == Response.SERVER_TIMEOUT, channel, ...)` 把这个布尔值一路带到 `RpcException`，这就是为什么你能在日志里看到"client elapsed / server elapsed"两段耗时——`getTimeoutMessage` 用 `sent - start` 和 `now - sent` 分别算的。

`getTimeoutMessage` 里那个 `+ (scan ? " by scan timer" : "")` 说明还存在另一条扫描式超时路径，但正常请求的超时都走时间轮。

为什么用 `HashedWheelTimer` 而不是 `ScheduledExecutorService`？因为一个消费者进程里同时在途的请求可能是几万个，每个都要一个定时任务。`ScheduledThreadPoolExecutor` 用的是 `DelayedWorkQueue`（二叉堆），插入和删除都是 O(log n)，几万个任务意味着每次调用要动堆；时间轮是 O(1) 插入，30ms 一格、默认 512 格，精度够用而开销恒定。代价是**取消不是立即生效**——`t.cancel()` 只是标记，任务槽要转到才会被清理，所以时间轮不适合"取消后必须立刻释放资源"的场景，但适合超时检测这种"晚 30ms 也无所谓"的场景。`TIME_OUT_TIMER` 用 `GlobalResourceInitializer` 包了一层，进程退出时 `DefaultFuture.destroy()` 会 `stop()` 它并清空两张 map。

再补一个和超时紧密相关的类：`ThreadlessExecutor`。消费者做同步调用时，`DubboInvoker.doInvoke` 会 `currentClient.request(request, timeout, executor)` 把 `executor` 一起传下去，`DefaultFuture.newFuture` 把它存进 `future.executor`。响应回来时 `WrappedChannelHandler.getPreferredExecutorService` 发现 msg 是 `Response`，就 `DefaultFuture.getFuture(response.getId())` 拿到这个 future，取它的 executor 来跑回调——于是回调直接跑在**发起调用的那个线程**上，`future.get()` 拿到的结果不需要再跨线程。异步调用时这个 executor 是 `null`，回退到共享业务池。这条路径是"同步调用为什么没有线程切换开销"的答案。

10.3 Netty 的 pipeline 与 Dispatcher 的五种线程模型

先看 pipeline 长什么样。服务端（`NettyServer.initServerBootstrap`）：

```
ch.pipeline().addLast("negotiation", new SslServerTlsHandler(getUrl()));
ch.pipeline()
        .addLast("decoder", adapter.getDecoder())
        .addLast("encoder", adapter.getEncoder())
        .addLast("server-idle-handler", new IdleStateHandler(0, 0, closeTimeout, MILLISECONDS))
        .addLast("handler", nettyServerHandler);
```

客户端（`NettyClient.initBootstrap`）几乎对称，只是 idle 参数反过来：

```
.addLast("decoder", adapter.getDecoder())
.addLast("encoder", adapter.getEncoder())
.addLast("client-idle-handler", new IdleStateHandler(heartbeatInterval, 0, 0, MILLISECONDS))
.addLast("handler", nettyClientHandler);
```

三个要点。第一，**编解码器是 Netty 的 handler，不是 Dubbo 的 `ChannelHandler`**：`NettyCodecAdapter.InternalEncoder` 继承 `MessageToByteEncoder`，`InternalDecoder` 继承 `ByteToMessageDecoder`，它们只是把 `ChannelBuffer` 适配成 Dubbo 的 `Codec2` 接口。第二，**idle 的方向是反的**：客户端 `readerIdle = heartbeatInterval`，读空闲就发心跳（`NettyClientHandler.userEventTriggered` 里 `new Request() + setEvent(HEARTBEAT_EVENT)`）；服务端 `allIdle = closeTimeout`，任何方向都空闲就 `channel.close()`。第三，`NettyServerHandler` 和 `NettyClientHandler` 都标了 `@Sharable`，它们是单例，每条连接只是往同一个 `channels` map 里塞自己的 `NettyChannel`。

`NettyChannel` 是 Dubbo 对 Netty `Channel` 的包装，`CHANNEL_MAP` 做一对一缓存，`send` 走 `Netty4BatchWriteQueue` 合并写，并且**只在 `encodeInIOThread=false`（默认）时在调用线程编码**：

```
// NettyChannel.java:190-196
if (!encodeInIOThread) {
    ByteBuf buf = channel.alloc().buffer();
    ChannelBuffer buffer = new NettyBackedChannelBuffer(buf);
    codec.encode(this, buffer, message);     // 调用线程序列化，不占 IO 线程
    outputMessage = buf;
}
ChannelFuture future = writeQueue.enqueue(outputMessage).addListener(...);
```

再看 Dubbo 自己的 handler 链。`ChannelHandlers.wrapInternal` 只有一行，但信息量很大：

```
// ChannelHandlers.java:43-49
return new MultiMessageHandler(new HeartbeatHandler(url.getOrDefaultFrameworkModel()
        .getExtensionLoader(Dispatcher.class)
        .getAdaptiveExtension()
        .dispatch(handler, url)));
```

外层固定是 `MultiMessageHandler(HeartbeatHandler(Dispatcher(...)))`，内层由 `Dispatcher` 决定。`Dispatcher` 的 `@SPI(value = AllDispatcher.NAME)` 意味着**默认就是 `all`**。五种实现的实际差别只在"哪个回调被丢进线程池"：

扩展名	connected / disconnected	received	适用场景
`all`（默认）	丢线程池	丢线程池（`getPreferredExecutorService`）	通用；IO 线程只做编解码
`direct`	不包装，直接在 IO 线程	只有 `executor instanceof ThreadlessExecutor` 才丢，否则直接调	消费者侧同步调用，让响应回到调用线程
`message`	不包装	丢线程池	只想把消息处理移出 IO 线程
`execution`	不包装	只对 `Request` 丢线程池	请求走业务线程，响应在 IO 线程就地处理
`connection`	丢独立的单线程 `connectionExecutor`	丢线程池	需要连接事件严格有序

为什么默认是 `all`？因为 Netty 的 worker 线程数是 `DEFAULT_IO_THREADS = min(cpu+1, 32)`，一条连接上的所有事件都绑在同一个 EventLoop 上。如果 `received` 里直接跑业务（反序列化 + 反射调用 + 可能的数据库访问），这条 EventLoop 上**其它所有连接**都会被拖住——一个慢请求能拖垮几十条连接。`all` 把 `connected`、`disconnected`、`received`、`caught` 全部搬到业务线程池，IO 线程只剩 `ByteToMessageDecoder` 和 `MessageToByteEncoder` 的活。

代价是：**同一个 channel 的 `received` 可能并发执行**。`AllChannelHandler.received` 是 `executor.execute(new ChannelEventRunnable(channel, handler, ChannelState.RECEIVED, message))`，线程池有 N 个线程，同一个连接的连续两个请求完全可能被两个线程同时处理。所以 Dubbo 的 handler 实现必须是线程安全的——这一点在第 11 章的 `DecodeableRpcInvocation` 上会体现出来。`ChannelEventRunnable.run` 里那句 `InternalThreadLocalMap internalThreadLocalMap = InternalThreadLocalMap.getAndRemove()` 也是为这个场景服务的：线程池的线程是复用的，上一个请求残留的 ThreadLocal（比如 RpcContext）必须清掉，否则会串到下一个请求。

`caught` 的回调同样被丢进池子，但要分清两种"异常"。业务代码在 `ChannelEventRunnable.run` 里抛的异常，会被它自己 catch 住并打日志（`ChannelEventRunnable handle RECEIVED operation error`），**不会**往上传播；真正会走 `caught` 的是"任务提交失败"——`executor.execute` 抛 `RejectedExecutionException` 时，`AllChannelHandler` 把它包成 `new ExecutionException(message, channel, ...)` 抛出，`HeaderExchangeHandler.caught` 判断 `e.getRequest()` 是不是一个 `twoWay && !isHeartbeat` 的 `Request`，是就地造一个 `status = SERVER_ERROR`（80）的响应发回去。所以在 `all` 模式下，`caught` 这条路的语义是"线程池扛不住了"，而不是"业务抛异常了"。

`direct` 常被误解成"所有事都在 IO 线程做"。看源码，它只处理一种情况：`getPreferredExecutorService(message)` 返回 `ThreadlessExecutor` 时把任务丢给它（本质是交给等待响应的那个调用线程），否则**直接 `handler.received`**。它服务于消费者侧的同步调用——`ThreadlessExecutor` 让响应在调用线程被处理，省掉一次线程切换。

`connection` 的实现在构造函数里建了一个 `ThreadPoolExecutor(1, 1, ..., new LinkedBlockingQueue<>(...))`，注释里还留着 `// FIXME There's no place to release connectionExecutor!`。它的价值是 `connected` / `disconnected` 严格串行，避免"连接事件乱序导致状态机错乱"。

最后是线程池打满时的行为，这块经常被忽略：`WrappedChannelHandler.sendFeedback` 会构造一个 `status = Response.SERVER_THREADPOOL_EXHAUSTED_ERROR`（100）的响应直接回给消费者，错误信息是 `Server side(ip,port) thread pool is exhausted`。所以看到 100 这个状态码，不用怀疑网络，就是服务端业务线程池满了。

10.4 断线：未完成的请求去哪了，以及怎么重连

这是最容易踩的坑：TCP 断了，`DefaultFuture` 还在等。如果什么都不做，调用方要等满 5 秒超时，而且拿到的是 `TimeoutException`，会误导排查方向。

`HeaderExchangeHandler.disconnected` 的 finally 块解决了这个问题：

```
// HeaderExchangeHandler.java:146-157
int shutdownTimeout = ...;
DefaultFuture.closeChannel(channel, ConfigurationUtils.reCalShutdownTime(shutdownTimeout));
HeaderExchangeChannel.removeChannel(channel);
```

`closeChannel` 遍历 `CHANNELS`，把属于这条连接、还没完成的 future 逐个处理：先给一个 `deadline` 让它们有机会等到"已经在路上的响应"，等不到就调 `respInactive` 造一个 `status = Response.CHANNEL_INACTIVE`（35）的响应，走同一条 `received` 路径完成掉。异常信息是 `Channel ... is inactive. Directly return the unFinished request : ...`——看到这句，就知道是连接断了而不是服务端慢。

同一套机制在 `HeaderExchangeChannel.close()` 里也用了，所以优雅下线（先 `startClose`，再 `close(timeout)`）时，客户端会先等 `hasFuture(channel)` 变空，最多等 `timeout`，然后才真正关连接。`AbstractPeer` 里 `closing` / `closed` 两个 volatile 标志就是为这个两阶段关闭服务的：`startClose()` 只置 `closing`，`isClosing()` 返回 `closing && !closed`，期间 `AbstractServer.connected` 会拒绝新连接（日志 `Close new channel ..., cause: server is closing or has been closed`）。

重连本身有两处触发点。

重连有两处触发点。

第一处是发送时兜底，`AbstractClient.send`：

```
// AbstractClient.java:239-243
if (needReconnect && !isConnected()) {
    connect();
}
```

`needReconnect` 来自 `url.getParameter(Constants.SEND_RECONNECT_KEY, true)`，默认开。`connect()` 用 `connectLock`（`ReentrantLock`）保护，进入后先 `if (isConnected()) return;` 双重检查，避免多线程同时建连。建连失败时的行为分三种：`lazy.connect=true` 只 warn 不抛（注释说"连接由 ReconnectTask 发起"）；`check=false` 也只 warn；否则 `close()` 后抛出。

第二处是定时器，`HeaderExchangeClient` 构造时（`startTimer=true`）会 `startReconnectTask(url)`，底层是一个共享的 `HashedWheelTimer`，线程名 `dubbo-client-heartbeat-reconnect`，tick 1 秒，每轮遍历所有 channel：

```
// ReconnectTimerTask.java:47-72
protected void doTask(Channel channel) {
    Long lastRead = lastRead(channel);
    Long now = now();
    if (!channel.isConnected()) {                       // 从没连上，或者已经断了
        ((Client) channel).reconnect();
    } else if (lastRead != null && now - lastRead > idleTimeout) {   // 连着但读不到数据
        ((Client) channel).reconnect();                 // 判定为假连接，主动重连
    }
}
```

注意这个类叫 `ReconnectTimerTask`，不是 `ReconnectTask`——找源码时别搜错。它的周期不是写死的：`AbstractClient.getReconnectDuration` 算的是 `max(LEAST_RECONNECT_DURATION, idleTimeout / HEARTBEAT_CHECK_TICK)`，其中 `HEARTBEAT_CHECK_TICK = 3`、`LEAST_RECONNECT_DURATION = 60000`。也就是说 idle 超时 180s 时，每 60s 检查一次；idle 只有 3s 时，也不会比 60s 更频繁。这个下限很关键——否则一个刚挂的 provider 会被每秒重连一次。

这里的 `idleTimeout` 也不是随手给的常数。`UrlUtils.getIdleTimeout` 算的是 `url.getParameter(HEARTBEAT_TIMEOUT_KEY, heartBeat * 3)`，并且强制 `idleTimeout >= heartBeat * 2`，否则直接抛 `IllegalStateException("idleTimeout < heartbeatInterval * 2")`。默认 `DEFAULT_HEARTBEAT = 60000`，所以 idle 是 180s、重连检查周期 60s。为什么至少两倍心跳？源码注释给了原因：`idleTimeout should be at least more than twice heartBeat because possible retries of client`——客户端心跳可能丢一次，一次丢失不应该被判定成连接已死。`getCloseTimeout` 里同样的约束也有一份（`closeTimeout < heartbeat * 2` 就抛），服务端的 `IdleStateHandler(0, 0, closeTimeout)` 用的就是它。

还有一个反直觉的设计：`LazyConnectExchangeClient.isConnected()` 在 `client == null` 时**返回 true**，源码注释写得很直白：`Before the request arrives, LazyConnectExchangeClient always exists in a normal connection state to prevent ReconnectTask from initiating a reconnection action.` 延迟连接的对象本来就没连，如果老老实实返回 false，重连任务会把它当故障连接反复重连，直接破坏"懒"的语义。

容易误解的地方：

1. **"Exchange 层就是心跳层"**。不是。Exchange 的核心是 Request/Response 配对与 Future；心跳只是 `Request.isEvent() == true` 且 `mData == HEARTBEAT_EVENT` 的一个特例，而且它在 `HeartbeatHandler.received` 就被拦掉了（`isHeartbeatRequest` 命中后直接 `channel.send(new HeartBeatResponse(...))` 并 `return`），根本到不了 `HeaderExchangeHandler`。`HeaderExchangeHandler.handlerEvent` 处理的是另外两个事件：`READONLY_EVENT` / `WRITEABLE_EVENT`，用来标记 `CHANNEL_ATTRIBUTE_READONLY_KEY`。
2. **"`Dispatcher` 就是 Netty 的 EventLoopGroup"**。完全是两层东西。`EventLoopGroup` 是 Netty 的 IO 线程；`Dispatcher` 是 Dubbo 的 `ChannelHandler` 装饰器，只决定 handler 在哪个线程跑。`all` 模式下线程名来自 `ExecutorUtil.setThreadName(url, SERVER_THREAD_POOL_NAME)`（`AbstractServer` 构造里那句 `executorRepository.createExecutorIfAbsent(...)`），跟 `EventLoop` 的线程名毫无关系——看线程名就能判断一段逻辑跑在哪一层。
3. **"超时后响应回来会抛异常"**。不会。`DefaultFuture.received` 里 `FUTURES.remove` 返回 null，只打一条 warn：`The timeout response finally returned at ...`。同理 `WrappedChannelHandler.getPreferredExecutorService` 对 `responseFuture == null` 的情况回退到共享线程池，不会 NPE。
4. **"客户端和服务端的 `sent` 回调路径一样"**。不一样。`NettyServerHandler.write` 里是 `super.write(...)` 之后直接 `handler.sent(channel, msg)`；而 `NettyClientHandler.write` 只有 `super.write(...)`，客户端的 `sent` 是由 `NettyChannel.send` 里那个 `ChannelFutureListener` 触发的，而且只对 `Request` 触发（`if (!(message instanceof Request)) return;`）。看错这一点，就会以为客户端发响应也会走 `sent`。

记住这一句：`Transporter` 只管把字节送出去，`Exchanger` 才管"这一条字节回来时，该唤醒哪个线程"——`FUTURES` 这张 `id → Future` 的表，就是整个 RPC 的应答路由表。

第11章　Dubbo 协议：16 字节头和两种序列化路径

一句话：16 字节头只回答四个问题——是不是 Dubbo、要不要回、用哪种序列化、body 多长，剩下全交给序列化层。

为什么需要它：服务端日志里出现 `Skip input stream 27`，或者 `Data length too large: 9437184, max payload: 8388608`，或者消费者抛 `Unrecognized serialize type from consumer: 23`。第一个是 header 里声明的 body 长度和实际读到的字节不一致（编码侧少写了或解码侧多读了），第二个是 payload 上限，第三个是 `header[2]` 低 5 位里的序列化 id 在本端找不到对应扩展。这三个问题都只在 header 的位布局里能解释清楚。这一章按"写头 → 写 body → 读头 → 读 body"的顺序，把 `ExchangeCodec` 和 `DubboCodec` 走一遍，再说清半包、粘包和序列化选型。

核心结构：

概念	作用
`ExchangeCodec.HEADER_LENGTH`	固定 16，不是 12
`MAGIC = (short) 0xdabb`	偏移 0-1，用于在字节流里定位帧边界
`header[2]` flag	`FLAG_REQUEST(0x80)` / `FLAG_TWOWAY(0x40)` / `FLAG_EVENT(0x20)` + 低 5 位序列化 id
`header[3]` status	仅响应使用，`Response.OK(20)` 等 12 个状态码
`Bytes.long2bytes(id, header, 4)`	偏移 4-11，8 字节 requestId
`Bytes.int2bytes(len, header, 12)`	偏移 12-15，4 字节 body 长度
`DubboCountCodec`	`Codec2` 扩展 `dubbo`，循环解码 + 打包成 `MultiMessage`
`DubboCodec.decodeBody`	按 flag 分 request/response 两条分支
`DecodeableRpcInvocation`	延迟反序列化的"壳"，持有 `InputStream` 和 `serializationType`
`Serialization#getContentTypeId`	写进 flag 低 5 位的那个 byte

关键代码：

```
// DubboCodec.java:188-216  同一个方法里，两条完全不同的反序列化路径
DecodeableRpcInvocation inv;
if (isDecodeDataInIoThread(channel)) {
    inv = new DecodeableRpcInvocation(frameworkModel, channel, req,
            new UnsafeByteArrayInputStream(readMessageData(is)), proto);
    inv.decode();                    // 路径 A：当场反序列化，发生在 IO 线程
} else {
    inv = new DecodeableRpcInvocation(frameworkModel, channel, req,
            new UnsafeByteArrayInputStream(readMessageData(is)), proto);
    // 路径 B：只读走字节、构造壳子，decode 留给 DecodeHandler 在业务线程做
}
data = inv;
```

```
// DubboCountCodec.java:55-75  一次读到多条完整消息时，打包成一个 MultiMessage
public Object decode(Channel channel, ChannelBuffer buffer) throws IOException {
    int save = buffer.readerIndex();
    MultiMessage result = MultiMessage.create();
    do {
        Object obj = codec.decode(channel, buffer);          // 内部可能返回 NEED_MORE_INPUT
        if (Codec2.DecodeResult.NEED_MORE_INPUT == obj) {
            buffer.readerIndex(save);                        // 回滚到这条消息的起点
            break;
        } else {
            result.addMessage(obj);
            logMessageLength(obj, buffer.readerIndex() - save);   // 把字节数塞进 attachment
            save = buffer.readerIndex();                     // 下一条消息的新起点
        }
    } while (true);
    if (result.isEmpty()) {
        return Codec2.DecodeResult.NEED_MORE_INPUT;          // 一个都不完整
    }
    if (result.size() == 1) {
        return result.get(0);                                // 只有一条就别包 MultiMessage
    }
    return result;
}
```

拆解：

11.1 16 字节头，以及 encode 侧往里写什么

`ExchangeCodec` 开头的常量就是全部协议：

```
// ExchangeCodec.java:60-69
protected static final int HEADER_LENGTH = 16;
protected static final short MAGIC = (short) 0xdabb;
protected static final byte MAGIC_HIGH = Bytes.short2bytes(MAGIC)[0];
protected static final byte MAGIC_LOW = Bytes.short2bytes(MAGIC)[1];
protected static final byte FLAG_REQUEST = (byte) 0x80;
protected static final byte FLAG_TWOWAY = (byte) 0x40;
protected static final byte FLAG_EVENT = (byte) 0x20;
protected static final int SERIALIZATION_MASK = 0x1f;
```

布局是：`0-1` magic `0xdabb`，`2` flag，`3` status，`4-11` requestId，`12-15` body 长度。加起来 16。

写请求（`encodeRequest`，第 257 行起）：

```
Bytes.short2bytes(MAGIC, header);
header[2] = (byte) (FLAG_REQUEST | serialization.getContentTypeId());
if (req.isTwoWay()) { header[2] |= FLAG_TWOWAY; }
if (req.isEvent())  { header[2] |= FLAG_EVENT; }
Bytes.long2bytes(req.getId(), header, 4);
```

写响应（`encodeResponse`，第 310 行起）：

```
header[2] = serialization.getContentTypeId();   // 注意：没有 FLAG_REQUEST
if (res.isHeartbeat()) { header[2] |= FLAG_EVENT; }
byte status = res.getStatus();
header[3] = status;
```

所以"是不是请求"只看 `0x80` 这一位。`decodeBody` 的第一行就是 `if ((flag & FLAG_REQUEST) == 0)` → 走响应分支。

序列化 id 占低 5 位（`SERIALIZATION_MASK = 0x1f`），所以**上限 31**。`Serialization` 接口的 javadoc 明确写了这条约束：`recommended that custom implementations use values different with any value of Constants and don't greater than ExchangeCodec.SERIALIZATION_MASK (31)`。现有 id：`hessian2=2`、`java=3`、`compactedjava=4`、`fastjson=6`、`protobuf=22`、`fastjson2=23`、`kryo2=25`、`msgpack=27`、`fury=28`、`custom message pack=31` ——31 已经顶到边界了，自定义扩展想加 id 要先看有没有空位。

`FLAG_TWOWAY` 只对请求有意义。响应分支里 `decodeBody` 根本不读这一位，因为响应天然是"对某个请求的回执"，没有"要不要回"的问题。这一点在写自定义 codec 时很容易搞混。

头写完之后才是 body。`ExchangeCodec.encodeRequest` 只负责"头 + 长度回填"：先 `buffer.writerIndex(savedWriteIndex + HEADER_LENGTH)` 跳过 16 字节，写 body，再 `int len = bos.writtenBytes()` 、`checkPayload` 、`Bytes.int2bytes(len, header, 12)` ，最后 `buffer.writerIndex(savedWriteIndex)` 把写指针拨回去写头，再跳到末尾。**先写 body 后回填头**是因为长度只有写完才知道。

body 的内容由 `DubboCodec.encodeRequestData` 决定（第 279 行起），顺序是死的：

```
out.writeUTF(version);                                  // dubbo 协议版本，如 "2.0.2"
String serviceName = inv.getAttachment(INTERFACE_KEY);   // 优先 interface，兼容老版本用 path
if (serviceName == null) { serviceName = inv.getAttachment(PATH_KEY); }
out.writeUTF(serviceName);
out.writeUTF(inv.getAttachment(VERSION_KEY));            // 服务版本，不是协议版本
out.writeUTF(inv.getMethodName());
out.writeUTF(inv.getParameterTypesDesc());               // 只有描述串，不是 Class[] 列表
for (int i = 0; i < args.length; i++) {
    out.writeObject(callbackServiceCodec.encodeInvocationArgument(channel, inv, i));
}
out.writeAttachments(inv.getObjectAttachments());
```

关键是第五个 `writeUTF` ：写的是 `getParameterTypesDesc()` ，一个形如 `Ljava/lang/String;I` 的 JVM 描述串，**不是类名数组**。provider 端拿到描述串后要自己"反查"成 `Class<?>[]` —— `DecodeableRpcInvocation.drawPts` 会先按 `keyWithoutGroup(path, version)` 在 `FrameworkServiceRepository` 里找 `ProviderModel` ，再按描述串匹配 `MethodDescriptor` 。找不到就抛 `Service not found: ...` ，或者走泛化调用/echo 的特殊分支。这解释了一个经典现象：**接口签名一改（哪怕只是参数类型从 `int` 变 `Integer` ），报的错是 "Service not found"，而不是 "method not found"**——因为它是拿描述串去反查方法，反查不到就当成"这个服务不存在"，报错文案和真实原因错位了一格。

响应侧（ `encodeResponseData` ）先写一个字节标记，取值是 0-5：

```
RESPONSE_WITH_EXCEPTION = 0;              RESPONSE_VALUE = 1;              RESPONSE_NULL_VALUE = 2;
RESPONSE_WITH_EXCEPTION_WITH_ATTACHMENTS = 3;  RESPONSE_VALUE_WITH_ATTACHMENTS = 4;  RESPONSE_NULL_VALUE_WITH_ATTACHMENTS =
5;
```

`attach` 与否由 `Version.isSupportResponseAttachment(version)` 决定： `getIntVersion(version)` 必须落在 `[LOWEST_VERSION_FOR_RESPONSE_ATTACHMENT(2000200), HIGHEST_PROTOCOL_VERSION(2009900)]` 这个闭区间里才返回 true。也就是说 2.0.2 之前不认 attachments，写了会解析错位。这是 Dubbo 协议里少数几个"按对端版本降级"的地方。写 attachments 时还会顺手塞一个 `DUBBO_VERSION_KEY` ，让消费者知道服务端的协议版本。

顺序不能动。 `writeUTF` / `writeObject` / `writeAttachments` 是一条严格的流水线，解码侧 `DecodeableRpcInvocation.decode` 按同样的顺序 `readUTF` 、 `readUTF` 、 `readUTF` 、 `readUTF` 、 `readUTF` 、 `readObject` 、 `readAttachments` 。attachments 之所以必须放最后，是因为它的条目数只有读的时候才知道（先写一个 size 再逐条写 key/value），如果后面还有字段，就得先知道 size 才能跳过去——流式格式没有"长度前缀"给你跳。这也是为什么给 dubbo 协议加字段极其困难：任何新字段只能追加在末尾，而且老客户端读到末尾就停了，新字段对它是透明的，但新客户端读老服务端的数据就会缺字段。 `Version.isSupportResponseAttachment` 这种版本判断，就是被这个限制逼出来的。

还有一个特例值得记：泛化调用时 `getParameterTypesDesc()` 可能为空串，此时 `drawPts` 走 `RpcUtils.isGenericCall(desc, getMethodName())` 分支，把参数类型硬编码成 `{String.class, String[].class, Object[].class}` ； `$invoke$` 之外的 `$echo$` 则是 `{Object.class}` 。两者都不在 `ProviderModel` 里，所以必须在 `drawPts` 返回 `EMPTY_CLASS_ARRAY` 之后做这层兜底，否则泛化调用永远报 `Service not found` 。

11.2 decode 侧：为什么默认不在 IO 线程反序列化

回到关键代码的第一段。 `isDecodeDataInIoThread(channel)` 的默认值来自 `DEFAULT_DECODE_IN_IO_THREAD` ，在 `dubbo-rpc-dubbo` 的 `Constants` 里是 `false` 。也就是说**默认走路径 B**： `DubboCodec.decodeBody` 只做 `readMessageData(is)` 把 body 字节读出来，塞进 `DecodeableRpcInvocation` ，真正的 `decode()` 不在这里调。

那什么时候调？看 handler 链的顺序： `MultiMessageHandler → HeartbeatHandler → Dispatcher(...) → DecodeHandler → HeaderExchangeHandler` 。 `DecodeHandler.received` 第一件事就是：

```
// DecodeHandler.java:39-52
if (message instanceof Decodeable) { decode(message); }
if (message instanceof Request)    { decode(((Request) message).getData()); }
if (message instanceof Response)   { decode(((Response) message).getResult()); }
handler.received(channel, message);
```

它在 `Dispatcher` 的**内层**，所以这段代码跑在业务线程池里（默认 `all`）。设计意图很清楚：反序列化是 CPU 密集型，还要加载业务类（`Class.forName` + 首次链接），放在 IO 线程会让同一 EventLoop 上的所有连接排队等类加载。Dubbo 把"读字节"和"解释字节"拆成两步，就是为了让 IO 线程只做前者。

路径 A（`decode.in.io.thread=true`）只在一种情况下是必须的：开启线程池隔离（`EXECUTOR_MANAGEMENT_MODE_ISOLATION`）时。`isDecodeDataInIoThread` 里有这段注释：

```
// Why? because obtaining the isolated thread pool requires the serviceKey of the service,
// and this part must be decoded before it can be obtained (more see DubboExecutorSupport)
```

隔离池是按 serviceKey 分的，而 serviceKey 藏在 body 里（`path + version + group`），不先 decode 就选不出池子——鸡生蛋问题，只能强制在 IO 线程解。注意它读的 url 参数键是 `decode.in.io.thread`（`DubboCodec` 里的私有常量），而 `Constants.DECODE_IN_IO_THREAD_KEY` 是 `decode.in.io`，两个字符串不一样，配置时容易写错。

`DecodeableRpcInvocation.decode()` 里还有两道校验值得记住。一是 `checkPayload(keyWithoutGroup)`，按 provider 粒度再查一次 payload 上限；二是序列化白名单：

```
// DecodeableRpcInvocation.java:150-158
if (CHECK_SERIALIZATION) {                      // 系统属性 serialization.security.check，默认 true
    PermittedSerializationKeeper keeper = frameworkModel.getBeanFactory().getBean(PermittedSerializationKeeper.class);
    if (!keeper.checkSerializationPermitted(keyWithoutGroup, serializationType)) {
        throw new IOException("Unexpected serialization id:" + serializationType + " received from network, ...");
    }
}
```

也就是说，**同一个 provider 被第一次调用时用的序列化方式会被记住**，后续如果对端换了序列化 id 过来，直接拒绝。这条防线针对的是"攻击者构造一个用 java 原生序列化的请求打进来"。

响应侧对应的类是 `DecodeableRpcResult`，但它的分支判断更直接——`channel.getUrl().getParameter(DECODE_IN_IO_THREAD_KEY, DEFAULT_DECODE_IN_IO_THREAD)`，没走 `isDecodeDataInIoThread` 那套 channel 属性缓存。两处判断并不等价：请求侧会把结果缓存到 channel 上、还会被隔离模式强制改写，响应侧每次读 URL。读代码时别把它们当成同一套逻辑。另外响应解码要先 `getRequestData(channel, res, id)` 从 `DefaultFuture` 里反查原始 `Invocation`——没有方法签名就没法反序列化返回值，这一步是绕不开的。

顺带说清 `Decodeable` 这个接口的作用。它只有一个 `void decode()`，实现类是 `DecodeableRpcInvocation` 和 `DecodeableRpcResult`，两者都是"半个对象"——字段（channel、request、inputStream、serializationType）已经就位，但方法名、参数、attachments 全是空的，等 `decode()` 才填。`decode()` 里用 `volatile boolean hasDecoded` 加一层判断，所以重复调用是幂等的：`DecodeHandler` 解过一次之后，`HeaderExchangeHandler` 或业务代码再碰到它也不会重复反序列化。这个设计还带来一个副作用——**解码失败不会中断连接**，`decode()` 的 catch 里做的是 `request.setBroken(true); request.setData(e)`，然后 `HeaderExchangeHandler.handleRequest` 发现 `req.isBroken()` 就回一个 `status = BAD_REQUEST` 的响应。一个畸形请求只会得到一次 400，连接继续可用。

11.3 半包、粘包：三层循环各管一段

TCP 是字节流，一次 `channelRead` 可能给你半个包、一个包、或三个半包。Dubbo 用三层循环处理：

第一层，Netty 的 `ByteToMessageDecoder` 负责累积。它内部维护一个 cumulation buffer，字节不够就等下一次读，`decode` 返回的 `out` 为空就不往下传。这层不用 Dubbo 管。

第二层，`NettyCodecAdapter.InternalDecoder.decode`：

```
// NettyCodecAdapter.java:101-116
do {
    int saveReaderIndex = message.readerIndex();
    Object msg = codec.decode(channel, message);
    if (msg == Codec2.DecodeResult.NEED_MORE_INPUT) {
        message.readerIndex(saveReaderIndex);   // 回滚，等更多字节
        break;
    } else {
        if (saveReaderIndex == message.readerIndex()) {
            throw new IOException("Decode without read data.");   // 防死循环
        }
        if (msg != null) { out.add(msg); }
    }
} while (message.readable());
```

`saveReaderIndex == message.readerIndex()` 这个检查很重要：如果某个 codec 返回了非 `NEED_MORE_INPUT` 却没消费任何字节，`while (message.readable())` 会永远为真，直接把 IO 线程转死。Dubbo 在这里主动抛异常止损。

第三层，`DubboCountCodec.decode`（关键代码第二段）。它自己也循环，因为 `ExchangeCodec.decode` 一次只解一帧；一次读到三个完整帧时，它把三个都解出来装进 `MultiMessage`。只有一条时返回单条，避免无谓包装。

"半包"的判定在 `ExchangeCodec.decode` 里，两个 `return DecodeResult.NEED_MORE_INPUT`：

```
// ExchangeCodec.java:113-131
if (readable < HEADER_LENGTH) {
    return DecodeResult.NEED_MORE_INPUT;      // 连头都不够 16 字节
}
int len = Bytes.bytes2int(header, 12);
...
int tt = len + HEADER_LENGTH;
if (readable < tt) {
    return DecodeResult.NEED_MORE_INPUT;      // 头够了，body 没到齐
}
```

注意 `readable` 是**调用前的总可读字节**，`header` 是 `new byte[Math.min(readable, HEADER_LENGTH)]` 读出来的前 16 字节。所以第一个判断用的是"读头之前"的快照，第二个用的是同一个快照加上解析出来的 len——两个判断配合，才能正确区分"头不全"和"体不全"。

magic 不对时会走 `super.decode(...)`（`TelnetCodec`），它会从字节流里向前找 `0xdabb`，找到就 `buffer.readerIndex(buffer.readerIndex() - header.length + i)` 把读指针挪过去，找不到就当 telnet 命令处理。这就是为什么你能 `telnet 127.0.0.1 20880` 然后敲 `ls` 看到服务列表——同一个端口同时兼容两种协议。

`DubboCountCodec` 里还有个容易被忽略的副作用：`logMessageLength(obj, buffer.readerIndex() - save)` 会把这一帧的字节数写进消息里——请求塞进 `RpcInvocation` 的 `INPUT_KEY` attachment，响应塞进 `AppResponse` 的 `OUTPUT_KEY` attachment。这不是给业务用的，是给 metrics 和 qos 用的：一次调用的请求字节数、响应字节数、以及 `Content-Length` 都能从这里拿到。它只在 `bytes > 0` 且类型匹配时才写，且整段包在 `try { } catch (Throwable e) { /* ignore */ }` 里——统计信息不能影响主流程。

超长包的处理分两侧。响应侧在 `decode` 里就拦：`finishRespWhenOverPayload(channel, len, header)` 如果发现 `len > payload`，直接构造一个 `status = CLIENT_ERROR` 的 `Response` 返回，**不读 body**（对应 issue 7021，避免为了一个注定失败的响应把几 MB 垃圾读进内存）。请求侧则在 `encodeRequest` 的 `checkPayload(channel, req.getPayload(), len)` 和 `DecodeableRpcInvocation.checkPayload` 里抛 `ExceedPayloadLimitException`，默认上限 `DEFAULT_PAYLOAD = 8 * 1024 * 1024`。日志里 `Skip input stream 27` 来自 `decode` 的 finally 块——`decodeBody` 没把 `ChannelBufferInputStream` 读干净时，剩下的字节会被 `StreamUtils.skipUnusedStream` 丢掉，同时 warn 一下。看到这个数字，通常意味着编码侧写的长度和实际写的字节数不一致。

11.4 序列化 SPI：id 写进头里，选择权在 URL

`CodecSupport.getSerialization(URL)` 只有一行：

```
// CodecSupport.java:87-91
return url.getOrDefaultFrameworkModel()
        .getExtensionLoader(Serialization.class)
        .getExtension(UrlUtils.serializationOrDefault(url));
```

`serializationOrDefault` 的优先级是：`prefer.serialization`（逗号分隔的候选列表，取**列表第一个元素**）→ `serialization`（`Constants.SERIALIZATION_KEY = "serialization"`）→ `DefaultSerializationSelector.getDefaultRemotingSerialization()`，而后者硬编码 `"hessian2"`（可被系统属性/环境变量覆盖）。注意 `serializationOrDefault` 是 `allSerializations(url).stream().findFirst()`，**不做存在性检查**—— `prefer.serialization=protobuf` 而本端没装 protobuf 时，会在 `getExtension` 处直接抛，不会静默回退。真正做存在性检查的是 `UrlUtils.serializationId(url)`：它遍历 prefer 列表，逐个 `CodecSupport.getIDByName(preferSerial)`，取第一个非 null 的。两个方法名字很像，行为不同。

真正的机制在 `encodeRequest` 那一行 `header[2] = FLAG_REQUEST | serialization.getContentTypeId()`：**选中的序列化 id 被写进了字节流**。解码侧 `decodeBody` 只看 `proto = flag & SERIALIZATION_MASK`，然后 `CodecSupport.getSerialization(Byte id)` 反查。找不到就抛 `Unrecognized serialize type from consumer: 23`。

这带来两个结论。第一，**两端可以用不同的默认序列化**，只要对端发来的 id 本端有对应扩展。第二，`DubboCodec` 覆写了 `getSerialization`，让响应复用请求的序列化方式：

```
// DubboCodec.java:339-347
protected Serialization getSerialization(Channel channel, Response res) {
    if (res instanceof HeartBeatResponse) {
        return CodecSupport.getSerializationById(((HeartBeatResponse) res).getProto());
    }
    if (!(res.getResult() instanceof AppResponse)) { return super.getSerialization(channel, res); }
    return DubboCodecSupport.getResponseSerialization(channel.getUrl(), (AppResponse) res.getResult());
}
```

`getResponseSerialization` 从 `AppResponse` 的 `INVOCATION_KEY` 属性里取 `Invocation`，再取 `SERIALIZATION_ID_KEY`——这个 key 是 `DecodeableRpcInvocation.decode()` 里 `this.put(SERIALIZATION_ID_KEY, serializationType)` 塞进去的。**所以响应一定用请求带来的序列化**，哪怕服务端配的是另一种。这个设计保证了"消费者用 fastjson2 发、服务端默认 hessian2"时响应不会解析错。

Hessian2 为什么是默认：它是自描述的二进制格式，字段增删不影响反序列化（缺字段给默认值，多字段忽略），跨语言实现成熟，对 Java 的继承/泛型支持好。代价是安全性——反序列化时能实例化任意类，历史上出过大量 gadget chain。Dubbo 的对策在 `Hessian2FactoryManager`：

```
// Hessian2FactoryManager.java:96-105
serializerFactory.getClassFactory().setWhitelist(true);
String allowPattern = SystemPropertyConfigUtils.getSystemProperty(DUBBO_HESSIAN_ALLOW);
for (String pattern : allowPattern.split(";")) {
    serializerFactory.getClassFactory().allow(pattern);
    serializeSecurityManager.addToAlwaysAllowed(pattern);
}
serializeSecurityManager.setCheckStatus(SerializeCheckStatus.STRICT);
```

对应系统属性 `dubbo.application.hessian2.whitelist=true` 加 `dubbo.application.hessian2.allow=com.foo.*`（注意键里是 `hessian2`，不是 `hessian`；另有一个 `dubbo.application.hessian2.deny` 用于黑名单模式）。默认（非白名单模式）会 `getClassFactory().allow("org.apache.dubbo.*")`，其它类走 `DefaultSerializeClassChecker` 的黑名单检查。**生产环境如果暴露公网，白名单是必配的**，不是可选项。

FastJson2（id 23）走的是 `JSONB` 而不是文本 JSON—— `FastJson2Serialization` 的静态块里检查的是 `com.alibaba.fastjson2.JSONB`，`getContentType()` 返回 `text/jsonb`。JSONB 是二进制 JSON，比文本紧凑，跨语言友好（JSON 语义谁都认），但类型信息比 Hessian2 弱：声明成 `Object` 的字段拿回来往往是 `JSONObject` / `JSONArray`，泛型嵌套容易丢类型，跨版本改字段名也没有 Hessian2 那种"缺字段给默认值"的宽容。它同样有安全检查，构造 `FastJson2ObjectInput` 时要传 `Fastjson2SecurityManager`，和 Hessian2 的白名单是两套机制。

protobuf 的情况要说清楚：`dubbo-serialization-api` 的 `Constants` 里还留着 `PROTOBUF_SERIALIZATION_ID = 22` 和 `PROTOBUF_JSON_SERIALIZATION_ID = 21`，但**这个 3.3 代码树的 `dubbo-serialization` 下只有 hessian2 和 fastjson2 两个模块注册了 `Serialization` SPI**（`META-INF/dubbo/internal/org.apache.dubbo.common.serialize.Serialization` 里只有 `hessian2=` 和 `fastjson2=`，外加一个 `wrapper=` 装饰器）。protobuf 的序列化能力在 3.x 里主要跟着 Triple 走（需要 IDL 和生成代码），不是 dubbo 协议下随手 `serialization=protobuf` 就能用的东西。看到 id 22 就以为"配一下就能用"，是常见的误判。

至于"3.x 默认推 Triple"，要说得准确一点：`CommonConstants.DEFAULT_PROTOCOL` 仍然是 `"dubbo"`，默认序列化仍然是 `hessian2`——源码里没有任何地方把默认协议改成 triple。官方推荐 triple 的理由在协议本身：dubbo 协议的 header 是自造的（`0xdabb` + 5 位序列化 id + 4 字节长度），只有 Java 生态实现，网关、Service Mesh、多语言客户端都认不出来；payload 上限 8MB 写死；没有流式语义；序列化 id 只剩 31 个坑位。Triple 走 HTTP/2，能被标准基础设施识别，支持 streaming，序列化可以选 protobuf。所以"推 Triple"是推**新项目选 triple**，不是"dubbo 协议被替换了"。

容易误解的地方：

1. **"Dubbo 协议是 12 字节头"**。源码里 `HEADER_LENGTH = 16`。12 是 **body 长度字段的偏移量**（`Bytes.bytes2int(header, 12)`），老文章说的"12 字节"多半指"前 12 字节（magic 2 + flag 1 + status 1 + id 8）固定，第 13-16 字节是长度"。按 12 去算 `len + 12` 会稳定地把每帧少算 4 字节，然后一路 `NEED_MORE_INPUT`。
2. **"requestId 是 UUID 或字符串"**。是 `long`，`AtomicLong.getAndIncrement()`，种子是 `ThreadLocalRandom.current().nextLong()`，可选 `SecureRandom`。溢出成负数也是合法 id（`Request.newId` 的注释明说 `the negative can be used as ID`）。
3. **"两端序列化方式必须一致"**。不需要。id 在 header 里，各端按 id 反序列化。但**两端都必须加载对应扩展**，否则就是 `Unrecognized serialize type from consumer`；而且开了 `serialization.security.check` 后，同一个 provider 只认第一次见到的那个 id。
4. **"`decode.in.io.thread=true` 性能更好"**。默认是 `false` 且有原因：开了等于让 IO 线程做反序列化和类加载，首次调用或大对象会阻塞同一 EventLoop 上的所有连接。只有在开线程池隔离时才必须开，而且那时是被强制覆盖的，配了也没用。
5. **"`DubboCountCodec` 负责处理粘包"**。它只负责"把已经完整的多条消息打包成 `MultiMessage`"，真正"等够字节"的是 `ExchangeCodec` 的两个 `NEED_MORE_INPUT` 判断加上 Netty 的 cumulation buffer。少了任何一层都不行：没有 `NEED_MORE_INPUT` 就不知道要等，没有 `DubboCountCodec` 就会把一次读到的多帧丢掉后面的。

记住这一句：头只描述"怎么解 body"，不描述"body 是什么"——所以 Dubbo 协议简单到能手写，也因此永远长不出跨语言、流式和标准网关支持，那是 Triple 存在的理由。

结语：一个 RPC 框架到底在解决什么

11 章读完，把书合上，其实只需要记住三句话。

一、三句话

第一句：RPC 框架的全部工作，是把"调一个方法"翻译成"发一段字节"，再翻译回来。 代理对象挡在前面让你以为在调本地方法，序列化层把参数变成字节，网络层把字节送过去，提供端反过来做一遍。所有接口都是为这个翻译过程分工位。

第二句：Dubbo 用 URL 当配置总线，用 SPI 当装配总线。 一个服务从注解到端口，中间所有决策（用哪个协议、哪个序列化、超时多少、重试几次、走哪条路由）都塞进 URL 的参数里；每一层拿到 URL，再去问 `ExtensionLoader` "这个名字该取哪个实现"。所以加一种实现不用改调用方，改配置就行——**这也是为什么读 Dubbo 必须先看 SPI**。

第三句：可靠性的代价全是运行时成本。 重试要多发几次请求，负载均衡要统计活跃数和响应时间，路由要维护地址列表并逐条筛选，连接要心跳和重连，`DefaultFuture` 要为每次请求留一个 Future 并管超时。设计这些机制时真正在权衡的不是"能不能做到"，而是"为选错付出多少代价"。

二、3.x 相比 2.x 的关键变化

网上多数源码分析写的是 2.7。下面这些是 3.3 里确实变了的，套旧结论会错：

变化清单

方面	2.x	3.3
领域模型	静态的 `ApplicationConfig` 等配置对象	`FrameworkModel` / `ApplicationModel` / `ModuleModel` 三层，支持多应用部署
导出时机	`ServiceBean` 在 Spring 事件里直接 `export()`	`DefaultModuleDeployer` 统一按模块启动/发布
Filter 链	`ProtocolFilterWrapper.buildInvokerChain` 一条链	`FilterChainBuilder` 拼装，节点为 `CopyOfFilterChainNode`，外层 `CallbackRegistrationInvoker`
监听器	`ListenableFilter`	已 `@Deprecated`，改为 `implements Filter, Filter.Listener`
消费端链	一条	`ClusterFilter`（选路前）+ 实例级 `Filter`（选路后）两条
服务发现	接口级（一个接口一堆地址）	应用级（应用 → 实例 → 接口元数据），地址量下降一个数量级
主推协议	dubbo	Triple（基于 gRPC / HTTP2），dubbo 协议仍完整保留
Spring Boot	`spring.factories`	`AutoConfiguration.imports`，自动配置类收敛为两个

三、附录：核心扩展点速查

想加自定义实现，先查这张表：接口名、默认值、配置文件放哪。

协议与代理

`@SPI` 接口	默认值	作用	配置文件
`Protocol`	`dubbo`	导出 / 引用，把 Invoker 变成端口或客户端	`META-INF/dubbo/org.apache.dubbo.rpc.Protocol`
`ProxyFactory`	`javassist`	生成代理对象 / 把实现类包成 Invoker	`META-INF/dubbo/org.apache.dubbo.rpc.ProxyFactory`
`Filter`	—（`@Activate`）	调用链上的横切逻辑	`META-INF/dubbo/org.apache.dubbo.rpc.Filter`
`Serialization`	`hessian2`	请求体编解码	`META-INF/dubbo/org.apache.dubbo.common.serialize.Serialization`

集群与选路

@SPI 接口	默认值	作用	配置文件
`Cluster`	`failover`	把 Directory 包成具备容错能力的 Invoker	`META-INF/dubbo/org.apache.dubbo.rpc.cluster.Cluster`
`LoadBalance`	`random`	从候选 Invoker 里选一个	`META-INF/dubbo/org.apache.dubbo.rpc.cluster.LoadBalance`
`RouterFactory`	—	按条件/标签筛地址，组成 `RouterChain`	`META-INF/dubbo/org.apache.dubbo.rpc.cluster.RouterFactory`
`RegistryFactory`	—	接注册中心，生成 `Registry`	`META-INF/dubbo/org.apache.dubbo.registry.RegistryFactory`

网络与线程

@SPI 接口	默认值	作用	配置文件
`Transporter`	`netty`	建立 Client / Server	`META-INF/dubbo/org.apache.dubbo.remoting.Transporter`
`Dispatcher`	`all`	决定事件派发到哪个线程池	`META-INF/dubbo/org.apache.dubbo.remoting.Dispatcher`
`Codec2`	`dubbo`	粘包半包处理与协议编解码	`META-INF/dubbo/org.apache.dubbo.remoting.Codec2`
`ThreadPool`	`fixed`	业务线程池实现	`META-INF/dubbo/org.apache.dubbo.common.threadpool.ThreadPool`

> 目录优先级：`META-INF/dubbo/internal/`（框架内置）→ `META-INF/dubbo/`（用户扩展）→ `META-INF/services/`（兼容 JDK SPI）。同名以用户扩展为准，这也是"为什么我加的实现没生效"的第一个排查点。

四、附录：高频 URL 参数速查

参数	生效层	语义 / 坑
`retries`	集群层	Failover 的重试次数，`+1` 后才是总调用次数；只对幂等操作安全
`timeout`	交换层	`DefaultFuture` 的超时，服务端慢和客户端超时是两件事
`loadbalance`	集群层	`random` / `roundrobin` / `leastactive` / `shortestresponse` / `consistenthash`
`check`	引用层	`false` 把启动期失败推迟到调用期，故障被藏得更深
`cluster`	集群层	`failover` / `failfast` / `failsafe` / `failback` / `forking` / `broadcast`
`serialization`	协议层	编解码方式，两端必须一致
`delay`	导出层	延迟导出，等依赖就绪
`warmup`	负载均衡	预热期内按比例降权，新机器不被瞬间打满
`tag` / `force`	路由层	标签路由；`force=true` 时无匹配地址会直接报错而不是回落
`broadcast.fail.percent`	集群层	Broadcast 允许失败的百分比（3.3 里没有 failover/failsafe 开关）
`dispatcher`	网络层	`all` / `direct` / `message` / `execution` / `connection`

五、怎么自己读一个 RPC 框架

这本书的读法可以迁移到任何框架上，四步：

步骤	做什么	判断标准
1	找到业务代码能直接调到的入口	能写出一行调用它的代码

步骤	做什么	判断标准
2	沿一条最普通的路径走到底，遇到分支先记下不跟	能画出"一次调用经过哪些类"
3	找出配置总线和装配总线	能回答"改一个配置，最终落到哪个类的哪个字段"
4	故意关掉一层，推演会出什么错	能说出"没有它会怎样"

第 4 步最关键。**能背出 `refresh()` 十二步或 `invoke()` 四步，不算懂；能说清"不注册 `BeanPostProcessor` 会怎样"、"没有 `DefaultFuture` 响应会还给谁"，才算懂。** 这本书每章的"容易误解的地方"，本质上都是第 4 步的练习题。

验证也别靠推理：在怀疑的方法上打断点看真实调用栈、用 `jstack` 看线程卡在哪、写个 30 行的最小复现把机制抽出来跑一遍。一次实验胜过十次推理。

最后一句

Dubbo 最值得学的不是那 11 条调用链，而是它把"变化"关进扩展点、把"决策"塞进 URL 的做法——**每一层只做一件事，层与层之间只通过接口和 URL 说话**。

理解了这一点，再看 Spring Cloud、gRPC 甚至任何一个中间件，你会发现结构都差不多：一个入口、一条链、几个扩展点、一份配置总线。框架的名字会换，这套分工位的方法不会。

具体的方法名会忘，"为什么这里要留一个接口"不会。